手把手实现 nanoGPT

用 Go 与 PyTorch 从零训练一个小 GPT

第 2 版 · 双实现 · 全部数字本机实测 · 附完整源码

数据与分词 / 梯度全推导 / Go 手写反向传播 / PyTorch autograd

跨实现逐元素对齐 / 现代组件实验 / KV cache 与 Go 推理服务

LLM 基础系列 · 实战册

2026 年 10 月

目录

前言 6

本书定位 6

第 2 版改了什么 6

怎么读、怎么做 6

第 0 章 环境、仓库与语料 8

0.1 一句话说清 8

0.2 环境 8

0.3 仓库结构 8

0.4 语料 9

0.5 一键命令与预期输出 9

0.6 符号表 9

第 1 章 目标与蓝图 11

1.1 目标：几百行代码，从零训练一个小 GPT 11

1.2 模型规格 11

1.3 架构蓝图 12

1.4 阅读与动手路径 14

第 2 章 数据与分词 15

2.1 一句话说清 15

2.2 为什么用字符级分词 15

2.3 词表构建：rune 不是 byte 15

2.4 滑动窗口：自回归数据的自监督 16

2.5 数据在两套技术栈里的形态 17

2.6 动手实验 17

2.7 小结 18

第 3 章 前向与反向：完整梯度推导 19

3.1 一句话说清 19

3.2 前向全链路 19

3.3 矩阵求导的两条法则 20

3.4 交叉熵：反向的起点 20

3.5 权重共享的输出层 20

3.6 LayerNorm 的反向 21

3.7 残差连接 . . . 21
3.8 FFN 的反向 . . . 22
3.9 注意力的反向 . . . 22
3.10 嵌入的反向：scatter-add . . . 23
3.11 一条完整的反向链 . . . 23
3.12 微型数值走读 . . . 24
3.13 梯度检验：手写反向的保命符 . . . 25
3.14 Adam：梯度之后的一步 . . . 25
3.15 算子卡片 . . . 27
3.16 小结 . . . 27

第 4 章 Go 实现：8 个里程碑 . . . 28

4.1 一句话说清 . . . 28
4.2 M1：矩阵工具 . . . 28
4.3 M2：分词与取 batch . . . 30
4.4 M3：算子的前向与反向 . . . 30
4.5 M4：前向与因果性 . . . 33
4.6 M5：模型定义与参数注册表 . . . 35
4.7 M6：反向传播与 Adam . . . 36
4.8 M7：训练循环 . . . 39
4.9 M8：采样与生成 . . . 40
4.10 动手实验 . . . 42
4.11 小结 . . . 42

第 5 章 PyTorch 实现 . . . 43

5.1 一句话说清 . . . 43
5.2 数据 . . . 43
5.3 模型 . . . 43
5.4 初始化 . . . 45
5.5 训练、评估与生成 . . . 45
5.6 实测输出 . . . 46
5.7 设备与性能 . . . 46
5.8 两种技术栈的分工 . . . 47
5.9 动手实验 . . . 47
5.10 小结 . . . 47

第 6 章 跨实现对齐与排错 . . . 48

6.1 一句话说清 . . . 48

6.2 跨实现逐元素对齐 . . . 48
6.3 曲线对照为什么只能当辅助证据 . . . 49
6.4 坑一：PyTorch 的默认初始化 . . . 50
6.5 坑二：残差处多加一份直连梯度 . . . 52
6.6 坑三：去掉因果掩码 . . . 53
6.7 手写或复现模型的排错清单 . . . 53
6.8 小结 . . . 54

第 7 章 进阶实验：从 GPT-2 到现代架构 . . . 55
7.1 一句话说清 . . . 55
7.2 新语料、验证集与基线 . . . 55
7.3 训练治理 . . . 56
7.4 现代组件 . . . 57
7.5 模型变大时 . . . 59
7.6 计算量与显存的估算 . . . 60
7.7 小结 . . . 61

第 8 章 推理与性能：Go 的主场 . . . 62
8.1 一句话说清 . . . 62
8.2 KV cache . . . 62
8.3 单机数据并行 . . . 65
8.4 PyTorch 训练，Go 推理 . . . 66
8.5 一个最小的 HTTP 推理服务 . . . 67
8.6 小结 . . . 68

第 9 章 从玩具到生产 . . . 69
9.1 一句话说清 . . . 69
9.2 规模对照 . . . 69
9.3 七道工程鸿沟 . . . 69
9.4 后台工程师的优势在哪里 . . . 70
9.5 推荐的后续路线 . . . 70
9.6 全书小结 . . . 71

附录 A 完整源码 . . . 72

附录 B 语料全文 . . . 111

附录 C 速查与排错 FAQ . . . 113
C.1 反向传播速查 . . . 113

C.2 排错 FAQ 113

附录 D 第 1 版勘误对照 115

D.1 公式与概念 115

D.2 代码 115

D.3 数据与表述 116

前言

本书定位

这是 LLM 基础系列的实战册。前几册讲了原理：神经元、反向传播、嵌入、注意力、token、训练与推理工程。本册把它们组装起来，从零写一个 GPT，用几百行代码训练它，再一步步验证它是对的。

全书用两套技术栈各实现一遍：

- **Go 版**：只用标准库。矩阵乘、前向、反向、Adam、采样、KV cache 全部手写，每个梯度都能单独打印、单独检验。
- **PyTorch 版**：主流训练栈。同一个模型用 autograd 实现，体会框架替你做了什么、没替你做什么。

两版共用同一份语料（20 首唐诗，628 个字符）和同一个模型（26,912 个参数）。读者画像：多年经验的后台工程师（Go 背景），正在转向 AI infra。

第 2 版改了什么

第 1 版的骨架是对的，但有三类问题：几处公式和代码错误、读者无法照书跑通、核心论证前后矛盾。第 2 版逐一处理：

1. **所有错误已修正，并配了能复现错误的测试。** 例如 LayerNorm 反向公式、注意力缩放系数重复、Adam 偏差修正的方向，正文给出正确推导，测试代码里保留了错误版本作为反例。完整清单见附录 D。
2. **代码完整可运行。** 附录 A 收录全部源码；正文按 8 个里程碑组织，每个里程碑都有单元测试和预期输出。本书所有数字都来自这份代码在本机的实测，命令和输出一一对应。
3. **验证方法升级。** 第 1 版把“两条 loss 曲线几乎重合”当作最强证据，但本书第 6 章会用实验说明：残差梯度写错的模型，loss 曲线和正确模型几乎一样。第 2 版改用“同一份权重、同一个 batch，逐元素比对 logits、梯度和一步 Adam 后的参数”，两套实现在 float64 下误差为 10^{-15} 量级。
4. **深度与广度扩展。** 新增微型数值走读、梯度检验的误差分析、初始化的方差推导、Adam 如何掩盖梯度 bug、现代组件（RMSNorm / RoPE / SwiGLU / GQA）对照实验、KV cache、Go 并行训练、PyTorch 训练后由 Go 加载推理并对外提供 HTTP 服务。

怎么读、怎么做

你的目标	建议路径
搞懂反向传播的每一步	第 3 章（推导 + 数值走读）→ 第 4 章 M3 – M6
照书敲出一个能训练的 GPT	第 0 章搭环境 → 第 2 章 → 第 4 章按里程碑推进，每步跑测试
学会验证“实现是对的”	第 3.8 节梯度检验 → 第 6 章
了解现代大模型改了哪些组件	第 7 章
关心推理与工程性能	第 8 章

每章大体按“一句话说清 → 原理与推导 → 实现 → 实测 → 动手实验 → 小结”展开。动手实验统一写成实验卡片：目的、改动、命令、预期结果、观察要点。预期结果尽量给出具体数值或区间，方便你判断自己做对了没有。

关于实测数字

本书数字在以下环境测得：Apple M4 Pro（12 核，arm64）、macOS、Go 1.26.0、Python 3.13.12、PyTorch 2.14.1。训练时 PyTorch 设为单线程。Go 版在同一 CPU 架构上、相同种子下可以逐位复现（`math/rand` 给定种子的序列是固定的）；换到 amd64 时，编译器是否把乘加融合成 FMA 指令会影响浮点末位，训练轨迹会有小幅差异，但量级和结论不变。PyTorch 版同理，耗时则随机器变化。

第 0 章 环境、仓库与语料

0.1 一句话说清

本章把“能跑起来”这件事一次说清：装什么、代码放在哪、语料长什么样、每个命令跑完应该看到什么。后面所有章节的数字都能用这里的命令复现。

0.2 环境

组件	最低版本	本书实测版本	用途
Go	1.22	1.26.0	Go 版全部代码，只用标准库
Python	3.10	3.13.12	PyTorch 版与实验脚本
PyTorch	2.2	2.14.1	CPU 即可；第 7 章在 Apple GPU（MPS）上更快
matplotlib	3.8	3.11.2	只用于画图，可选

```
go version                          # go1.22 以上
python3 -m venv .venv && source .venv/bin/activate
pip install -r py/requirements.txt
```

0.3 仓库结构

```
nanogpt/
├── corpus.txt              20 首唐诗，628 字符（全文见附录 B）
├── Makefile                一键命令
├── go/                     Go 版（package main，单目录多文件）
│   ├── mat.go              M1  矩阵与工具函数
│   ├── ops.go              M3  LayerNorm / Softmax / ReLU / 交叉熵（含反向）
│   ├── model.go            M5  配置、参数注册表、初始化
│   ├── forward.go          M4  前向（带缓存）
│   ├── backward.go         M6  反向
│   ├── train.go            M2/M6/M7  分词、取 batch、Adam、训练步、并行
│   ├── generate.go         M8  采样、生成、KV cache
│   ├── gradcheck.go        数值梯度检验
│   ├── weights.go          权重读写、gen / serve 命令
│   ├── main.go             命令行入口：train / gradcheck / align / walk / kvbench / speed
│   └── *_test.go           每个里程碑的单元测试
├── py/
│   ├── nanogpt.py          PyTorch 版（第 5 章）
│   ├── align.py            跨实现逐元素对齐（第 6 章）
│   ├── experiments.py      初始化、激活函数等对照实验
│   ├── export.py           PyTorch 训练 → 导出 Go 可加载的权重（第 8 章）
│   ├── modern.py           现代组件与训练治理实验（第 7 章）
│   └── figures.py          生成书中插图
└── out/                    所有实验的原始输出（CSV / JSON / 文本）
```

Go 版没有拆成多个包，所有文件都在 `package main` 里。这样做的代价是包边界不清晰，好处是读者可以按里程碑一个文件一个文件地敲，每敲完一个就能 `go test` 验证。

0.4 语料

语料是 20 首唐诗，每首一行，共 628 个字符（含 20 个换行符和全部标点），去重后 289 个不同字符，所以词表大小 $V = 289$。文件以换行结尾，两套实现都按 UTF-8 读取整个文件，不做任何清洗。

统计项	数值	说明
字符数	628	含标点与换行；按 rune 计，不是字节数
词表大小 V	289	按 Unicode 码点排序后编号，两套实现完全一致
训练窗口数	612	$N - T = 628 - 16$，见 2.4 节
SHA-256	`c755662e…ecbe1d53`	完整值见附录 B

> **字符数为什么和第 1 版不同**
>
> 第 1 版使用的语料没有随书给出，字符数（579）和词表（273）无法复现。第 2 版重新整理了 20 首诗并随书附上全文，所有数字随之更新：参数量从 26,400 变为 26,912（多出的 512 来自词表多了 16 个字符，$16 \times 32 = 512$），初始 loss 理论值从 $\ln 273 \approx 5.61$ 变为 $\ln 289 \approx 5.67$。

0.5 一键命令与预期输出

命令	做什么	预期结果（本机实测）
`make test`	18 个单元测试	全部 PASS，约 1.5 秒
`make train-go`	Go 版训练 600 步	step 1 loss 5.6868，600 步后全窗口平均 loss 0.4701，耗时约 1.4 秒
`make train-py`	PyTorch 版训练 600 步	step 1 loss 5.6863，全窗口平均 loss 0.4137，耗时约 1.7 秒
`make gradcheck`	全模型梯度检验	正确实现最大相对误差约 10^{-5}；残差 bug 版为 1.0
`make align`	跨实现逐元素对齐	float64 下 logits 最大误差 2.2×10^{-15}
`make walk`	微型数值走读	打印 d=4、T=3、V=5 模型的全部中间量
`make kv`	KV cache 测速	生成 508 个 token 时加速约 227 倍，输出与无 cache 完全一致
`make export gen`	PyTorch 训练 3000 步，Go 加载后生成	两侧全窗口 loss 都是 0.164963

0.6 符号表

全书统一使用下面的符号。第 1 版里 $\hat{X}$ 同时表示“LN2 的输出”和“LN 内部的标准化值”，第 2 版把它们分开了。

符号	含义	形状
V, d, h, d_h, d_{ff}, L	词表、模型维度、头数、头维 d/h、FFN 隐藏维、层数、窗口长	289, 32, 2, 16, 64, 2, 16
E_{tok}, E_{pos}	token 嵌入、位置嵌入（代码中 `TokE`、`PosE`）	$V \times d$, $T \times d$
$X^{(\ell)}$	第 ℓ 层的输入（残差流），$X^{(0)}$ 为嵌入之和	$T \times d$

符号	含义	形状
$\tilde{X}$	LN1 的输出，Q/K/V 的共同输入（代码中 `Xln1`）	$T \times d$
X'	注意力残差之后的残差流（代码中 `Xres1`）	$T \times d$
$\bar{X}$	LN2 的输出，FFN 的输入（代码中 `Xln2`）	$T \times d$
$\hat{x}$，r	LayerNorm 内部的标准化值与 $1/\sqrt{\sigma^2+\varepsilon}$	每行 d 维 / 每行一个标量
S，P，C	某个头的注意力分数、softmax 概率、输出	$T \times T$，$T \times T$，$T \times d_h$
G	$\partial L/\partial$ logits	$T \times V$
dZ	任意张量 Z 的梯度 $\partial L/\partial Z$，与 Z 同形状	—

第 1 章 目标与蓝图

1.1 目标：几百行代码，从零训练一个小 GPT

Karpathy 的 nanoGPT 是最著名的 GPT 教学项目：用几百行代码实现 GPT-2 的完整架构并真正训练它。本书在两套技术栈下各实现一遍，并且多做一件事：**证明两套实现在数学上完全等价**。

实测结果（同一语料、同一模型规模、各训练 600 步）：

	Go 版	PyTorch 版
step 1 的 loss	5.6868	5.6863
step 600 的 loss（单个 batch）	0.3702	0.3122
600 步后全部 612 个窗口的平均 loss	0.4701	0.4137
5 个随机种子的平均（全窗口 loss）	0.429	0.426
训练耗时	1.40 秒	1.65 秒
同一份权重、同一 batch 的 logits	作为基准	与 Go 的最大差 2.2×10^{-15}（float64）

最后一行才是“两边都写对了”的证据：把 Go 训练到一半的权重原样搬进 PyTorch，喂同一个 batch，两边算出的 logits、loss、全部 36 个参数矩阵的梯度、走一步 Adam 后的参数，在 float64 下的差都在 10^{-13} 以下，也就是浮点舍入误差的量级。

前几行的差异来自随机性：两边的随机数生成器不同，初始权重和每步抽到的窗口都不同。单个 batch 只有 4 个窗口，单步 loss 的波动远大于两套实现之间的差别，所以 step 600 的单步 loss 不能用来比较对错。第 6 章会专门讨论这一点。

600 步之后，模型能复写训练集里的诗句，也会拼出训练集里没有的组合，例如“白日依山出，黄河在，城春眠不教胡马度阴山”。字符级小模型学到的是局部的字与字之间的统计规律，它在拼接，不是在查表背诵。训练到 3000 步，全窗口 loss 降到 0.17，模型基本能逐字背出整首诗。

1.2 模型规格

项	取值	说明
语料	20 首唐诗，628 字符	字符级分词，词表 $V = 289$
d（d_model）	32	嵌入与隐藏维度
h（头数）	2	每头 $d_h = 16$
L（层数）	2	Transformer Block 数
d_{ff}	64	2 倍 d；GPT-2 的惯例是 4 倍
T（block_size）	16	上下文窗口
输出头	与 E_{tok} 共享权重	weight tying，省下 $289 \times 32 = 9{,}248$ 个参数
初始化	权重 $\mathcal{N}(0, 0.02^2)$，偏置 0，LN 的 $\gamma = 1, \beta = 0$	nanoGPT 惯例，第 6.4 节解释为什么重要

项	取值	说明
优化器	Adam，$\eta = 3 \times 10^{-3}$，$\beta_1 = 0.9, \beta_2 = 0.999$	每步 4 个窗口，共 600 步

参数量逐项手算：

组件	计算	参数量
E_{tok}	289×32	9,248
E_{pos}	16×32	512
每层 QKV 投影	$3 \times (32 \times 32 + 32)$	3,168
每层输出投影 W_o	$32 \times 32 + 32$	1,056
每层 FFN	$(32 \times 64 + 64) + (64 \times 32 + 32)$	4,192
每层两个 LN	$2 \times (32 + 32)$	128
每层合计		8,544
两层	$2 \times 8{,}544$	17,088
最终 LN	$32 + 32$	64
合计	$9{,}248 + 512 + 17{,}088 + 64$	**26,912**

程序打印的参数量与手算一致，这是第一个零成本的自检（`TestParamCount`）。输出投影与 E_{tok} 共享，不另占参数。

1.3 架构蓝图

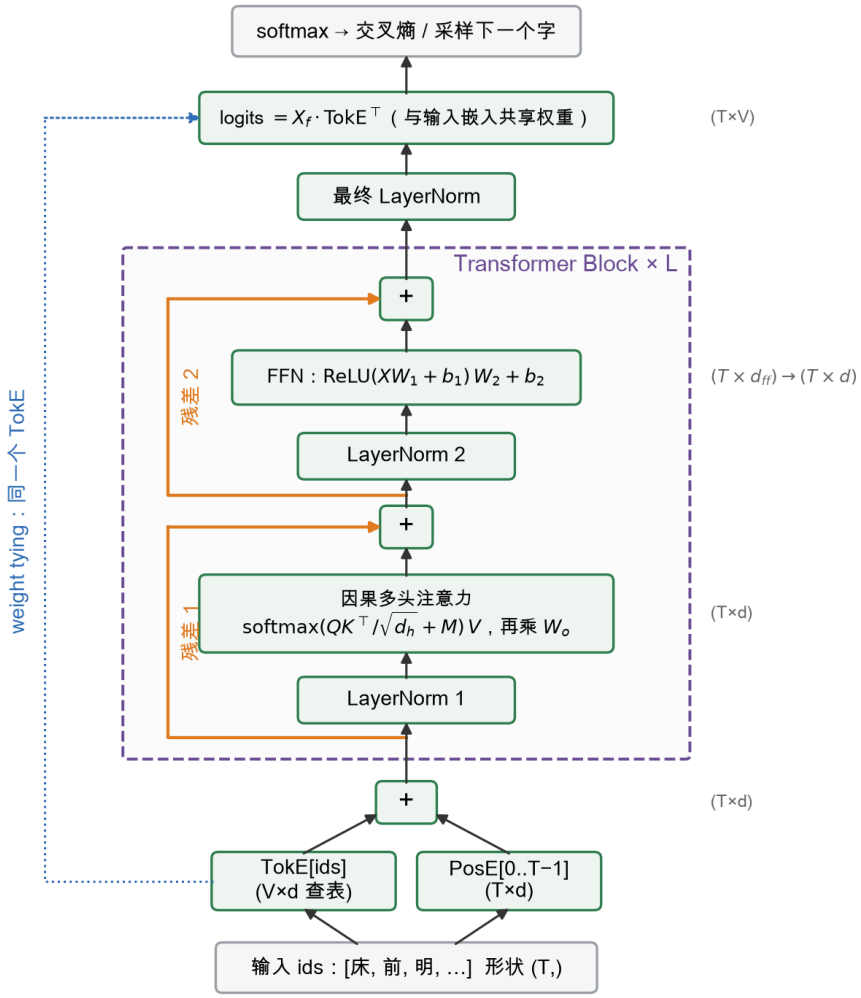


图 1-1 本书 GPT 的结构。橙线是残差直连，蓝色虚线表示输出投影与 token 嵌入是同一个矩阵

整个模型只有五种部件：嵌入查表、LayerNorm、因果多头注意力、FFN、残差相加。前向是“嵌入 → 2 个 Block → 最终 LN → 与嵌入矩阵相乘得到 logits”。

这个结构与 GPT-2 同构：Pre-LN、学习式位置嵌入、GELU/ReLU 的 FFN、weight tying。今天的主流开源大模型（例如 Llama 系列）保留了同一个计算骨架，但把其中几个部件换成了改进版本。下表列出本书每个部件在生产模型里的对应物：

本书的部件	生产模型中的对应	本书哪里有实验
字符级分词，$V = 289$	字节级 BPE，词表约 3 万到 25 万	第 2 章
学习式位置嵌入 E_{pos}	RoPE（旋转位置编码）	7.4 节
LayerNorm	RMSNorm	7.4 节
ReLU 的 FFN，$d_{ff} = 2d$	SwiGLU，隐藏维约 $\frac{8}{3}d$	7.4 节
多头注意力，K/V 头数 = Q 头数	GQA（多个 Q 头共享一组 K/V）；FlashAttention 融合算子	7.4 节、第 8 章
稠密 FFN	部分模型使用 MoE（稀疏专家）	第 9 章
手写反向传播	autograd	第 5 章
无 KV cache 的生成	KV cache + 连续批处理 + PagedAttention	第 8 章

本书的部件	生产模型中的对应	本书哪里有实验
单进程 CPU 训练	数据并行 / 张量并行 / 流水线并行	8.3 节、第 9 章

第 1 版的一处表述

第 1 版写“与千亿参数的 GPT-4 架构同构”。GPT-4 的架构没有公开，这个说法无法核实。准确的说法是：与 GPT-2 同构，与主流开源大模型在计算骨架上一致、在部件上有演进。

1.4 阅读与动手路径

第 4 章把 Go 实现拆成 8 个里程碑。每个里程碑都能独立测试，做完一个再做下一个，出错时能定位到具体的里程碑：

里程碑	内容	验收测试	验收标准
M1	`Mat`、矩阵乘、偏置、列和、切列	`TestMM`、`TestBiasColSumSlice`	与手算结果逐元素相等
M2	分词器、取 batch	`TestTokenizerAndBatch`	编解码往返一致；y 是 x 右移一位；最后一个窗口能被采到
M3	各算子的前向与反向	`TestLinearGrad` 等 7 个	每个算子单独做梯度检验，相对误差 $< 10^{-6}$
M4	注意力与 Block 前向	`TestCausality`	改动第 10 个位置，前 10 个位置的 logits 完全不变
M5	完整模型	`TestParamCount`、`TestInitLoss`	参数量 26,912；初始 loss 与 $\ln V$ 相差 < 0.05
M6	反向 + Adam	`TestGradCheckFull`、`TestOverfitOneBatch`	全模型梯度检验通过；固定一个 batch 训练 200 步，loss < 0.05
M7	训练循环、并行	`TestParallelMatchesSerial`	并行与串行的梯度差 $< 10^{-12}$
M8	生成、KV cache	`TestKVCacheMatchesFull`	带 cache 与不带 cache 的 logits 完全一致

建议顺序：第 2、3 章把数据和数学吃透 → 第 4 章按里程碑把 Go 版敲出来（本书主菜）→ 第 5 章 PyTorch 版对照 → 第 6 章跨实现对齐与排错 → 第 7、8 章按兴趣选读。

第 2 章 数据与分词

2.1 一句话说清

模型只认数字。本章把 628 个字符变成两样东西：一张“字符 ↔ 编号”的词表，以及大量用滑动窗口切出来的（输入, 目标）编号对。前者叫分词，后者是自回归语言模型的训练数据。

2.2 为什么用字符级分词

主流大模型用 BPE 把文本切成子词（GPT-2 的词表 50,257，GPT-4o 使用的 o200k 约 20 万）。本书故意用字符级，理由有三：

1. **零依赖**。BPE 要先训练词表（反复合并高频相邻对、处理特殊 token），字符级只需要“去重 + 排序 + 编号”，二十几行代码。
2. **词表小**。20 首诗只有 289 个不同字符，嵌入表只有 289×32，训练一次 1 秒多。
3. **结果直观**。生成的每一个字都能立刻判断对不对。

代价是序列更长、语义单元被打碎，所以真实系统不用字符级。但分词只影响“文本 → 编号”这一步，后面的模型、训练、生成完全不变，换成 BPE 时只需要替换分词器。

2.3 词表构建：rune 不是 byte

Go 里有一个经典陷阱：`len("床") == 3`，因为一个汉字在 UTF-8 里占 3 个字节。按字节遍历会把汉字拆碎，必须按 rune（Unicode 码点）遍历。`for _, r := range s` 正是按 rune 遍历的。

编号顺序也有讲究。第 1 版 Go 按“首次出现的顺序”编号，Python 按 `sorted(set(text))` 编号，两边的同一个字编号不同，权重就没法直接互换。第 2 版两边都按码点排序，同一个字在两边的编号完全一样，这是第 6 章做逐元素对齐的前提。

```
// Tokenizer 字符级分词器。词表按 Unicode 码点排序后编号，
// 与 Python 版 sorted(set(text)) 完全一致，两边的 id 可以直接互换。
type Tokenizer struct {
    stoi map[rune]int
    itos []rune
}
```

```
func NewTokenizer(text string) *Tokenizer {
    seen := map[rune]bool{}
    for _, r := range text { // range string 按 rune 遍历，不会把汉字拆成字节
        seen[r] = true
    }
    tk := &Tokenizer{stoi: map[rune]int{}}
    for r := range seen {
        tk.itos = append(tk.itos, r)
    }
    sort.Slice(tk.itos, func(i, j int) bool { return tk.itos[i] < tk.itos[j] })
    for i, r := range tk.itos {
        tk.stoi[r] = i
    }
    return tk
}
```

编码和解码就是查表。遇到词表外的字直接 panic，而不是静默跳过，这样错误会立刻暴露：

```
func (tk *Tokenizer) Encode(s string) []int {
    ids := make([]int, 0, len(s))
    for _, r := range s {
        id, ok := tk.stoi[r]
        if !ok {
            panic("字符不在词表中: " + string(r))
        }
        ids = append(ids, id)
    }
    return ids
}
```

2.4 滑动窗口：自回归数据的自监督

语言模型训练不需要人工标注，因为“下一个字”本身就是标签。具体做法是在语料上切一个长为 T 的窗口作为输入 x，把它整体右移一位作为目标 y：

```
语料: … 床 前 明 月 光 ， 疑 是 地 上 霜 …
x   : [床 前 明 月 光 ， 疑 是 地 上 霜 。 举 头 望 明]      (T = 16)
y   : [前 明 月 光 ， 疑 是 地 上 霜 。 举 头 望 明 月]      x 整体右移一位
```

模型在位置 0 看到“床”，要预测“前”；在位置 1 看到“床前”，要预测“明”……一个窗口同时提供 T 个训练信号。因果掩码（第 3 章）保证位置 t 只能看到 $x_0, \dots, x_t$，所以这 T 个信号互不泄露答案。

窗口起点 i 必须满足 $i + T \le N - 1$（y 的最后一个元素是 x_{i+T}），合法起点是 $0, 1, \dots, N-T-1$，共 $N - T = 612$ 个。

```
// GetBatch 随机取 B 个长度为 T 的窗口；y 是 x 整体右移一位。
// 合法起点 i ∈ [0, N-T-1]，共 N-T 个，所以是 Intn(N-T)。
func GetBatch(ids []int, B, T int, rng *rand.Rand) (x, y [][]int) {
    x, y = make([][]int, B), make([][]int, B)
    for b := 0; b < B; b++ {
        i := rng.Intn(len(ids) - T)
        x[b] = ids[i : i+T]
        y[b] = ids[i+1 : i+T+1]
    }
    return
}
```

> **第 1 版的两处 bug**
>
> 1. 第 1 版的 `getBatch` 使用命名返回值 `x, y [][]int`，但没有 `make`。命名返回值初始为 nil，`x[b] = …` 会直接 panic（index out of range）。
> 2. 第 1 版写 `rng.Intn(len(ids) - T - 1)`，取值范围是 $[0, N-T-2]$，最后一个合法起点 $N-T-1$ 永远取不到。PyTorch 版的 `torch.randint(0, len(ids) - T - 1)` 也是同样的问题。第 2 版两边都改为 $N-T$，`TestTokenizerAndBatch` 专门检查最后一个窗口能被采到。

几个工程要点：

- **窗口可以重叠。**起点 i 与 $i+1$ 的窗口有 15 个字重叠，但每个位置要预测的“下一个字”所处的上下文不同，监督信号并不重复。
- **T 是上下文长度上限。**模型永远看不到 16 个字之前的内容，位置嵌入表也只有 16 行。生成时同样只喂最近 16 个字。

- **随机起点是有放回采样**。每步独立抽 4 个起点，同一个窗口可能在一步里被抽到两次。它和“打乱后按顺序遍历”（无放回）在统计上接近，但不等价。
- **数据量与过拟合**。600 步 × 4 个窗口 = 2,400 个窗口样本，平均每个窗口被看到约 3.9 次。语料只有 628 字，模型最终会把它背下来。这是刻意的演示，不代表泛化能力；第 7 章换成 123 万字的语料，单独划出验证集来讨论泛化。

2.5 数据在两套技术栈里的形态

环节	Go 版	PyTorch 版
文本 → 编号	`[]int`，按 rune 遍历	`torch.tensor(ids, dtype=torch.long)`
词表	`map[rune]int` + `[]rune`，按码点排序	`dict` + `list`，`sorted(set(text))`
取 batch	`rng.Intn(N-T)` 随机起点，切片（不拷贝）	`torch.randint(0, N-T, (B,))`，`torch.stack`
batch 形状	`[][]int`，B 个长为 T 的切片	张量 `(B, T)`，int64
一步的计算方式	逐个窗口前向、反向，再平均梯度	4 个窗口叠成一个张量一次算完

最后一行是两版唯一的实质差别。数学上完全等价（第 6 章会逐元素验证），但 PyTorch 的批量计算可以交给向量化指令和 GPU 并行执行。Go 版在第 8.3 节用 goroutine 按窗口并行，相当于手写了一个单机的数据并行。

2.6 动手实验

实验 2-1 只用《静夜思》训练

目的：感受词表、窗口数和“背诵”的关系。

改动与命令：`head -1 corpus.txt > out/corpus_jys.txt`，然后 `cd go && go run . train -corpus ./out/corpus_jys.txt -every 100`

预期：25 个字符（含换行）、词表 $V = 20$、参数量 18,304，合法窗口只有 $25 - 16 = 9$ 个。step 1 的 loss 为 3.0036，接近 $\ln 20 = 2.9957$；100 步降到 0.051，600 步降到 0.0015。生成结果是整首诗原样输出，后面跟着几个换行。

观察：9 个窗口、1.8 万个参数，模型能把每个窗口都记住。loss 接近 0 只说明它记住了，不说明它学会了写诗。

实验 2-2 改变上下文长度 T

命令：`go run . train -T 4` 与 `go run . train -T 32`

预期（600 步，种子 1）：

T	参数量	全窗口平均 loss	训练耗时
4	26,528	2.81	0.44 秒
16（默认）	26,912	0.47	1.40 秒
32	27,424	0.18	3.23 秒

观察：参数量只有 E_{pos} 一项随 T 变化（每多一个位置多 32 个参数）。$T=4$ 时模型只能看到前 3 个字，不知道自己处在诗的哪一句，loss 降不下去，生成的句子支离破碎。$T=32$ 时一个窗口能覆盖整整一句七言，loss 更低，但每步的计算量也更大：注意力部分随 T^2 增长，其余部分随 T 线性增长。

2.7 小结

- 分词就是把文本变成编号序列。本书用字符级：628 字符 → $V=289$。按码点排序编号，让两套实现的编号完全一致。
- 自监督数据 = 滑动窗口 + 右移一位作为标签。合法起点有 $N-T$ 个，一个窗口提供 T 个监督信号。
- B、T、V 三个数定下来，数据管道的形状就定了：每步消费一个 (B,T) 的编号矩阵，产出 $B\times T$ 个预测任务。

第 3 章 前向与反向：完整梯度推导

3.1 一句话说清

训练就是两件事：前向算出 loss，反向算出 loss 对每个参数的偏导。前向是一串矩阵运算；反向沿计算图倒着走，每经过一个算子用一次链式法则。本章把这条链上的每一环写成公式并标上形状，第 4 章的 Go 代码就是这些公式的逐行翻译。

本章新增两样第 1 版没有的东西：一个可以手算核对的微型数值例子（3.12 节），以及梯度检验的误差分析（3.13 节）。

3.2 前向全链路

所有矩阵按“行 = token 位置”组织：$X \in \mathbb{R}^{T\times d}$ 的第 t 行是位置 t 的向量。下面每个公式右侧标注形状。

① **嵌入**：token 嵌入加位置嵌入。

$$X_t^{(0)} = E_{tok}[x_t] + E_{pos}[t], \qquad t = 0,\ \dots,\ T-1 \qquad [T\times d]$$

② **每个 Block**（$\ell = 1,\ \dots, L$），先是注意力子层：

$$\tilde{X} = \mathrm{LN}_1(X^{(\ell-1)}), \qquad Q = \tilde{X}W_q + b_q, \quad K = \tilde{X}W_k + b_k, \quad V = \tilde{X}W_v + b_v \qquad [T\times d]$$

按列切成 $h = 2$ 个头，每个头 $d_h = 16$ 列，各自计算：

$$S^{(i)} = \frac{Q^{(i)}K^{(i)\top}}{\sqrt{d_h}} \quad [T\times T], \qquad S^{(i)}_{tj} = -\infty \ \ (j > t), \qquad P^{(i)} = \mathrm{softmax}(S^{(i)}), \qquad C^{(i)} = P^{(i)}V^{(i)} \quad [T\times d_h]$$

两头按列拼回，过输出投影，加残差：

$$A = [C^{(1)}, C^{(2)}]\,W_o + b_o, \qquad X' = X^{(\ell-1)} + A \qquad [T\times d]$$

然后是 FFN 子层：

$$\bar{X} = \mathrm{LN}_2(X'), \quad H = \bar{X}W_1 + b_1 \ [T\times d_{ff}], \quad U = \mathrm{ReLU}(H), \quad F = UW_2 + b_2, \quad X^{(\ell)} = X' + F$$

③ **输出**：最终 LN，再乘嵌入矩阵的转置（weight tying）：

$$X_f = \mathrm{LN}_f(X^{(L)}), \qquad \mathrm{logits} = X_f E_{tok}^{\top} \qquad [T\times d]\cdot[d\times V] = [T\times V]$$

④ **损失**：逐位置交叉熵，对 T 个位置取平均。记 z_t 为 logits 的第 t 行：

$$L = \frac{1}{T}\sum_{t=0}^{T-1}\left(-z_{t,\,y_t} + \log\sum_{v=1}^{V} e^{z_{t,\,v}}\right)$$

初始化时权重很小（std=0.02），logits 接近 0，softmax 接近均匀分布，每个位置的交叉熵约为 $\ln V$。本模型实测初始 logits 的标准差约 0.11，初始 loss 约 5.66 到 5.67（随种子略有不同），理论值 $\ln 289 = 5.6664$。这个“初始 loss ≈ ln V”是排错时第一个要看的数。

3.3 矩阵求导的两条法则

反向传播里九成的公式都来自同一个算子：$Y = XW + b$，其中 X 是 $[T \times m]$，W 是 $[m \times n]$，Y 是 $[T \times n]$。已知上游梯度 $dY = \partial L/\partial Y$，求 dX、dW、db。

逐元素写出来：$Y_{tj} = \sum_k X_{tk} W_{kj} + b_j$。$W_{kj}$ 出现在所有行 t 的 Y_{tj} 里，所以

$$\frac{\partial L}{\partial W_{kj}} = \sum_t X_{tk}\, dY_{tj} \;\Rightarrow\; dW = X^\top dY, \qquad \frac{\partial L}{\partial X_{tk}} = \sum_j dY_{tj}\, W_{kj} \;\Rightarrow\; dX = dY\, W^\top, \qquad db = \mathrm{colsum}(dY)$$

不需要背。记住一个检查方法就够了：**梯度与原量同形状，用形状把乘法拼出来**。dW 必须是 $[m \times n]$，手上只有 $X : [T \times m]$ 和 $dY : [T \times n]$，唯一拼得出来的就是 $X^\top dY$。dX 必须是 $[T \times m]$，唯一拼得出来的是 $dY\, W^\top$。偏置被加到了每一行上，所以它的梯度是各行之和。

在 Go 代码里，这两条法则分别是 `MM(x, dy, true, false)` 和 `MM(dy, w, false, true)`，全书的反向代码里一共出现 20 多次。

3.4 交叉熵：反向的起点

对单个位置 $\ell_t = -z_y + \log \sum_v e^{z_v}$ 求导：

$$\frac{\partial \ell_t}{\partial z_v} = -\mathbf{1}[v = y] + \frac{e^{z_v}}{\sum_u e^{z_u}} = p_v - \mathbf{1}[v = y]$$

再算上对 T 个位置的平均，得到反向链的入口：

$$G = \frac{\partial L}{\partial \,\mathrm{logits}} = \frac{1}{T}(P - \mathrm{onehot}(y)) \qquad [T \times V]$$

G 的每一行之和为 0，目标列为负，其余列为正。这个性质可以直接拿来核对代码（3.12 节）。

3.5 权重共享的输出层

由 $\mathrm{logits} = X_f E_{tok}^\top$，按 3.3 节的法则：

$$dX_f = G\, E_{tok} \quad [T \times V] \cdot [V \times d], \qquad dE_{tok} \mathrel{+}= G^\top X_f \quad [V \times T] \cdot [T \times d]$$

这里写的是“+=”：E_{tok} 在嵌入查表里还被用过一次（3.10 节），两份梯度必须累加。这是 weight tying 在反向传播里唯一的代价。

3.6 LayerNorm 的反向

LayerNorm 对每一行独立计算。设一行为 $x \in \mathbb{R}^d$：

$$\mu = \frac{1}{d}\sum_j x_j, \quad \sigma^2 = \frac{1}{d}\sum_j (x_j-\mu)^2, \quad r = \frac{1}{\sqrt{\sigma^2+\varepsilon}}, \quad \hat{x}_j = (x_j-\mu)\,r, \quad y_j = \gamma_j \hat{x}_j + \beta_j$$

γ、β 的梯度直接可得（对所有行求和）：

$$d\gamma = \mathrm{colsum}(dY \odot \hat{X}), \qquad d\beta = \mathrm{colsum}(dY)$$

求 dx 分两步。第一步，$y_j = \gamma_j \hat{x}_j + \beta_j$，所以对 $\hat{x}_j$ 的梯度是 $g_j = \gamma_j\, dy_j$。第二步求 $\partial \hat{x}_j / \partial x_k$。注意 $\sum_j (x_j-\mu) = 0$，因此 $\partial \sigma^2/\partial x_k = \frac{2}{d}(x_k-\mu)$，$\partial r/\partial x_k = -r^3 (x_k-\mu)/d$，于是

$$\frac{\partial \hat{x}_j}{\partial x_k} = r\left(\delta_{jk} - \frac{1}{d}\right) + (x_j-\mu)\cdot\left(-\frac{r^3(x_k-\mu)}{d}\right) = r\left[\delta_{jk} - \frac{1}{d} - \frac{\hat{x}_j\hat{x}_k}{d}\right]$$

把 g 乘进去求和：

$$\frac{\partial L}{\partial x_k} = \sum_j g_j \frac{\partial \hat{x}_j}{\partial x_k} = r\left[g_k - \frac{1}{d}\sum_j g_j - \hat{x}_k \cdot \frac{1}{d}\sum_j g_j \hat{x}_j\right], \qquad g = \gamma \odot dy$$

三项分别对应：直接路径、减均值带来的耦合、除以标准差带来的耦合。前向需要缓存 $\hat{x}$ 和每行的 r。

> **第 1 版的公式错在哪里**
>
> 第 1 版写成 $\gamma_j r\,[dy_j - \frac{1}{d}\sum_k dy_k - \frac{\hat{x}_j}{d}\sum_k dy_k \hat{x}_k]$：把 γ_j 提到括号外，括号里的求和却用 dy_k 而不是 $\gamma_k dy_k$。只有当 γ 的所有分量都相等时，它才和正确公式一致。初始化时 $\gamma = 1$，所以错误公式在第一步碰巧是对的；训练几步之后 γ 不再均匀，梯度就错了。
>
> 第 1 版的 Go 代码先算 `dyh = dy*gamma` 再求和，代码是对的，只是公式写错了。第 2 版的 `TestLayerNormWrongFormula` 保留了错误公式作为反例：取一组非均匀的 γ，错误公式与正确梯度的最大差为 1.68。

3.7 残差连接

由 $X^{(\ell)} = X' + F(X')$，X' 通过两条路径影响 loss：直接加到输出上，以及先经过 FFN 分支再加到输出上。链式法则要求把两条路径的梯度相加：

$$dX' = dX^{(\ell)} + (\partial F/\partial X')^{\top}\, dX^{(\ell)}$$

第一项是直连路径，第二项是 FFN 分支。

在代码里就是 `dx2 = dx + dxFfn`：上游梯度一份原样流向浅层，一份经 FFN 和 LN2 反传后再加回来。**每条路径各加一次**。这就是常说的“梯度高速公路”：不管分支里的梯度被压得多小，直连那一份都能完整传到浅层。注意力子层的残差同理：$dX^{(\ell-1)} = dX' + dX_{\text{attn}}$。

3.8 FFN 的反向

$F = UW_2 + b_2$，$U = \mathrm{ReLU}(H)$，$H = \bar{X}W_1 + b_1$。从上游 dF 出发，连续用两次 3.3 节的法则，中间夹一个 ReLU：

$$dW_2 = U^\top dF, \quad db_2 = \mathrm{colsum}(dF), \quad dU = dF\, W_2^\top, \quad dH = dU \odot \mathbf{1}[H > 0]$$

$$dW_1 = \bar{X}^\top dH, \quad db_1 = \mathrm{colsum}(dH), \quad d\bar{X} = dH\, W_1^\top$$

最后 $d\bar{X}$ 过 LN2 的反向（3.6 节）得到 FFN 分支对 X' 的梯度 `dxFfn`。ReLU 的反向就是一个掩码：前向时为正的位置梯度照传，其余置零。

3.9 注意力的反向

先过输出投影：$A = C_{cat}W_o + b_o$，所以 $dW_o = C_{cat}^\top dA$，$dC_{cat} = dA\, W_o^\top$。按列切开得到每个头的 dC。对单个头（省略上标 (i)）：

$C = PV$ 的反向（两条法则）：

$$dP = dC\, V^\top \quad [T \times T], \qquad dV = P^\top dC \quad [T \times d_h]$$

softmax 的反向（按行）。记 $Z = QK^\top/\sqrt{d_h}$ 为缩放后的分数，$P = \mathrm{softmax}(Z)$：

$$dZ_{tj} = P_{tj}\left(dP_{tj} - \sum_k dP_{tk}P_{tk}\right)$$

缩放与 $QK^\top$ 的反向。$Z = S/\sqrt{d_h}$，其中 $S = QK^\top$，所以 $dS = dZ/\sqrt{d_h}$，再用两条法则：

$$dS = \frac{dZ}{\sqrt{d_h}}, \qquad dQ = dS\, K, \qquad dK = dS^\top Q$$

缩放系数只乘一次。

> **第 1 版在这里多乘了一次**
>
> 第 1 版先说“softmax 反向后再乘缩放系数 $1/\sqrt{d_h}$ 得到 dS”，接着又写 $dQ = \frac{1}{\sqrt{d_h}} dS\, K$，缩放乘了两次。代码只乘了一次，所以代码是对的、公式是错的。`TestAttentionGrad` 用梯度检验确认了只乘一次的版本：dQ、dK、dV 的相对误差都在 10^{-8} 量级。

因果掩码在反向时不需要特殊处理：被掩码的位置前向时概率 $P_{tj} = 0$，代入 softmax 反向公式，梯度自然为 0。

QKV 投影的反向。三个投影结构相同，各自得到 $dW_q = \tilde{X}^\top dQ$、$db_q = \mathrm{colsum}(dQ)$ 等。关键在于 $\tilde{X}$ 同时喂给了三个投影，它的梯度是三条链之和：

$$d\tilde{X} = dQ\, W_q^\top + dK\, W_k^\top + dV\, W_v^\top$$

再过 LN1 的反向，得到注意力分支对 $X^{(\ell-1)}$ 的梯度 `dxAttn`。

3.10 嵌入的反向：scatter-add

嵌入查表的反向不是矩阵乘，而是“哪一行被查过，梯度就加回哪一行”：

$$dE_{tok}[x_t] \mathrel{+}= dX_t^{(0)}, \qquad dE_{pos}[t] \mathrel{+}= dX_t^{(0)}$$

同一个字在一个窗口里可能出现多次（比如“山”），它那一行会收到多份梯度，必须累加。没出现过的字，这一份梯度为 0。PyTorch 的 `nn.Embedding` 默认也是这样产生一个稠密的梯度矩阵；设置 `sparse=True` 后改为稀疏梯度，推荐系统里几千万行的大嵌入表常用这个选项，语言模型一般不用。

3.11 一条完整的反向链

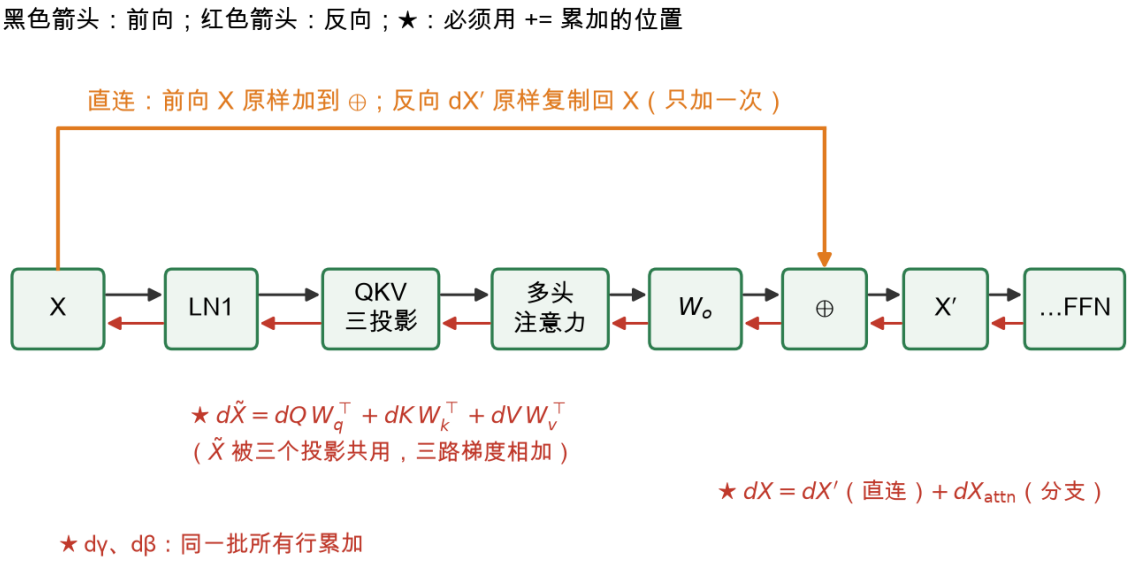


图 3-1 一个 Block 内的前向与反向。★ 标出必须用 += 累加的位置

把 3.4 到 3.10 节串起来，就是第 4 章 `Backward` 函数的结构：

```
G = (P − onehot(y)) / T                          交叉熵（3.4）
dE_tok += Gᵀ·Xf ;  dXf = G·E_tok                 权重共享（3.5）
dx = LN_f 反向(dXf)                               （3.6）
for ℓ = L … 1:
    FFN 分支反向(dx)              → dxFfn         （3.8，含 LN2 反向）
    dx2 = dx + dxFfn                              残差 2（3.7）
    注意力分支反向(dx2)           → dxAttn        （3.9，含 W_o、各头、QKV、LN1）
    dx  = dx2 + dxAttn                            残差 1
dE_tok[x_t] += dx_t ;  dE_pos[t] += dx_t          嵌入 scatter-add（3.10）
```

需要累加的地方一共四类：E_{tok}（两处来源）、$\tilde{X}$（三个投影）、LN 的 γ 和 β（所有行）、两处残差（直连 + 分支）。手写反向时，九成的 bug 出在这四类地方。

3.12 微型数值走读

公式读懂了，不等于能对上数字。下面用一个极小的模型把前向、反向的中间量全部打印出来：$d=4$、$T=3$、$V=5$、单头、单层、$d_{ff}=8$，共 212 个参数。为了让数值不至于太小，初始化标准差取 0.5（种子 42）。运行 `make walk`（即 `go run . walk`）。

输入编号 $x=[0,3,1]$，目标 $y=[3,1,4]$。LN1 的输出 $\tilde{X}$ 每一行均值为 0、方差约为 1：

位置	$\tilde{X}$ 第 1 列	第 2 列	第 3 列	第 4 列
0	0.9505	−0.2342	−1.5382	0.8219
1	−0.0883	−0.0838	−1.3229	1.4950
2	0.6331	1.3077	−0.9163	−1.0244

注意力概率 P 是下三角的，每行和为 1。位置 0 只能看到自己，所以第一行是 $[1,0,0]$：

位置	看位置 0	看位置 1	看位置 2
0	1.0000	0	0
1	0.4779	0.5221	0
2	0.2790	0.3548	0.3662

logits 与 $G=\partial L/\partial \text{logits}$：

位置	logits	G（目标列加粗）
0（目标 3）	1.5504, 0.8060, 2.0896, 1.7365, −0.0205	0.0724, 0.0344, 0.1242, **−0.2461**, 0.0151
1（目标 1）	1.4752, 0.9282, 1.9418, 1.7396, −0.0827	0.0711, **−0.2922**, 0.1134, 0.0927, 0.0150
2（目标 4）	1.1913, 0.8498, 1.5873, 1.1637, −0.4294	0.0763, 0.0542, 0.1134, 0.0743, **−0.3182**

loss = 2.1756，高于 $\ln 5=1.6094$：初始化标准差 0.5 太大，logits 不再接近 0，模型一开始就“自信地猜错”。第 6.4 节会在真实模型上遇到同一个现象。

动手核对：

1. G 每行之和应为 0。第 0 行：$0.0724+0.0344+0.1242-0.2461+0.0151=0.0000$。
2. 手算 $G_{0,3}$。第 0 行 logits 做 softmax：$e^{1.5504}=4.713$，$e^{0.8060}=2.239$，$e^{2.0896}=8.082$，$e^{1.7365}=5.677$，$e^{-0.0205}=0.980$，和为 21.69，所以 $p_3=5.677/21.69=0.2617$，$G_{0,3}=(0.2617-1)/3=-0.2461$，与程序输出一致。
3. 程序最后对全部 212 个参数逐一做梯度检验，最大相对误差 2.2×10^{-6}。

3.13 梯度检验：手写反向的保命符

手写反向一定会写错，标准的验证手段是数值梯度检验。对参数 θ 的某个元素，用中心差分近似偏导：

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta+\varepsilon)-L(\theta-\varepsilon)}{2\varepsilon}, \qquad \text{rel} = \frac{|a-n|}{\max(|a|+|n|,\ 10^{-4})}$$

其中 a 是解析梯度，n 是数值梯度。两个细节决定了这个工具好不好用。

一、ε 怎么取。 中心差分有两种误差：截断误差约为 $\varepsilon^2|L'''|/6$，随 ε 变小而变小；舍入误差约为 $10^{-16}|L|/\varepsilon$，随 ε 变小而变大。本模型还有一个特殊情况：初始化时残差流很小（标准差约 0.03），LayerNorm 会把它放大约 35 倍，函数的高阶导数很大；FFN 里 ReLU 的折点也会让差分跨过不可导点。实测初始化状态下 `l0.b2` 的最差元素：

ε	最大相对误差	解释
10^{-4}	1.5×10^{-1}	截断误差和 ReLU 折点主导
10^{-5}	8.9×10^{-2}	仍然偏大，容易误判为 bug
10^{-6}	2.7×10^{-8}	正常

本书取 $\varepsilon = 10^{-6}$，全程 float64。

二、相对误差的分母要设下限。 有些元素的梯度本身就很小，例如初始化时 W_k 的部分元素只有 10^{-7} 量级（初始注意力接近均匀，Q、K 对 loss 的影响很弱），而差分的舍入误差约为 10^{-10}，算出来的“相对误差”会虚高到 10^{-3}。分母设下限 10^{-4}，相当于对小梯度改用绝对误差衡量。

有了这两点，判据就很清楚：

情况	全模型最大相对误差（每个参数矩阵抽 50 个元素）
正确实现，初始化状态	9.3×10^{-6}
正确实现，训练 200 步后	3.0×10^{-6}
残差处多加一份直连梯度（第 6.5 节的 bug）	1.0

正确与错误之间差了五个数量级。`TestGradCheckFull` 的阈值取 10^{-4}。

3.14 Adam：梯度之后的一步

有了梯度，用 Adam 更新参数。每个参数各自维护一阶矩 m 和二阶矩 v：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}, \qquad \theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\varepsilon}$$

超参数：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\varepsilon = 10^{-8}$，$\eta = 3\times10^{-3}$。

偏差修正修的是什么。 m、v 都从 0 开始，前几步被严重低估。$\beta_2 = 0.999$ 时 v 的低估远比 m 严重，所以如果不修正，更新量 $m/\sqrt{v}$ 会**偏大**。不修正时的实际步长是修正后的 $(1-\beta_1^t)/\sqrt{1-\beta_2^t}$ 倍：

步数 t	1	10	50	100	300	600
步长放大倍数	3.16	6.53	4.50	3.24	1.96	1.49

第 1 版把方向说反了

第 1 版写“没有偏差修正，初期更新会明显偏小”。实际是偏大，前 100 步里步长被放大 3 到 6.5 倍，600 步时仍是 1.5 倍。Adam 原论文第 3 节也指出，不做偏差修正会导致训练初期步长过大。

实测（Go 版，种子 1，600 步）：

优化器	step 100	step 300	step 600	全窗口平均 loss
Adam（有偏差修正）	3.64	1.28	0.37	0.47
Adam（关闭偏差修正）	4.46	2.37	0.76	1.19
SGD，$\eta = 0.5$	5.01	2.57	0.87	1.05
SGD，$\eta = 3 \times 10^{-3}$	5.56	5.42	5.26	5.26

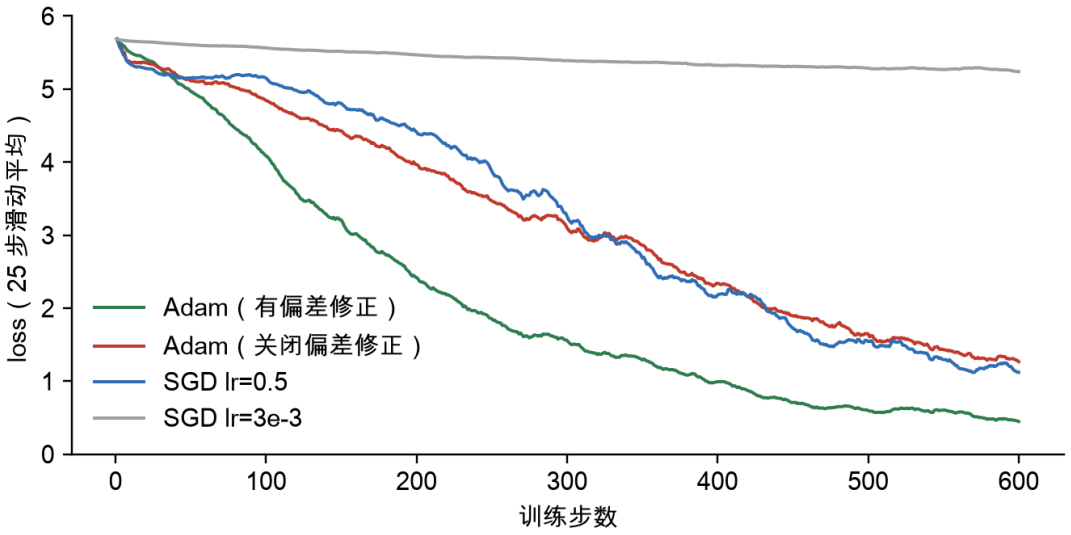


图 3-2 优化器对照（25 步滑动平均）

两个观察：

1. 关闭偏差修正后，前期步长过大，loss 在前 100 步几乎不降，最终明显落后。
2. 同样的学习率 3×10^{-3}，SGD 几乎不动；SGD 要把学习率调到 0.5 才能接近 Adam。原因是本模型的梯度量级很小（多数元素在 10^{-4} 到 10^{-1} 之间），SGD 的步长与梯度成正比，而 Adam 的步长约等于 η，与梯度的绝对大小无关。

第二点还有一个推论：**如果某个参数的梯度整体被放大或缩小了常数倍，Adam 的更新几乎不受影响**，因为 $\hat{m}/\sqrt{\hat{v}}$ 的分子分母同时缩放。这会让一类反向 bug 在 loss 曲线上完全看不出来，第 6.5 节会遇到。

3.15 算子卡片

算子	前向	反向	需要缓存	Go 函数	PyTorch
线性层	$Y = XW + b$	$dX = dYW^\top$，$dW = X^\top dY$，$db = \text{colsum}(dY)$	X	`MM`、`AddBias`、`ColSum`	`nn.Linear`
ReLU	$\max(0, h)$	$dh = du \odot \mathbf{1}[h > 0]$	H	`ReLU`、`ReLUBackward`	`nn.ReLU`
LayerNorm	$\gamma \odot \hat{x} + \beta$	见 3.6 节，$g = \gamma \odot dy$	$\hat{x}$、r	`LayerNorm`、`LNBackward`	`nn.LayerNorm`
因果 softmax	按行，$j > t$ 置 0	$dz = p \odot (dp - \langle dp, p \rangle)$	P	`Softmax`、`SoftmaxBackward`	在 SDPA 内部
注意力	$\text{softmax}(QK^\top/\sqrt{d_h})V$	见 3.7 节，缩放只乘一次	Q, K, V, P	在 `Forward`、`Backward` 内	`F.scaled_dot_product_attention`
交叉熵	$-z_y + \log \sum e^z$	$(p - \text{onehot})/T$	无（反向时重算）	`CrossEntropy`	`F.cross_entropy`
嵌入	查表	scatter-add	输入编号	在 `Forward`、`Backward` 内	`nn.Embedding`

3.16 小结

- 前向 = 嵌入 → L 个（LN → 注意力 → 残差 → LN → FFN → 残差）→ LN → 与嵌入矩阵相乘 → 交叉熵。
- 反向只有三种“新”结构：残差的梯度分流、LN 的行内耦合、softmax 的行内耦合。其余都是 3.3 节的两条转置法则，用形状就能拼出来。
- 共享输入或共享参数的地方，梯度必须累加：E_{tok}、$\tilde{X}$、γ/β、残差。
- 梯度检验要选对 ε（本书 10^{-6}），相对误差的分母要设下限。正确实现在 10^{-5} 量级，bug 在 10^{-2} 以上。
- Adam 的偏差修正防止的是初期步长过大，不是过小。Adam 对梯度的整体缩放不敏感，这既是它好用的原因，也是它会掩盖 bug 的原因。

第 4 章 Go 实现：8 个里程碑

4.1 一句话说清

这一章把第 3 章的每个公式翻译成 Go。只用标准库，核心代码约 900 行（矩阵、算子、模型、前向、反向、训练、生成与 KV cache，含注释），另有约 400 行测试。第 1 版只给了约三分之一的代码，`Forward` 和 `Backward` 都在中途截断；第 2 版正文按里程碑给出每一段，附录 A 收录全部源码。

开发节奏固定为：**写完一个里程碑 → 跑它的测试 → 通过了再往下走**。手写反向传播最怕的是写完 700 行才第一次运行，出错时根本无从定位。

```
cd go
go test -run 'TestMM|TestBiasColSumSlice' -v     # M1
go test -run TestTokenizerAndBatch -v            # M2
go test -run 'Grad$|WrongFormula' -v             # M3：各算子梯度检验
go test -v ./...                                 # 全部
```

4.2 M1：矩阵工具

一个行优先的稠密矩阵 `Mat`，加上一个支持转置标记的通用乘法，就是全部的线性代数。

```
// Mat 行优先稠密矩阵：第 i 行第 j 列存放在 D[i*C+j]。
// 全书约定：矩阵的一行 = 一个 token 位置的向量。
type Mat struct {
    R, C int
    D    []float64
}
```

```
// MM 通用矩阵乘：out = op(a)·op(b)，ta/tb 为 true 表示先转置。
// 反向传播里绝大多数矩阵乘只有两种形态：
//
//    dW = Xᵀ·dY  →  MM(x, dy, true, false)
//    dX = dY·Wᵀ  →  MM(dy, w, false, true)
func MM(a, b *Mat, ta, tb bool) *Mat {
    ar, ac := a.R, a.C
    if ta {
        ar, ac = a.C, a.R
    }
    br, bc := b.R, b.C
    if tb {
        br, bc = b.C, b.R
    }
    if ac != br {
        panic(fmt.Sprintf("MM 形状不匹配: (%d×%d)·(%d×%d)", ar, ac, br, bc))
    }
    out := NewMat(ar, bc)
    for i := 0; i < ar; i++ {
        orow := out.D[i*bc : (i+1)*bc]
        for k := 0; k < ac; k++ {
            var av float64
            if ta {
                av = a.D[k*a.C+i]
            } else {
                av = a.D[i*a.C+k]
            }
            if tb { // b 转置：按列读 b，步长为 b.C
                for j := 0; j < bc; j++ {
                    orow[j] += av * b.D[j*b.C+k]
                }
            } else { // 不转置：b 的第 k 行是连续内存，内层循环对缓存友好
```

```
                brow := b.D[k*b.C : (k+1)*b.C]
                for j := 0; j < bc; j++ {
                    orow[j] += av * brow[j]
                }
            }
        }
    }
    return out
}
```

两处实现细节：

- **循环顺序是 i-k-j**。最内层循环沿 `out` 和 `b` 的一行连续访问内存，对 CPU 缓存友好。第 1 版的写法是用闭包 `at(i,k)`、`bt(k,j)` 取元素，每次乘加都多一次函数调用和分支判断；在本书的规模下差别不大，矩阵变大后会明显变慢。第 8.3 节讨论性能时会回到这个话题。
- **形状不匹配立刻 panic**，并打印两边的形状。手写反向时最常见的错误是转置标记写反，这个检查能让它在第一次运行时就暴露。

其余工具函数都很短：

```
// AddInplace a += b，用于梯度累加与残差相加。
func AddInplace(a, b *Mat) {
    if a.R != b.R || a.C != b.C {
        panic(fmt.Sprintf("AddInplace 形状不匹配: %d×%d += %d×%d", a.R, a.C, b.R, b.C))
    }
    for i := range a.D {
        a.D[i] += b.D[i]
    }
}
```

```
// AddBias 每一行加上同一个偏置行向量 b（1×C），即广播加法。
func AddBias(x, b *Mat) {
    for i := 0; i < x.R; i++ {
        for j := 0; j < x.C; j++ {
            x.D[i*x.C+j] += b.D[j]
        }
    }
}
```

```
// ColSum 按列求和得到 1×C，是偏置梯度的计算方式：∂L/∂b = colsum(dY)。
func ColSum(x *Mat) *Mat {
    out := NewMat(1, x.C)
    for i := 0; i < x.R; i++ {
        for j := 0; j < x.C; j++ {
            out.D[j] += x.D[i*x.C+j]
        }
    }
    return out
}
```

```
// SliceCols 取第 [c0, c1) 列，拷贝成新矩阵。多头注意力"切头"就是它。
func SliceCols(m *Mat, c0, c1 int) *Mat {
    out := NewMat(m.R, c1-c0)
    for i := 0; i < m.R; i++ {
        copy(out.D[i*out.C:(i+1)*out.C], m.D[i*m.C+c0:i*m.C+c1])
    }
    return out
}
```

```
// SetCols 把 src 写入 dst 从第 c0 列开始的位置。多头"拼回"就是它。
func SetCols(dst, src *Mat, c0 int) {
    for i := 0; i < src.R; i++ {
        copy(dst.D[i*dst.C+c0:i*dst.C+c0+src.C], src.D[i*src.C:(i+1)*src.C])
    }
}
```

验收：`TestMM` 用一个 2×3 乘 3×2 的例子与手算结果比对，并覆盖四种转置组合；`TestBiasColSumSlice` 检查偏置、列和、切列、写列。

```
$ go test -run 'TestMM|TestBiasColSumSlice' -v
--- PASS: TestMM (0.00s)
--- PASS: TestBiasColSumSlice (0.00s)
```

4.3 M2：分词与取 batch

代码已在第 2.3、2.4 节给出（`Tokenizer`、`GetBatch`）。`TestTokenizerAndBatch` 检查三件事：编码再解码等于原文；y 恰好是 x 右移一位；最后一个合法窗口能被采到（第 1 版的 off-by-one 正是在这里）。

4.4 M3：算子的前向与反向

LayerNorm 按行计算，前向返回反向需要的缓存 $\hat{x}$ 和每行的 r：

```
// LayerNorm 按行归一化：xˆ = (x−μ)·r，r = 1/√(σ²+ε)，y = γ⊙xˆ + β。
// 返回 y 以及反向需要的缓存 xˆ 和每行的 r。
func LayerNorm(x, gamma, beta *Mat) (y, xhat *Mat, rstd []float64) {
    n := x.C
    y, xhat = NewMat(x.R, n), NewMat(x.R, n)
    rstd = make([]float64, x.R)
    for i := 0; i < x.R; i++ {
        row := x.D[i*n : (i+1)*n]
        var mu, va float64
        for _, v := range row {
            mu += v
        }
        mu /= float64(n)
        for _, v := range row {
            d := v - mu
            va += d * d
        }
        va /= float64(n) // 有偏方差（除以 n），与 PyTorch 一致
        r := 1.0 / math.Sqrt(va+lnEps)
        rstd[i] = r
        for j, v := range row {
            h := (v - mu) * r
            xhat.D[i*n+j] = h
            y.D[i*n+j] = gamma.D[j]*h + beta.D[j]
        }
    }
    return
}
```

反向是 3.6 节公式的直接翻译。先算 $g=\gamma\odot dy$，再求两个行内均值；$d\gamma$、$d\beta$ 是累加的：

```
// LNBackward 令 g = γ⊙dy（逐元素），则
//
//    dx_j = r · [ g_j − mean(g) − xˆ_j · mean(g⊙xˆ) ]
//
// dγ、dβ 是累加的：同一批里所有行共享一组 γ、β。
func LNBackward(dy, xhat, gamma *Mat, rstd []float64, dgamma, dbeta *Mat) *Mat {
    n := dy.C
    dx := NewMat(dy.R, n)
    g := make([]float64, n)
    for i := 0; i < dy.R; i++ {
        var sumG, sumGX float64
        for j := 0; j < n; j++ {
            d, h := dy.D[i*n+j], xhat.D[i*n+j]
            g[j] = d * gamma.D[j]
            sumG += g[j]
            sumGX += g[j] * h
```

```
            dgamma.D[j] += d * h // 累加
            dbeta.D[j] += d      // 累加
        }
        meanG, meanGX := sumG/float64(n), sumGX/float64(n)
        for j := 0; j < n; j++ {
            dx.D[i*n+j] = rstd[i] * (g[j] - meanG - xhat.D[i*n+j]*meanGX)
        }
    }
    return dx
}
```

softmax 带因果掩码：第 i 行只对前 $i+1$ 列做 softmax，其余位置的概率保持为 0，等价于把这些分数设为 $-\infty$。减去行最大值防止 `exp` 溢出：

```
// Softmax 按行计算；causal 为 true 时第 i 行只看第 0..i 列（j>i 的概率直接置 0，
// 等价于把这些分数设成 −∞）。减去行最大值保证 exp 不溢出。
func Softmax(s *Mat, causal bool) *Mat {
    p := NewMat(s.R, s.C)
    for i := 0; i < s.R; i++ {
        last := s.C
        if causal {
            last = i + 1
        }
        row := s.D[i*s.C : i*s.C+last]
        mx := row[0]
        for _, v := range row {
            if v > mx {
                mx = v
            }
        }
        var sum float64
        for j, v := range row {
            e := math.Exp(v - mx)
            p.D[i*s.C+j] = e
            sum += e
        }
        for j := 0; j < last; j++ {
            p.D[i*s.C+j] /= sum
        }
    }
    return p
}
```

```
// SoftmaxBackward 按行：ds_j = p_j · (dp_j − Σ_k dp_k p_k)。
// 被掩码的位置 p=0，梯度自然为 0，不需要特殊处理。
func SoftmaxBackward(dp, p *Mat) *Mat {
    ds := NewMat(dp.R, dp.C)
    for i := 0; i < dp.R; i++ {
        var dot float64
        for j := 0; j < dp.C; j++ {
            dot += dp.D[i*dp.C+j] * p.D[i*p.C+j]
        }
        for j := 0; j < dp.C; j++ {
            ds.D[i*dp.C+j] = p.D[i*p.C+j] * (dp.D[i*dp.C+j] - dot)
        }
    }
    return ds
}
```

交叉熵同时返回 loss 和 $G = (P - \text{onehot})/T$。注意 `mx` 的用法：$\log \sum e^{z} = m + \log \sum e^{z-m}$，数学上相等，数值上不会溢出。

```
// CrossEntropy 逐位置交叉熵并对 T 个位置取平均。
// 返回 loss 和 ∂L/∂logits = (softmax(logits) − onehot(y)) / T。
func CrossEntropy(logits *Mat, targets []int) (float64, *Mat) {
    T, V := logits.R, logits.C
    dlogits := NewMat(T, V)
```

```
    var loss float64
    for t := 0; t < T; t++ {
        row := logits.D[t*V : (t+1)*V]
        mx := row[0]
        for _, v := range row {
            if v > mx {
                mx = v
            }
        }
        var sum float64
        for _, v := range row {
            sum += math.Exp(v - mx)
        }
        // -log softmax_y = -logit_y + log Σ exp(logit) = -logit_y + mx + log Σ exp(logit-mx)
        loss += -row[targets[t]] + mx + math.Log(sum)
        for j, v := range row {
            p := math.Exp(v-mx) / sum
            if j == targets[t] {
                p -= 1
            }
            dlogits.D[t*V+j] = p / float64(T)
        }
    }
    return loss / float64(T), dlogits
}
```

验收：每个算子单独做梯度检验。 套路是构造一个标量 $L = \sum(\text{out} \odot R)$，$R$ 是随机矩阵，于是 $\partial L/\partial \,\text{out} = R$。把 R 当作上游梯度喂给反向函数，再与中心差分逐元素比较：

```
// checkGrad 对 x 的每个元素做中心差分，与 analytic 比较。
func checkGrad(t *testing.T, name string, x, analytic *Mat, loss func() float64) {
    t.Helper()
    const eps = 1e-6
    worst := 0.0
    for j := range x.D {
        orig := x.D[j]
        x.D[j] = orig + eps
        lp := loss()
        x.D[j] = orig - eps
        lm := loss()
        x.D[j] = orig
        num := (lp - lm) / (2 * eps)
        den := math.Abs(num) + math.Abs(analytic.D[j])
        if den > 1e-9 {
            worst = math.Max(worst, math.Abs(num-analytic.D[j])/den)
        }
    }
    if worst > 1e-6 {
        t.Fatalf("%s 梯度检验失败：最大相对误差 %.2e", name, worst)
    }
    t.Logf("%-14s 最大相对误差 %.2e", name, worst)
}
```

```
func TestLayerNormGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(3))
    x, R := randMat(4, 8, rng), randMat(4, 8, rng)
    gamma, beta := randMat(1, 8, rng), randMat(1, 8, rng) // γ 刻意取非均匀值
    f := func() float64 { y, _, _ := LayerNorm(x, gamma, beta); return dot(y, R) }
    _, xhat, rstd := LayerNorm(x, gamma, beta)
    dg, db := NewMat(1, 8), NewMat(1, 8)
    dx := LNBackward(R, xhat, gamma, rstd, dg, db)
    checkGrad(t, "LN dx", x, dx, f)
    checkGrad(t, "LN dγ", gamma, dg, f)
    checkGrad(t, "LN dβ", beta, db, f)
}
```

注意 γ 取的是随机值而不是全 1。如果 $\gamma=1$，第 1 版那个错误的 LN 公式也能通过这个测试。实测结果（`go test -run 'Grad$' -v`）：

测试	检查对象	最大相对误差
`TestLinearGrad`	dX、dW、db	1.0×10^{-8}、2.7×10^{-9}、5.0×10^{-10}
`TestReLUGrad`	dh	2.8×10^{-10}
`TestLayerNormGrad`	dx、$d\gamma$、$d\beta$	1.4×10^{-8}、3.5×10^{-8}、1.2×10^{-8}
`TestSoftmaxGrad`	因果 softmax	2.5×10^{-8}
`TestCrossEntropyGrad`	G	1.7×10^{-8}
`TestAttentionGrad`	dQ、dK、dV（含缩放）	7.9×10^{-9}、3.8×10^{-8}、2.0×10^{-8}

单个算子的误差比全模型小两到三个数量级，因为这里没有多层 LN 和 ReLU 折点叠加。`TestLayerNormWrongFormula` 额外确认第 1 版的错误公式会被识别出来（与正确梯度最大差 1.68）。

4.5 M4：前向与因果性

反向需要前向的中间结果，所以前向要把它们存下来。下面三个结构体就是这个模型的“计算图”：

```
// HeadCache 单个注意力头反向需要的中间结果。
type HeadCache struct{ Q, K, V, P *Mat } // Q/K/V: T×dh，P: T×T（softmax 后）

// LayerCache 一个 Block 反向需要的全部中间结果。
type LayerCache struct {
    Xhat1 *Mat      // LN1 内部标准化值 x^
    Rstd1 []float64 // LN1 每行的 r
    Xln1  *Mat      // LN1 输出（QKV 的共同输入）
    Heads []HeadCache
    Cat   *Mat // 多头拼接结果 T×D
    Xhat2 *Mat
    Rstd2 []float64
    Xln2  *Mat // LN2 输出（FFN 输入）
    H, U  *Mat // FFN 隐藏层 ReLU 之前 / 之后
}
```

```
// LayerCache 一个 Block 反向需要的全部中间结果。
type LayerCache struct {
    Xhat1 *Mat      // LN1 内部标准化值 x^
    Rstd1 []float64 // LN1 每行的 r
    Xln1  *Mat      // LN1 输出（QKV 的共同输入）
    Heads []HeadCache
    Cat   *Mat // 多头拼接结果 T×D
    Xhat2 *Mat
    Rstd2 []float64
    Xln2  *Mat // LN2 输出（FFN 输入）
    H, U  *Mat // FFN 隐藏层 ReLU 之前 / 之后
}
```

```
type FwdCache struct {
    Ids    []int
    Layers []LayerCache
    Xhatf  *Mat
    Rstdf  []float64
    Xf     *Mat // 最终 LN 输出
    Logits *Mat // T×V
}
```

```
// Forward 单窗口前向，返回带全部缓存的 FwdCache。ids 长度 ≤ Cfg.T。
// 只读模型参数，可以被多个 goroutine 并发调用。
func (m *Model) Forward(ids []int) *FwdCache {
```

```
    c := m.Cfg
    T, D, dh := len(ids), c.D, c.Dh()
    fc := &FwdCache{Ids: ids}

    // ① 嵌入：X⁰[t] = TokE[ids[t]] + PosE[t]
    x := NewMat(T, D)
    for t, id := range ids {
        for j := 0; j < D; j++ {
            x.D[t*D+j] = m.TokE.D[id*D+j] + m.PosE.D[t*D+j]
        }
    }

    scale := 1.0 / math.Sqrt(float64(dh))
    for _, ly := range m.Layers {
        var lc LayerCache
        // ② 注意力子层（Pre-LN）
        xln1, xhat1, rstd1 := LayerNorm(x, ly.LN1g, ly.LN1b)
        lc.Xhat1, lc.Rstd1, lc.Xln1 = xhat1, rstd1, xln1
        q := MM(xln1, ly.Wq, false, false)
        AddBias(q, ly.Bq)
        k := MM(xln1, ly.Wk, false, false)
        AddBias(k, ly.Bk)
        v := MM(xln1, ly.Wv, false, false)
        AddBias(v, ly.Bv)

        cat := NewMat(T, D)
        for h := 0; h < c.NHead; h++ {
            qh := SliceCols(q, h*dh, (h+1)*dh)
            kh := SliceCols(k, h*dh, (h+1)*dh)
            vh := SliceCols(v, h*dh, (h+1)*dh)
            s := MM(qh, kh, false, true) // S = Q·Kᵀ / √dh   (T×T)
            Scale(s, scale)
            p := Softmax(s, m.Bug != BugNoMask)         // 因果掩码 + softmax
            SetCols(cat, MM(p, vh, false, false), h*dh) // C = P·V
            lc.Heads = append(lc.Heads, HeadCache{qh, kh, vh, p})
        }
        lc.Cat = cat
        attn := MM(cat, ly.Wo, false, false)
        AddBias(attn, ly.Bo)
        x2 := x.Clone()
        AddInplace(x2, attn) // 残差 1：X' = X + A

        // ③ FFN 子层（Pre-LN）
        xln2, xhat2, rstd2 := LayerNorm(x2, ly.LN2g, ly.LN2b)
        lc.Xhat2, lc.Rstd2, lc.Xln2 = xhat2, rstd2, xln2
        hid := MM(xln2, ly.W1, false, false)
        AddBias(hid, ly.B1)
        u := ReLU(hid)
        lc.H, lc.U = hid, u
        f := MM(u, ly.W2, false, false)
        AddBias(f, ly.B2)
        x3 := x2.Clone()
        AddInplace(x3, f) // 残差 2：X = X' + F

        fc.Layers = append(fc.Layers, lc)
        x = x3
    }

    // ④ 最终 LN + 共享嵌入的输出投影：logits = Xf · TokEᵀ
    xf, xhatf, rstdf := LayerNorm(x, m.LNfg, m.LNfb)
    fc.Xhatf, fc.Rstdf, fc.Xf = xhatf, rstdf, xf
    fc.Logits = MM(xf, m.TokE, false, true) // (T×D)·(D×V) = T×V
    return fc
}
```

`SliceCols` 和 `SetCols` 按列切头、拼头，每头 16 列。“多头”的全部实质就是这两个索引操作，没有新的数学。

缓存还是重算

第 1 版前向时丢弃了 LN 的缓存（`xln1, _, _ := LayerNorm(...)`），反向时再对 LN2 重新做一遍前向，文字却说“前向必须缓存 $\hat{x}$ 和 r”。两种做法都对：缓存省计算、费内存；重算省内存、费计算。第 2 版统一改为缓存，与文字一致。生产训练里，“反向时重算一部分前向”有专门的名字：activation checkpointing，是大模型训练节省显存的标准手段。

验收：因果性测试。 梯度检验只能证明“反向与前向一致”，证明不了“前向本身是对的”。如果因果掩码写错了，前向和反向会一起错，而且错得自洽，梯度检验照样通过（第 6.6 节有实测）。能抓住这类 bug 的是一个行为测试：改动第 10 个位置的输入，位置 0 到 9 的 logits 必须一个比特都不变。

```
// 因果性测试：改动位置 t+1 之后的输入，位置 0..t 的 logits 必须完全不变。
// 梯度检验抓不到"掩码写错"（前向和反向是自洽的），只有这个测试能抓到。
func TestCausality(t *testing.T) {
    _, ids, cfg := testSetup(t)
    for _, bug := range []Bug{BugNone, BugNoMask} {
        m := NewModel(cfg, rand.New(rand.NewSource(1)))
        m.Bug = bug
        a := append([]int{}, ids[:cfg.T]...)
        b := append([]int{}, a...)
        b[10] = (b[10] + 1) % cfg.V // 只改第 10 个位置
        la, lb := m.Forward(a).Logits, m.Forward(b).Logits
        leak := 0.0
        for i := 0; i < 10*cfg.V; i++ { // 位置 0..9
            leak = math.Max(leak, math.Abs(la.D[i]-lb.D[i]))
        }
        if bug == BugNone && leak != 0 {
            t.Fatalf("正常模型泄露了未来信息：%.2e", leak)
        }
        if bug == BugNoMask && leak == 0 {
            t.Fatal("去掉掩码后应当能检测到泄露")
        }
        t.Logf("bug=%d 位置 0..9 的 logits 最大变化 %.2e", bug, leak)
    }
}
```

```
bug=0 位置 0..9 的 logits 最大变化 0.00e+00      正常模型：完全不变
bug=2 位置 0..9 的 logits 最大变化 2.58e-02      去掉掩码：前面的位置"看到"了后面的改动
```

4.6 M5：模型定义与参数注册表

没有 autograd，就要自己维护“参数 ↔ 梯度”的对应关系。`reg` 每注册一个参数，就同时分配一个同形状的梯度矩阵，并记下名字。名字用于第 6 章与 PyTorch 对齐、第 8 章加载权重。

```
// Config 模型超参数。默认值见 DefaultConfig。
type Config struct {
    V      int     // 词表大小（由语料决定）
    D      int     // d_model：嵌入/隐藏维度
    NHead  int     // 注意力头数
    NLayer int     // Transformer Block 层数
    DFF    int     // FFN 隐藏维
    T      int     // block_size：上下文窗口长度
    Std    float64 // 权重初始化标准差
}
```

```
// Model 没有 autograd，我们自己维护"参数 ↔ 梯度"的一一对应（参数注册表）。
type Model struct {
    Cfg        Config
    Bug        Bug
    TokE, PosE *Mat // V×D, T×D；TokE 同时充当输出投影（weight tying）
    Layers     []*Layer
    LNfg, LNfb *Mat
```

```
    Params     []*Mat
    Names      []string
    Grads      []*Mat
    idx        map[*Mat]int
}
```

```
func (m *Model) reg(name string, p *Mat) *Mat {
    m.idx[p] = len(m.Params)
    m.Params = append(m.Params, p)
    m.Names = append(m.Names, name)
    m.Grads = append(m.Grads, NewMat(p.R, p.C))
    return p
}
```

```
// NewModel 初始化：权重 ~ N(0, Std²)，偏置为 0，LN 的 γ=1、β=0。
func NewModel(cfg Config, rng *rand.Rand) *Model {
    if cfg.D%cfg.NHead != 0 {
        panic("D 必须能被 NHead 整除")
    }
    m := &Model{Cfg: cfg, idx: map[*Mat]int{}}
    D, s := cfg.D, cfg.Std
    m.TokE = m.reg("tok", randn(cfg.V, D, s, rng))
    m.PosE = m.reg("pos", randn(cfg.T, D, s, rng))
    for l := 0; l < cfg.NLayer; l++ {
        p := fmt.Sprintf("l%d.", l)
        ly := &Layer{}
        ly.LN1g, ly.LN1b = m.reg(p+"ln1.g", ones(1, D)), m.reg(p+"ln1.b", NewMat(1, D))
        ly.Wq, ly.Bq = m.reg(p+"wq", randn(D, D, s, rng)), m.reg(p+"bq", NewMat(1, D))
        ly.Wk, ly.Bk = m.reg(p+"wk", randn(D, D, s, rng)), m.reg(p+"bk", NewMat(1, D))
        ly.Wv, ly.Bv = m.reg(p+"wv", randn(D, D, s, rng)), m.reg(p+"bv", NewMat(1, D))
        ly.Wo, ly.Bo = m.reg(p+"wo", randn(D, D, s, rng)), m.reg(p+"bo", NewMat(1, D))
        ly.LN2g, ly.LN2b = m.reg(p+"ln2.g", ones(1, D)), m.reg(p+"ln2.b", NewMat(1, D))
        ly.W1, ly.B1 = m.reg(p+"w1", randn(D, cfg.DFF, s, rng)), m.reg(p+"b1", NewMat(1, cfg.DFF))
        ly.W2, ly.B2 = m.reg(p+"w2", randn(cfg.DFF, D, s, rng)), m.reg(p+"b2", NewMat(1, D))
        m.Layers = append(m.Layers, ly)
    }
    m.LNfg, m.LNfb = m.reg("lnf.g", ones(1, D)), m.reg("lnf.b", NewMat(1, D))
    return m
}
```

验收：

```
$ go test -run 'TestParamCount|TestInitLoss' -v
model_test.go:26: V=289，参数量 26912（每层 8544）
model_test.go:38: 初始 loss 5.6586，ln V = 5.6664
```

`TestParamCount` 用 1.2 节的公式手算参数量并与程序比较；`TestInitLoss` 在 16 个随机窗口上算初始 loss，要求与 $\ln V$ 相差小于 0.05。

4.7 M6：反向传播与 Adam

对照 3.11 节那条反向链逐行翻译。标 ★ 的四处是最容易写错的地方：两处残差、QKV 三路累加、嵌入 scatter-add。

```
// Backward 单窗口反向：把 ∂L/∂θ 累加进 grads（与 m.Params 一一对应）。
// 对照第 3.5 节的反向链逐行翻译。grads 传 m.Grads 即为默认行为。
func (m *Model) Backward(fc *FwdCache, dlogits *Mat, grads []*Mat) {
    c := m.Cfg
    T, D, dh := len(fc.Ids), c.D, c.Dh()
    g := func(p *Mat) *Mat { return grads[m.idx[p]] }

    // logits = Xf·TokEᵀ：TokE 的第一份梯度（weight tying）
    AddInplace(g(m.TokE), MM(dlogits, fc.Xf, true, false)) // (V×T)·(T×D)
    dxf := MM(dlogits, m.TokE, false, false)               // (T×V)·(V×D)
    dx := LNBackward(dxf, fc.Xhatf, m.LNfg, fc.Rstdf, g(m.LNfg), g(m.LNfb))
```

```
    scale := 1.0 / math.Sqrt(float64(dh))
    for li := c.NLayer - 1; li >= 0; li-- { // 逆序过层
        ly, lc := m.Layers[li], fc.Layers[li]

        // ---- FFN 分支：F = ReLU(Xln2·W1 + b1)·W2 + b2 ----
        AddInplace(g(ly.B2), ColSum(dx))
        AddInplace(g(ly.W2), MM(lc.U, dx, true, false))
        du := MM(dx, ly.W2, false, true)
        dH := ReLUBackward(du, lc.H)
        AddInplace(g(ly.B1), ColSum(dH))
        AddInplace(g(ly.W1), MM(lc.Xln2, dH, true, false))
        dxln2 := MM(dH, ly.W1, false, true)
        dxFfn := LNBackward(dxln2, lc.Xhat2, ly.LN2g, lc.Rstd2, g(ly.LN2g), g(ly.LN2b))

        // ★ 残差 2：X = X' + F  ⇒  ∂L/∂X' = dx（直连） + dxFfn（分支），各加一次
        dx2 := dx.Clone()
        if m.Bug == BugResidualDouble {
            AddInplace(dx2, dx) // 故意制造的 bug：直连梯度多加了一份
        }
        AddInplace(dx2, dxFfn)

        // ---- 注意力分支：A = Cat·Wo + bo ----
        AddInplace(g(ly.Bo), ColSum(dx2))
        AddInplace(g(ly.Wo), MM(lc.Cat, dx2, true, false))
        dcat := MM(dx2, ly.Wo, false, true)

        dq, dk, dv := NewMat(T, D), NewMat(T, D), NewMat(T, D)
        for h := 0; h < c.NHead; h++ {
            hc := lc.Heads[h]
            dC := SliceCols(dcat, h*dh, (h+1)*dh)
            dP := MM(dC, hc.V, false, true)               // dP = dC·Vᵀ
            SetCols(dv, MM(hc.P, dC, true, false), h*dh)  // dV = Pᵀ·dC
            dS := SoftmaxBackward(dP, hc.P)               // 对缩放后分数的梯度
            Scale(dS, scale)                              // 乘 1/√dh：变为对 Q·Kᵀ 的梯度（只乘这一次）
            SetCols(dq, MM(dS, hc.K, false, false), h*dh) // dQ = dS·K
            SetCols(dk, MM(dS, hc.Q, true, false), h*dh)  // dK = dSᵀ·Q
        }

        // QKV 三个投影：结构完全相同
        AddInplace(g(ly.Bq), ColSum(dq))
        AddInplace(g(ly.Bk), ColSum(dk))
        AddInplace(g(ly.Bv), ColSum(dv))
        AddInplace(g(ly.Wq), MM(lc.Xln1, dq, true, false))
        AddInplace(g(ly.Wk), MM(lc.Xln1, dk, true, false))
        AddInplace(g(ly.Wv), MM(lc.Xln1, dv, true, false))
        // ★ Xln1 同时喂给 Q、K、V，梯度是三条链之和
        dxln1 := MM(dq, ly.Wq, false, true)
        AddInplace(dxln1, MM(dk, ly.Wk, false, true))
        AddInplace(dxln1, MM(dv, ly.Wv, false, true))
        dxAttn := LNBackward(dxln1, lc.Xhat1, ly.LN1g, lc.Rstd1, g(ly.LN1g), g(ly.LN1b))

        // ★ 残差 1：X' = X + A  ⇒  ∂L/∂X = dx2 + dxAttn
        AddInplace(dx2, dxAttn)
        dx = dx2
    }

    // ★ 嵌入的反向是 scatter-add：哪一行被查过，梯度就加回哪一行
    gT, gP := g(m.TokE), g(m.PosE) // TokE 的第二份梯度
    for t, id := range fc.Ids {
        for j := 0; j < D; j++ {
            gT.D[id*D+j] += dx.D[t*D+j]
            gP.D[t*D+j] += dx.D[t*D+j]
        }
    }
}
```

函数签名里的 `grads []*Mat` 是第 2 版加的：梯度写进调用方给的缓冲区，而不是固定写进 `m.Grads`。这样第 8.3 节做并行训练时，每个 goroutine 可以有自己的一套梯度，互不干扰。

`m.Bug` 那一行是为第 6 章准备的“可控 bug”：打开后，残差 2 处会把直连梯度多加一份。正常训练时它不生效。

Adam 的实现：

```
type Adam struct {
    LR, B1, B2, Eps float64
    BiasCorrection  bool // 关掉它用于第 3.7 节的对照实验
    t               int
    m, v            []*Mat
}
```

```
func (a *Adam) Step(params, grads []*Mat) {
    a.t++
    bc1, bc2 := 1.0, 1.0
    if a.BiasCorrection {
        bc1 = 1 - math.Pow(a.B1, float64(a.t))
        bc2 = 1 - math.Pow(a.B2, float64(a.t))
    }
    for i, p := range params {
        g, m, v := grads[i].D, a.m[i].D, a.v[i].D
        for j := range p.D {
            m[j] = a.B1*m[j] + (1-a.B1)*g[j]
            v[j] = a.B2*v[j] + (1-a.B2)*g[j]*g[j]
            mh, vh := m[j]/bc1, v[j]/bc2
            p.D[j] -= a.LR * mh / (math.Sqrt(vh) + a.Eps)
        }
    }
}
```

验收一：全模型梯度检验。 每个参数矩阵随机抽 10 个元素，要求最大相对误差小于 10^{-4}（判据的来由见 3.13 节）。完整报告用 `go run . gradcheck -per 50` 打印，第 6.5 节会对比正确实现与 bug 版的报告。

验收二：单 batch 过拟合。 固定一个 batch，反复训练 200 步，loss 必须降到 0.05 以下。这是性价比最高的检查：任何正确实现的模型都应该能把一小撮数据背下来，做不到就说明反向、优化器或学习率至少有一处有问题。实测 200 步后 loss 为 0.0163。

```
func TestOverfitOneBatch(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    xs, ys := GetBatch(ids, 4, cfg.T, rand.New(rand.NewSource(4)))
    adam := NewAdam(m, 3e-3)
    var loss float64
    for s := 1; s <= 200; s++ {
        loss = m.LossAndGrads(xs, ys, 1)
        adam.Step(m.Params, m.Grads)
    }
    if loss > 0.05 {
        t.Fatalf("固定一个 batch 训练 200 步后 loss=%.4f，应当 < 0.05", loss)
    }
    t.Logf("单 batch 200 步后 loss = %.4f", loss)
}
```

三个自检问题

1. **为什么 E_{tok} 的梯度在函数开头和结尾各加一次？** weight tying：E_{tok} 既是输出投影，又是输入嵌入，两个计算节点共享同一份参数，两份梯度相加。
2. **残差 2 处为什么写 `dx2 := dx.Clone()`，而不是直接 `AddInplace(dx, dxFfn)`？** 执行到这里时 FFN 分支已经反传完，之后不会再读 `dx`，所以直接原地相加也是对的。`Clone` 是防御性写法：避免切片别名在后续修改时误改 `dx`，也方便插入 bug 开关。真正必须保证的是“直连梯度与分支梯度各加一次”。（第 1 版的答案说“dx 还要参与 FFN 分支的反传，必须先复制”，这个理由不成立。）
3. **`dxln1` 为什么要累加三个矩阵乘？** Q、K、V 都由 $\tilde{X}$ 线性变换而来，$\tilde{X}$ 的梯度是三条链之和。

4.8 M7：训练循环

一个训练步：取 B 个窗口，逐个窗口前向、反向（梯度累加），最后统一除以 B 取平均，Adam 走一步。`workers` 大于 1 时按窗口并行，第 8.3 节详细讨论。

```
// LossAndGrads 对 B 个窗口做前向+反向，梯度写入 m.Grads 并按 B 取平均，返回平均 loss。
// workers>1 时按窗口并行：每个 worker 一套私有梯度，最后归约（单机版"数据并行 + AllReduce"）。
func (m *Model) LossAndGrads(xs, ys [][]int, workers int) float64 {
    B := len(xs)
    m.ZeroGrads()
    var loss float64
    if workers <= 1 {
        for b := 0; b < B; b++ {
            fc := m.Forward(xs[b])
            l, dl := CrossEntropy(fc.Logits, ys[b])
            loss += l
            m.Backward(fc, dl, m.Grads)
        }
    } else {
        local := make([][]*Mat, workers)
        losses := make([]float64, workers)
        var wg sync.WaitGroup
        for w := 0; w < workers; w++ {
            wg.Add(1)
            go func(w int) {
                defer wg.Done()
                local[w] = m.NewGrads()
                for b := w; b < B; b += workers {
                    fc := m.Forward(xs[b])
                    l, dl := CrossEntropy(fc.Logits, ys[b])
                    losses[w] += l
                    m.Backward(fc, dl, local[w])
                }
            }(w)
        }
        wg.Wait()
        for w := 0; w < workers; w++ { // 归约：所有 worker 的梯度求和
            loss += losses[w]
            for i := range m.Grads {
                AddInplace(m.Grads[i], local[w][i])
            }
        }
    }
    for _, g := range m.Grads {
        Scale(g, 1/float64(B))
    }
    return loss / float64(B)
}
```

主循环（`main.go` 中的 `cmdTrain`）：

```
        fmt.Fprintln(logf, "step,loss")
    }
    start := time.Now()
    for step := 1; step <= *steps; step++ {
        xs, ys := GetBatch(ids, *B, cfg.T, dataRng)
        loss := m.LossAndGrads(xs, ys, *workers)
        if *opt == "sgd" {
            SGDStep(m.Params, m.Grads, *lr)
        } else {
            adam.Step(m.Params, m.Grads)
        }
        if logf != nil {
            fmt.Fprintf(logf, "%d,%.6f\n", step, loss)
        }
        if step == 1 || step%*every == 0 {
            fmt.Printf("step %4d  loss %.4f\n", step, loss)
```

完整输出（`make train-go`，种子 1）：

```
语料 628 字符, 词表 V=289
参数量: 26912 (d=32, 2层, 2头, T=16)
理论初始 loss ln(V) = 5.6664
step    1  loss 5.6868        ← 与 ln V 相差 0.02：前向与损失没写错的第一个信号
step   50  loss 4.5531
step  100  loss 3.6374
step  150  loss 2.9971
step  200  loss 2.6407
step  250  loss 1.9273
step  300  loss 1.2797
step  350  loss 1.1487
step  400  loss 1.1718        ← 单步 loss 有涨有跌，这是 batch=4 的采样噪声
step  450  loss 0.8417
step  500  loss 0.5107
step  550  loss 0.5349
step  600  loss 0.3702
训练耗时: 1.40s
全部 612 个窗口的平均 loss: 0.4701
--- 生成样例（temp=0.8, top-k=10）---
"月落知多少。\n白日依山出，黄河在，城春眠不教胡马度阴山。\n寒雨声"
"春风不度门关。\n日照炉生紫烟，遥看瀑布挂前川。飞流直下三千尺，三"
"两岸猿声啼不住，轻舟已过万重山。\n故故里江枫火对愁眠不见人。姑苏"
```

单步 loss 只基于 4 个窗口，波动很大；最后一行“全部 612 个窗口的平均 loss”才是稳定的指标，本书比较不同配置时都用它。生成样例用 `%q` 打印，`\n` 是诗与诗之间的换行符。

4.9 M8：采样与生成

自回归生成：每一步把最近 T 个字喂进模型，取最后一个位置的 logits，除以温度，可选只保留概率最高的 k 个，softmax 后按概率抽一个字，拼回序列。

```
// Sample 从一行 logits 中采样：temp=0 取 argmax；topK>0 时只在概率最高的 K 个里采样。
func Sample(logits []float64, temp float64, topK int, rng *rand.Rand) int {
    V := len(logits)
    if temp == 0 {
        best := 0
        for i, v := range logits {
            if v > logits[best] {
                best = i
            }
        }
        return best
    }
    idx := make([]int, V)
    for i := range idx {
        idx[i] = i
```

```
    }
    if topK > 0 && topK < V {
        sort.Slice(idx, func(a, b int) bool { return logits[idx[a]] > logits[idx[b]] })
        idx = idx[:topK]
    }
    mx := math.Inf(-1)
    for _, i := range idx {
        mx = math.Max(mx, logits[i]/temp)
    }
    probs := make([]float64, len(idx))
    var sum float64
    for k, i := range idx {
        probs[k] = math.Exp(logits[i]/temp - mx)
        sum += probs[k]
    }
    r := rng.Float64() * sum // 轮盘赌
    for k, p := range probs {
        r -= p
        if r <= 0 {
            return idx[k]
        }
    }
    return idx[len(idx)-1]
}
```

```
// Generate 自回归生成（无 KV cache）：每生成一个字都把最近 T 个字完整前向一遍。
func (m *Model) Generate(prompt []int, n int, temp float64, topK int, rng *rand.Rand) []int {
    ids := append([]int{}, prompt...)
    T, V := m.Cfg.T, m.Cfg.V
    for i := 0; i < n; i++ {
        ctx := ids
        if len(ctx) > T {
            ctx = ctx[len(ctx)-T:] // 只看最近 T 个
        }
        lg := m.Forward(ctx).Logits
        last := lg.D[(lg.R-1)*V : lg.R*V] // 只用最后一个位置的 logits
        ids = append(ids, Sample(last, temp, topK, rng))
    }
    return ids
}
```

每生成一个字都要把最近 16 个字完整前向一遍，前面位置的 K、V 被反复重算。第 8.1 节用 KV cache 解决这个问题。

温度的影响（同一个训练好的模型，不做 top-k 截断）：

温度	生成样例（提示词"月落"）	特点
0.1	月落乌啼霜霜满天，江枫渔火对愁眠。举头望明月，江天，江枫渔火对愁	几乎总选概率最高的字，接近背诵，容易陷入重复
1.0	月落落九天。朝朝辞白鹭上白云间，西山中断楚江天天际流。	按模型分布采样，有拼接、有错字
2.0	月落间，楼碧径破啼不住，孤松下三千江枫渔最看布挂前灭。	分布被拉平，低概率字频繁出现，基本不成句

4.10 动手实验

实验 4-1 亲手制造一个反向 bug

目的：体会梯度检验的定位能力。

改动：把 `backward.go` 里残差 2 的 `AddInplace(dx2, dxFfn)` 删掉（或用现成的开关：`go run . gradcheck -per 50 -bug residual`）。

预期：全模型最大相对误差从 10^{-5} 量级跳到 1 左右。误差大的参数集中在出错位置的"上游"（更靠近输入的那些层），出错位置之后的参数（第 2 层 FFN、最终 LN）仍然正常。第 6.5 节给出完整报告。

实验 4-2 改变模型规模

命令：`go run . train -d 64` 与 `go run . train -layers 3`

预期（600 步，种子 1）：

配置	参数量	全窗口平均 loss	训练耗时
默认（d=32，2 层）	26,912	0.47	1.40 秒
d=64（d_{ff} 随之变为 128）	86,592	0.34	4.20 秒
3 层	35,456	0.41	2.06 秒

观察：d 翻倍，参数量约变为 3.2 倍（嵌入表随 d 线性增长，Block 内的矩阵随 d^2 增长），耗时约 3 倍。多一层只多 8,544 个参数。在 628 字的语料上，更大的模型背得更快；第 7 章换成大语料后，这个结论会变得不那么简单。

实验 4-3 SGD 与 Adam

命令：`go run . train -opt sgd -lr 3e-3` 与 `go run . train -opt sgd -lr 0.5`

预期：前者 600 步后全窗口 loss 仍为 5.26，几乎没动；后者为 1.05，仍不如 Adam 的 0.47。原因见 3.14 节。

4.11 小结

- Go 版 = 第 3 章公式的逐行翻译。核心工程结构有两个：参数注册表（`Params`/`Grads`/`Names`）和前向缓存（`FwdCache`）。
- 每个里程碑都有测试。算子级梯度检验的误差在 10^{-8} 量级，全模型在 10^{-5} 量级。
- 梯度检验抓不到"前向本身写错"，因果性测试可以；单 batch 过拟合能抓住优化器和学习率的问题。
- 看训练效果用全窗口平均 loss，不要看单步 loss。

第 5 章 PyTorch 实现

5.1 一句话说清

同一个模型，PyTorch 版的模型与训练部分约 100 行，整个文件连同命令行参数和生成约 170 行。省掉的不是模型，而是反向传播和矩阵运算的全部手工劳动。本章逐段给出代码，并回答一个问题：这一行背后，框架替我做了第 4 章里的哪些事？

5.2 数据

逻辑与第 2 章完全一致。两处与第 1 版不同：显式指定 `encoding="utf-8"`（不同系统的默认编码可能不同）；起点范围改为 $N-T$。

```
def load_corpus(path):
    text = open(path, encoding="utf-8").read()     # 显式 utf-8，含换行与标点
    chars = sorted(set(text))                       # 按码点排序，与 Go 版词表 id 完全一致
    stoi = {c: i for i, c in enumerate(chars)}
    ids = torch.tensor([stoi[c] for c in text], dtype=torch.long)
    return text, chars, stoi, ids
```

```
def get_batch(ids, B, T, gen, device):
    ix = torch.randint(0, len(ids) - T, (B,), generator=gen)  # 合法起点共 N-T 个
    x = torch.stack([ids[i:i + T] for i in ix])
    y = torch.stack([ids[i + 1:i + T + 1] for i in ix])
    return x.to(device), y.to(device)
```

`get_batch` 用独立的 `torch.Generator` 取数，不依赖全局随机状态，这样初始化和取数互不干扰，便于复现。

5.3 模型

```
@dataclass
class Config:
    V: int = 289    # 词表大小（由语料决定）
    D: int = 32     # d_model
    NH: int = 2     # 头数
    NL: int = 2     # 层数
    DFF: int = 64   # FFN 隐藏维
    T: int = 16     # block_size
```

```
class Block(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.c = c
        self.ln1 = nn.LayerNorm(c.D)
        self.qkv = nn.Linear(c.D, 3 * c.D)   # Wq/Wk/Wv 合并成一个投影
        self.proj = nn.Linear(c.D, c.D)      # 即 Go 版的 Wo
        self.ln2 = nn.LayerNorm(c.D)
        self.ffn = nn.Sequential(nn.Linear(c.D, c.DFF), nn.ReLU(), nn.Linear(c.DFF, c.D))

    def forward(self, x):
        B, T, D = x.shape
        nh, dh = self.c.NH, D // self.c.NH
        q, k, v = self.qkv(self.ln1(x)).split(D, dim=2)
        q = q.view(B, T, nh, dh).transpose(1, 2)        # (B, nh, T, dh)：切头
        k = k.view(B, T, nh, dh).transpose(1, 2)
        v = v.view(B, T, nh, dh).transpose(1, 2)
        o = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

```
        o = o.transpose(1, 2).contiguous().view(B, T, D)  # 拼回
        x = x + self.proj(o)                              # 残差 1
        x = x + self.ffn(self.ln2(x))                     # 残差 2
        return x
```

逐行对照 Go 版：

- `nn.Linear(D, 3*D)`：把 W_q、W_k、W_v 合并成一个 32×96 的投影再切开。数学上等价，少两次矩阵乘的调用开销，是生产代码的标准写法。注意 `nn.Linear` 存的权重是 $W^\top$（形状为 输出×输入），第 6 章做对齐时要转置。
- `view(B, T, nh, dh).transpose(1, 2)`：对应 Go 的 `SliceCols`，但它只是改变张量的“视图”，不拷贝数据。
- `F.scaled_dot_product_attention(q, k, v, is_causal=True)`：一次调用完成 $QK^\top$、缩放、因果掩码、softmax、乘 V，对应 Go 版 `Forward` 里注意力部分约 15 行和 `Backward` 里约 15 行。切头和拼头仍由前后的 `view`/`transpose` 完成。在 GPU 上，它会调度到 FlashAttention 一类的融合算子。
- 反向传播：零行。

```
class GPT(nn.Module):
    def __init__(self, c: Config, init="nanogpt"):
        super().__init__()
        self.c = c
        self.tok = nn.Embedding(c.V, c.D)
        self.pos = nn.Embedding(c.T, c.D)
        self.blocks = nn.ModuleList([Block(c) for _ in range(c.NL)])
        self.lnf = nn.LayerNorm(c.D)
        if init == "nanogpt":
            self.apply(self._init_weights)  # 不调用则使用 PyTorch 默认初始化（见 6.3 节）

    @staticmethod
    def _init_weights(m):
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)
            nn.init.zeros_(m.bias)
        elif isinstance(m, nn.Embedding):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        for blk in self.blocks:
            x = blk(x)
        return self.lnf(x) @ self.tok.weight.T            # weight tying
```

`self.lnf(x) @ self.tok.weight.T` 就是 Go 版的 `MM(xf, m.TokE, false, true)`。autograd 会自动把这条路径的梯度累加进 `tok.weight`，等价于 Go 版里对 E_{tok} 的两次 `AddInplace`。

`torch.arange(T, device=idx.device)`：位置编号必须和输入在同一个设备上。第 1 版写的是 `torch.arange(t)`，默认在 CPU，模型搬到 GPU 后会报设备不一致的错误。

5.4 初始化

`_init_weights` 不是摆设。去掉它，PyTorch 会让 `nn.Embedding` 使用默认初始化 $\mathcal{N}(0,1)$，比 nanoGPT 惯例的 0.02 大 50 倍。实测 step 1 的 loss 从 5.69 变成 21.0，600 步后全窗口 loss 停在 2.93。为什么会这样、为什么问题出在 weight tying 上，第 6.4 节用方差推导和对照实验讲清楚。

5.5 训练、评估与生成

```
def train(model, ids, steps=600, B=4, lr=3e-3, seed=1, device="cpu", log=None, every=50):
    gen = torch.Generator().manual_seed(seed + 1000)
    opt = torch.optim.Adam(model.parameters(), lr=lr, betas=(0.9, 0.999), eps=1e-8)
    V, T, losses = model.c.V, model.c.T, []
    for step in range(1, steps + 1):
        x, y = get_batch(ids, B, T, gen, device)
        logits = model(x)
        loss = F.cross_entropy(logits.view(-1, V), y.view(-1))
        opt.zero_grad()
        loss.backward()   # ← Go 版的整个 Backward
        opt.step()        # ← Go 版的 Adam.Step
        losses.append(loss.item())
        if log and (step == 1 or step % every == 0):
            log(f"step {step:4d}  loss {loss.item():.4f}")
    return losses
```

`loss.backward()` 这一行展开，是 autograd 沿前向时记录的计算图反向遍历，对每个算子调用它注册的反向函数。第 4 章手写的每一个反向公式，PyTorch 里都有一份对应的实现。

```
@torch.no_grad()
def eval_all(model, ids):
    """全部窗口上的平均 loss，与 Go 版的同名统计口径一致。"""
    T = model.c.T
    xs = torch.stack([ids[i:i + T] for i in range(len(ids) - T)])
    ys = torch.stack([ids[i + 1:i + T + 1] for i in range(len(ids) - T)])
    dev = next(model.parameters()).device
    logits = model(xs.to(dev))
    return F.cross_entropy(logits.reshape(-1, model.c.V), ys.to(dev).reshape(-1)).item()
```

```
@torch.no_grad()
def generate(model, prompt_ids, n, temp=0.8, top_k=10, gen=None):
    idx = torch.tensor([prompt_ids], dtype=torch.long, device=next(model.parameters()).device)
    for _ in range(n):
        logits = model(idx[:, -model.c.T:])[0, -1] / temp   # 只看最近 T 个、只取最后一个位置
        if top_k:
            kth = torch.topk(logits, top_k).values[-1]
            logits[logits < kth] = -float("inf")
        nxt = torch.multinomial(F.softmax(logits, dim=-1), 1, generator=gen)
        idx = torch.cat([idx, nxt.view(1, 1)], dim=1)
    return idx[0].tolist()
```

`@torch.no_grad()` 告诉 autograd 不要记录计算图：推理时不需要反向，省下保存中间激活的内存。`idx[:, -T:]` 与 Go 版“只看最近 T 个字”同义，也同样没有 KV cache。

5.6 实测输出

```
$ python nanogpt.py --log ../out/py_s1.csv
语料 628 字符，词表 V=289
参数量：26912 (d=32, 2层, 2头, T=16)        ← 与 Go 版一致
理论初始 loss ln(V) = 5.6664
step    1  loss 5.6863
step  100  loss 3.6578
step  200  loss 2.3128
step  300  loss 1.2015
step  400  loss 1.0105
step  500  loss 0.4859
step  600  loss 0.3122
训练耗时：1.65s
全部 612 个窗口的平均 loss：0.4137
--- 生成样例（temp=0.8, top-k=10）---
"月落笠翁，粒皆辛苦。\n月落九天。\n月花三月下扬州。孤帆远影碧空尽"
"春风雨浥轻尘，客舍青青青柳色新。劝君更尽一杯酒，西出阳关无故人。"
"两岸青柳色新。劝君更月复照香炉生南国，春来发几几枝。愿君多采烟，"
```

与 Go 版相比，单步 loss 的轨迹不同（初始权重和每步抽到的窗口都不同），全窗口 loss 一个是 0.41、一个是 0.47。这个差别说明不了谁对谁错，第 6.3 节会用 5 个随机种子说明：两套实现各自的种子间波动（约 ±0.05）比它们之间的差别大。

5.7 设备与性能

把模型放到 GPU 上需要三处一致：模型参数、输入张量、在 `forward` 里新建的张量（这里是位置编号）。本书的写法是：

```
model = GPT(c).to(device)              # 参数
x, y = get_batch(ids, B, T, gen, device)  # 输入在 get_batch 里 .to(device)
torch.arange(T, device=idx.device)     # forward 里新建的张量跟随输入的设备
```

第 1 版说“加一行 `model.cuda()` 就能零改动加速”，实际会因为后两处报错。

本机实测每个训练步的耗时（`py/speed.py` 与 Go 版 `go run . speed`）：

实现	B=4	B=32	B 变为 8 倍时耗时变为
Go，单 goroutine	2.46 ms	19.29 ms	7.8 倍
PyTorch，CPU 单线程	0.66 ms	1.57 ms	2.4 倍
PyTorch，Apple GPU（MPS）	4.03 ms	4.10 ms	1.0 倍

三个观察：

1. Go 版的耗时与 B 成正比，因为它逐个窗口循环。PyTorch 把整个 batch 叠成一个张量，一次矩阵乘处理所有窗口，单线程下也能用上向量化指令，B 增大 8 倍耗时只增加 1.4 倍。
2. 在这么小的模型上，GPU 反而最慢：每一步要启动几十个 kernel，启动开销（毫秒级）远大于计算本身。GPU 的优势要在矩阵足够大时才体现，第 7 章在 64 万参数、每步 2,048 个 token 的设置下就换成了 GPU。
3. Go 版用 goroutine 按窗口并行后，B=32 时可以从 19.3 ms 降到 4.9 ms（第 8.3 节）。

5.8 两种技术栈的分工

维度	Go 手写版	PyTorch 版
代码量	核心约 900 行，测试约 400 行	约 170 行（模型与训练约 100 行）
反向传播	手写约 80 行，配梯度检验	`loss.backward()`
数值精度	float64	float32（生产中常用 bf16 混合精度）
优化器	手写 Adam	`torch.optim.Adam`（数学上与 Go 版逐元素一致，见 6.2 节）
硬件	CPU；可用 goroutine 并行	CPU / GPU，改设备即可
擅长	看清每个梯度；推理部署（第 8 章）	快速实验；扩展到大规模训练

两个版本不是竞争关系。Go 版让你看见反向传播，PyTorch 版让你放心地不再去看它。

5.9 动手实验

实验 5-1 去掉初始化

命令：`python nanogpt.py --init default`

预期：step 1 的 loss 约 21.0，100 步时 5.91，600 步时 3.05，全窗口 loss 2.93（对照：正常初始化为 0.41）。原因见第 6.4 节。

实验 5-2 ReLU 换成 GELU

改动：`nn.ReLU()` 换成 `nn.GELU()`（nanoGPT 原版的选择）。`experiments.py` 已经做好了这组对照，各跑 5 个种子。

预期：

激活函数	5 个种子的全窗口 loss	均值 ± 标准差
ReLU	0.414, 0.508, 0.403, 0.421, 0.383	0.426 ± 0.048
GELU	0.356, 0.469, 0.395, 0.500, 0.387	0.421 ± 0.061

观察：均值差 0.005，远小于种子间的标准差。在这个规模上看不出 GELU 的优势。如果只跑一个种子就下结论，种子 1 会让你以为 GELU 好了 0.06，种子 4 会让你以为它差了 0.08。比较两个配置时，至少要跑几个种子。

5.10 小结

- PyTorch 版与 Go 版逐组件对应：嵌入加位置、Pre-LN Block、因果多头注意力、weight tying、Adam，参数量都是 26,912。
- 框架替你做的：反向传播、算子融合、批量向量化、设备迁移。框架替不了的：初始化约定、超参数、设备一致性，以及 loss 不正常时去哪里查。
- 比较两个配置要跑多个种子，单次结果的差异可能只是噪声。

第 6 章 跨实现对齐与排错

6.1 一句话说清

“loss 在降”证明不了实现是对的，“两条 loss 曲线差不多”也证明不了。本章先把验证手段按证明力排个序，然后做四组实验：跨实现逐元素对齐，以及三个可复现的坑。第三个坑（去掉因果掩码）的 loss 比正确模型还低，生成的却是乱码。

检查	能证明什么	证明不了什么	本书结果
初始 loss $\approx \ln V$	前向与损失大体正确，初始化正常	反向是否正确	5.69 vs 5.67
参数量 = 手算值	结构、维度、权重共享都对	计算是否正确	26,912
单 batch 过拟合	反向与优化器“大方向”对	梯度是否精确	200 步到 0.016
数值梯度检验	反向与前向**一致**	前向本身是否正确（如掩码）	10^{-5} 量级
因果性测试	前向没有泄露未来信息	其他前向错误	变化恰为 0
跨实现逐元素对齐	两份独立实现在数学上等价	两边犯了同一个概念性错误	10^{-15} 量级
loss 曲线对照	弱证据：两边都“能训练”	几乎什么都证明不了（6.3 节）	—

第 1 版把最后一行当作“最强证据”。下面用实验说明它为什么应该排在最后。

6.2 跨实现逐元素对齐

做法：Go 先训练 50 步（让 γ、β、偏置都离开初始值，比对更有说服力），然后导出以下内容：全部参数、一个固定的 batch、这个 batch 上的 logits 与 loss、全部梯度、用一个新建的 Adam 走一步之后的参数。PyTorch 加载同样的参数，在 float64 下算同样的量，逐元素求最大差。

```
cd go && go run . align          # 导出 ../out/align.json（约 2 MB）
cd ../py && python align.py      # 加载、计算、比对
```

加载时要处理两处布局差异：`nn.Linear` 的权重是 $W^\top$；PyTorch 的 QKV 是合并的一个 96×32 矩阵，要把 Go 的三个 32×32 矩阵转置后按行拼起来。

```
@torch.no_grad()
def load_from_go(model, d, c):
    P = {n: torch.tensor(v).view(*s) for n, s, v in zip(d["names"], d["shapes"], d["params"])}
    model.tok.weight.copy_(P["tok"])
    model.pos.weight.copy_(P["pos"])
    for l, b in enumerate(model.blocks):
        p = f"l{l}."
        b.ln1.weight.copy_(P[p + "ln1.g"].view(-1)); b.ln1.bias.copy_(P[p + "ln1.b"].view(-1))
        b.qkv.weight.copy_(torch.cat([P[p + "wq"].T, P[p + "wk"].T, P[p + "wv"].T], dim=0))
        b.qkv.bias.copy_(torch.cat([P[p + "bq"], P[p + "bk"], P[p + "bv"]], dim=1).view(-1))
        b.proj.weight.copy_(P[p + "wo"].T); b.proj.bias.copy_(P[p + "bo"].view(-1))
        b.ln2.weight.copy_(P[p + "ln2.g"].view(-1)); b.ln2.bias.copy_(P[p + "ln2.b"].view(-1))
        b.ffn[0].weight.copy_(P[p + "w1"].T); b.ffn[0].bias.copy_(P[p + "b1"].view(-1))
        b.ffn[2].weight.copy_(P[p + "w2"].T); b.ffn[2].bias.copy_(P[p + "b2"].view(-1))
    model.lnf.weight.copy_(P["lnf.g"].view(-1)); model.lnf.bias.copy_(P["lnf.b"].view(-1))
```

实测结果：

精度	logits	loss	全部 36 个梯度矩阵	Adam 一步后的参数
float64	2.2×10^{-15}	8.9×10^{-16}	1.4×10^{-16}	2.7×10^{-13}
float32	7.2×10^{-7}	6.7×10^{-7}	1.2×10^{-7}	1.1×10^{-4}

float64 下所有误差都在机器精度附近，两套实现在数学上等价。Go 侧前向、反向、Adam 全部手写，PyTorch 侧全部用框架，没有共享任何代码，这是比“曲线相似”强得多的证据。

float32 那一行有一个值得解释的数字：Adam 一步后参数差 1.1×10^{-4}，比梯度的误差大了三个数量级。误差最大的参数是 `l0.bk`，也就是 K 的偏置。

K 的偏置梯度在数学上恒为 0。 给所有 key 加同一个偏置 b_k，第 t 行的每个分数都加上同一个常数 $q_t\cdot b_k$，而 softmax 对整行加常数不变。所以 b_k 不影响输出，梯度严格为 0。float64 下算出来是 10^{-20} 量级的舍入噪声，float32 下是 10^{-9} 量级。Adam 第一步的更新约为 $\eta\cdot g/(|g|+\varepsilon)$，$\varepsilon=10^{-8}$：当 $|g|$ 和 ε 同一量级时，g 的微小差别会被放大成 10^{-4} 量级的参数差。这是 Adam 在零梯度附近的数值特性，不是实现错误。顺带一提，很多实现（包括一些主流开源模型）干脆不给 K 加偏置，原因正在于此。

6.3 曲线对照为什么只能当辅助证据

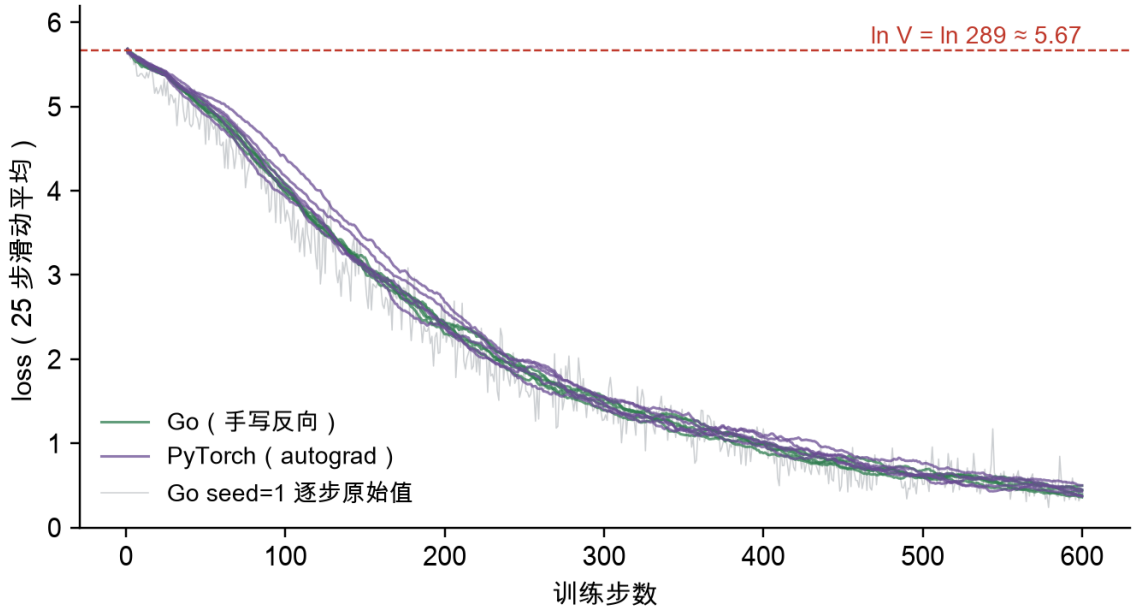


图 6-1 Go 与 PyTorch 各 5 个随机种子的 loss 曲线（25 步滑动平均），灰色为 Go 种子 1 的逐步原始值

10 条曲线交织在一起，看起来很有说服力。但请看三组数字：

	Go（5 个种子）	PyTorch（5 个种子）
step 600 的单步 loss	0.370, 0.369, 0.574, 0.236, 0.336	0.312, 0.434, 0.488, 0.481, 0.422
step 600 前 25 步平均	0.427 ± 0.053	0.424 ± 0.055
全窗口平均 loss	0.429 ± 0.057	0.426 ± 0.048

同一个实现、不同种子之间的标准差约 0.05；单个种子内，逐步 loss 偏离 25 步均值的幅度平均 0.11、最大 0.59（500 到 588 步）。在这样的噪声水平下，一个让最终 loss 变差 0.05 的 bug 是看不出来的。下面这张图就是证据：

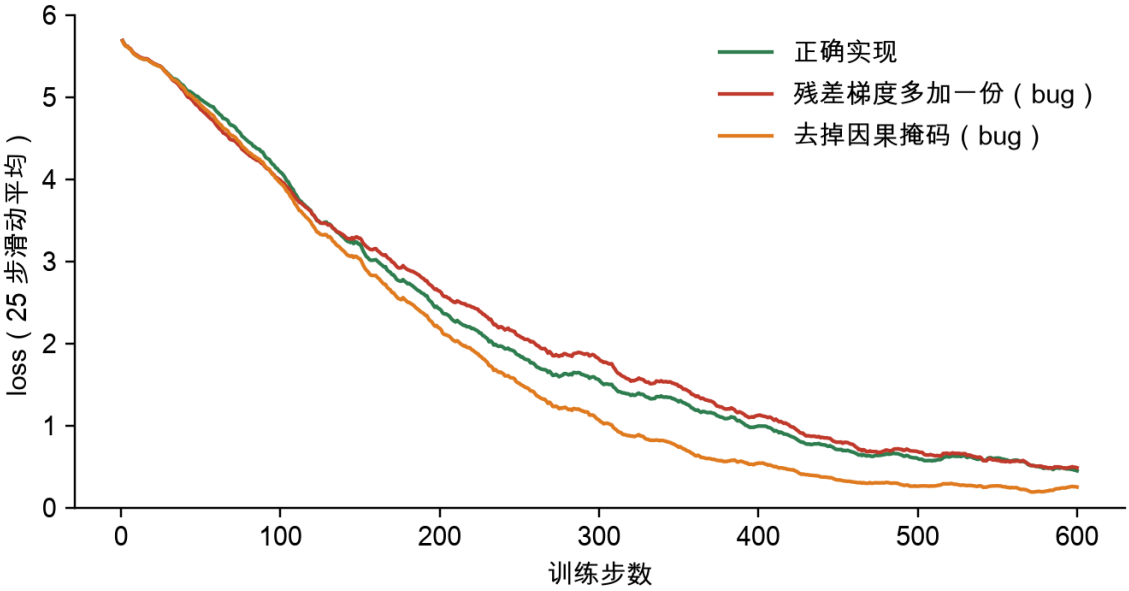


图 6-2 Go 版的正确实现与两个故意制造的 bug（25 步滑动平均）

版本	step 600 前 25 步平均	全窗口平均 loss	梯度检验	因果性测试	生成
正确实现	0.447	0.470	9.3×10^{-6}	通过	能复写、能拼接
残差梯度多加一份	0.484	0.495	1.0	通过	能复写、能拼接
去掉因果掩码	0.250	0.227	1.6×10^{-5}	失败	乱码

残差 bug 的曲线落在正常种子的波动范围里；掩码 bug 的曲线比正确实现还低。只看 loss 曲线，第一个 bug 发现不了，第二个甚至会被当成“改进”。

6.4 坑一：PyTorch 的默认初始化

现象：去掉 `_init_weights`，PyTorch 版 step 1 的 loss 是 21.0，而理论值是 $\ln 289 = 5.67$。

根因：weight tying 把 $\mathcal{N}(0,1)$ 的嵌入矩阵直接用作了输出投影。 推导如下。最终 LN 的输出 x 每一维大约是零均值、单位方差（$\gamma=1,\beta=0$）。logits 的第 v 个分量是 $z_v=\sum_{j=1}^{d} x_j E_{vj}$，若 $E_{vj}\sim\mathcal{N}(0,\sigma^2)$ 且与 x 独立，则

$$\mathrm{Var}(z_v)=d\,\sigma^2,\qquad \mathrm{std}(z_v)=\sqrt{d}\,\sigma$$

$\sigma=1$ 时 logits 的标准差约为 $\sqrt{32}=5.66$，$\sigma=0.02$ 时约为 0.113。当 logits 是 V 个标准差为 s 的独立正态变量、s 较大时，$\log\sum e^{z}$ 由最大的那个主导，约为 $s\sqrt{2\ln V}$，而目标位置的 logit 平均为 0，所以交叉熵约为 $5.66\times\sqrt{2\ln 289}\approx 19$。

实测与推导吻合，并用一个对照组定位根因（各 5 个种子平均）：

配置	初始 logits 标准差	初始 loss
默认初始化 + weight tying	5.80（理论 5.66）	21.4（估算约 19）
std=0.02 + weight tying	0.114（理论 0.113）	5.67（$\ln V=5.67$）

配置	初始 logits 标准差	初始 loss
默认初始化 + **独立的输出头**（`nn.Linear` 默认初始化）	0.577	5.84

第三行是关键：输入嵌入仍然是 $\mathcal{N}(0,1)$，但输出头换成独立的 `nn.Linear`，初始 loss 就回到了 $\ln V$ 附近。`nn.Linear` 默认用均匀分布 $U(-1/\sqrt{d}, 1/\sqrt{d})$，标准差为 $1/\sqrt{3d} = 0.102$，logits 标准差为 $\sqrt{d} \times 0.102 = 0.577$，与实测一致。可见输入侧的 $\mathcal{N}(0,1)$ 本身无害（后面紧跟着 LN），问题只出在它被拿去当输出投影。

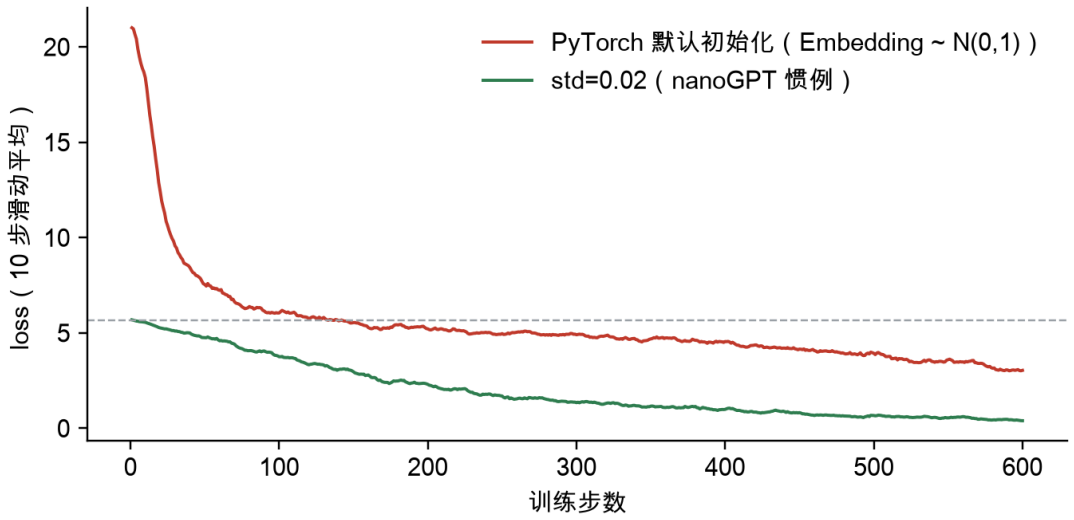


图 6-3 PyTorch 默认初始化与 nanoGPT 惯例初始化（10 步滑动平均）

初始化	step 1	step 100	step 300	step 600	全窗口平均 loss
默认（bug）	21.00	5.91	4.51	3.05	2.93
std=0.02（修复）	5.69	3.66	1.20	0.31	0.41

前 100 步，模型几乎全在“消化”过大的初始 logits，之后也没追上。

教训

跨框架复现时，初始化约定是最常见的隐性差异。排错的第一个检查点永远是：初始 loss 是否约等于 $\ln V$。它零成本，而且信息量很大：偏高说明 logits 太大（初始化或输出层的问题），偏低说明模型在“作弊”（标签泄露）。

第 1 版把原因归结为“默认初始化假设有归一化、有残差的深网”，这个解释不准确：本模型恰恰有归一化和残差。真正的原因是 weight tying 下输出投影的尺度。

6.5 坑二：残差处多加一份直连梯度

制造 bug：在残差 2 处把直连梯度多加一份，即 `dx2 = 2·dx + dxFfn`（`-bug residual` 开关）。两层都受影响。

梯度检验能精确定位它（`go run . gradcheck -per 50 -bug residual`，节选）：

参数	最大相对误差	说明
`lnf.g`、`lnf.b`	3.0×10^{-6}、3.9×10^{-6}	在 bug 位置的下游，正常
`l1.w1`、`l1.b1`、`l1.w2`、`l1.b2`、`l1.ln2.*`	10^{-8} 到 10^{-5}	第 2 层 FFN 分支，梯度来自 bug 之前的 `dx`，正常
`l1.wq`、`l1.wk`、`l1.wo`、`l1.ln1.*`	0.42 到 1.0	第 2 层注意力分支，用的是被污染的 `dx2`
`l0.*`	0.68 到 1.0	第 1 层全部参数（`l0.bk` 除外，它的梯度恒为 0，见 6.2 节）
`tok`、`pos`	1.0	嵌入

误差恰好从“第 2 层残差 2”这个位置往输入方向全部爆炸，往输出方向全部正常。只看这张表就能定位到出错的那一行代码。

但 loss 曲线看不出来。 600 步后全窗口 loss 为 0.495，正确实现为 0.470，落在种子间的正常波动里。为什么梯度错了，训练还照样有效？`go run . bugcos` 对同一个 batch 比较正确梯度与 bug 梯度：

参数组	余弦相似度	范数之比（bug / 正确）	逐元素同号比例
第 2 层 FFN、最终 LN	1.000	1.00	100%
第 2 层注意力	0.966 到 0.991	1.84 到 1.99	93% 到 100%
第 1 层 FFN	0.979 到 0.989	1.81 到 1.99	91% 到 100%
第 1 层注意力	0.952 到 0.988	3.59 到 3.77	90% 到 97%
`tok`	0.810	2.28	97%

错误的梯度方向与正确梯度高度一致（余弦 0.95 以上），主要差在大小：每经过一个出错的残差，梯度被放大约 2 倍，两层叠加约 3.6 倍。而 Adam 恰好对梯度的整体大小不敏感（3.14 节）：更新量约为 $\eta\cdot\hat{m}/\sqrt{\hat{v}}$，梯度整体放大 c 倍时分子分母同时放大，更新几乎不变。训练初期 Adam 的更新接近 $\eta\cdot\mathrm{sign}(g)$，而 90% 以上的元素符号相同。所以 bug 版和正确版走的几乎是同一条路。

> **关于第 1 版的"坑二"**
>
> 第 1 版描述的现象是“loss 卡在 0.6 到 0.8 下不去，梯度检验误差在 10^{-2} 量级”。按本书的实测，同类 bug 的梯度检验误差是 1.0 量级而不是 10^{-2}，而 loss 并不会卡住。第 1 版的数字前后也对不上：解析梯度偏大约 2 倍时，相对误差约为 0.33，不可能是 10^{-2}。第 2 版用真实可复现的 bug 重写了本节。

6.6 坑三：去掉因果掩码

制造 bug：前向时不加因果掩码（`-bug nomask`），每个位置都能看到整个窗口。

结果：训练 loss 比正确实现低得多（全窗口 0.227 vs 0.470），生成的却是乱码：

```
"月落落落
```

6.8 小结

- 两套独立实现的正确性证据，是同一份权重下逐元素对齐（float64 下 10^{-15}），不是曲线相似。
- loss 曲线的种子间波动约 ± 0.05，单步波动可达 0.6，足以掩盖一个梯度放大 2 到 4 倍的 bug；Adam 的尺度不变性让这类 bug 更难被发现。
- 梯度检验验证“反向与前向一致”，因果性测试验证“前向没有偷看”，两者缺一不可。
- 初始 loss 偏离 $\ln V$，无论偏高还是偏低，都是值得立即停下来查的信号。

第 7 章 进阶实验：从 GPT-2 到现代架构

7.1 一句话说清

628 个字的语料只够演示“背诵”。本章换成 123 万字的《全唐诗》子集，单独划出验证集，用验证 loss 回答三个问题：训练技巧（学习率调度、梯度裁剪、dropout）什么时候有用；现代大模型换掉的那几个部件（RMSNorm、RoPE、SwiGLU、GQA）各自带来多少收益；模型变大时要注意什么。

本章所有实验用 PyTorch 在 Apple GPU（MPS）上完成，每组约 1 到 2 分钟，代码在 `py/modern.py`。

7.2 新语料、验证集与基线

语料：chinese-poetry 开源项目中《全唐诗》的前 20 个文件（`poet.tang.0.json` 到 `poet.tang.19000.json`，约 2 万首）。原文是繁体，用 OpenCC 的繁简字符对照表逐字转换为简体；去掉空白、缺字符号和校勘注记，只保留 10 到 400 字的诗。按诗随机划分，95% 训练、5% 验证，**验证集里的诗训练时一首都没见过**。

统计项	训练集	验证集
诗的数量	18,934 首	996 首
字符数	1,230,706	62,301
词表（两者合并）	6,780	
验证集中训练集没出现过的字		0.10%

模型与训练配置（基线）：$d = 64$、4 头、4 层、$d_{ff} = 256$、$T = 64$、weight tying，共 638,080 个参数；每步 32 个窗口（2,048 个 token），Adam，学习率 3×10^{-3} 恒定，训练 3,000 步，共约 610 万个 token，相当于把训练集过了约 5 遍。

参照线：先用最简单的统计模型算出几个参照值（单位都是每字的交叉熵，nats）：

模型	验证 loss
均匀猜测（$\ln V$）	8.82
unigram（只看字频）	6.59
bigram（看前一个字，与 unigram 插值）	5.61
本章基线 Transformer，3 个种子	5.019 ± 0.012

基线比 bigram 低 0.59，说明它学到了超出“前一个字”的上下文规律。种子间标准差约 0.012，下面的比较里，小于 0.03 左右的差别可以视为噪声。

另一个与第 2 章截然不同的现象：基线训练结束时训练 loss 为 4.91、验证 loss 为 5.03，两者很接近。模型**还没有开始过拟合**，它的瓶颈是训练量不够，而不是数据被背下来了。这一点决定了下面很多技巧的效果。

7.3 训练治理

学习率调度

先线性 warmup 100 步，再按余弦曲线降到峰值的 10%：

```
def lr_at(step, steps, lr, sched, warmup=100):
    if sched == "const":
        return lr
    if step < warmup:                                    # 线性 warmup
        return lr * (step + 1) / warmup
    p = (step - warmup) / max(1, steps - warmup)         # 余弦退火到 lr 的 10%
    return lr * (0.1 + 0.9 * 0.5 * (1 + math.cos(math.pi * p)))
```

梯度裁剪

把所有参数的梯度视为一个长向量，若其 L2 范数超过阈值 c，就整体缩放到 c：

$$g \leftarrow g \cdot \min\left(1,\ \frac{c}{\|g\|_2}\right)$$

PyTorch 里是一行：`torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)`。

结果

配置	验证 loss	相对基线	训练 loss
基线（3 个种子均值）	5.019	—	4.89
warmup + 余弦退火	5.009	−0.01	4.88
学习率 × 3（9×10^{-3}）	5.318	+0.30	5.27
学习率 × 3 + 梯度裁剪（阈值 1.0）	6.006	+0.99	6.08
dropout 0.1	5.186	+0.17	5.24
dropout 0.2	5.359	+0.34	5.50

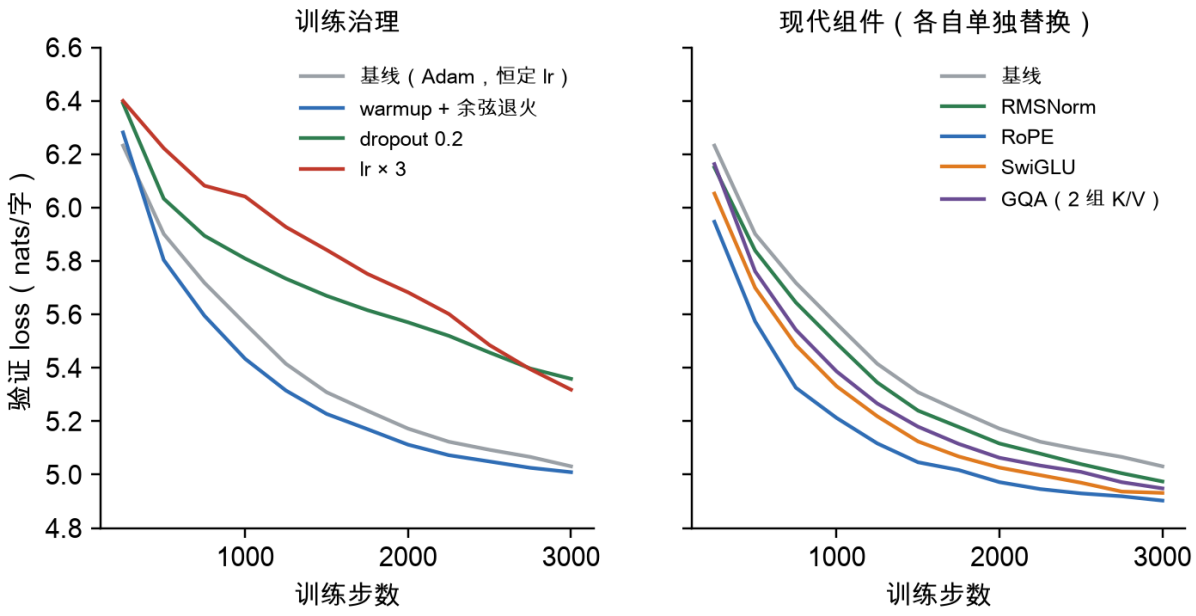


图 7-1 训练治理与现代组件的验证 loss 曲线

三个结论，有两个和直觉相反：

1. **余弦退火的收益在噪声范围内。** 3,000 步对这个模型来说还在“快速下降期”，学习率衰减的好处要在训练后期、loss 开始平台化时才明显。
2. **梯度裁剪救不了过大的学习率。** 训练日志显示，学习率 × 3 时全局梯度范数在 0.15 到 0.47 之间，几乎从不超过阈值 1.0，裁剪基本没有触发；两次运行的差异来自高学习率下训练本身的不稳定。裁剪针对的是偶发的梯度尖峰（loss spike），而学习率过大造成的是每一步都“走过头”，这要靠降低学习率或 warmup 解决。
3. **dropout 让结果变差。** dropout 是正则化手段，作用是缩小训练与验证之间的差距。这里两者本来就很接近，模型处在欠拟合区，再加噪声只会让它学得更慢。对照第 2 章：628 字的语料上训练 loss 能降到 0.2 以下，那种情况才需要正则化。**先看训练与验证的差距，再决定要不要正则化。**

7.4 现代组件

下面四个部件是 GPT-2 之后主流开源模型的标准改动。每个实验只替换一个部件，其余与基线完全相同。

RMSNorm

去掉 LayerNorm 的减均值和偏置 β，只按均方根缩放：

$$y = \frac{x}{\sqrt{\frac{1}{d}\sum_j x_j^2 + \varepsilon}} \odot \gamma$$

```
class RMSNorm(nn.Module):
    """y = x / sqrt(mean(x²) + ε) · γ：去掉了 LayerNorm 的减均值和 β。"""
    def __init__(self, d, eps=1e-5):
        super().__init__()
        self.eps, self.weight = eps, nn.Parameter(torch.ones(d))

    def forward(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) * self.weight
```

计算更省（少一次求均值），参数少一半。它的反向比 LN 简单：只剩“直接路径”和“除以均方根带来的耦合”两项，可以照 3.6 节的方法自己推一遍作为练习。

RoPE（旋转位置编码）

不再给输入加位置嵌入，而是在每个头里，把 q、k 的分量两两配对，按位置 t 旋转一个角度 $t\theta_i$，第 i 对的频率为 $\theta_i = 10000^{-2i/d_h}$：

$$q'_{2i} = q_{2i}\cos t\theta_i - q_{2i+1}\sin t\theta_i, \qquad q'_{2i+1} = q_{2i}\sin t\theta_i + q_{2i+1}\cos t\theta_i$$

旋转之后，位置 t 的 q 与位置 s 的 k 的点积只依赖于相对距离 $t-s$。模型不再需要从数据里学“第 5 个位置长什么样”，而是直接拿到“两个字相距多远”。

```
def rope_cache(T, dh, base=10000.0):
    inv = 1.0 / base ** (torch.arange(0, dh, 2).float() / dh)      # (dh/2,)
    ang = torch.outer(torch.arange(T).float(), inv)                 # (T, dh/2)：位置 × 频率
    return ang.cos(), ang.sin()
```

```
def apply_rope(x, cos, sin):
    """把每个头的向量两两配对，按位置旋转一个角度：q·k 因此只依赖相对位置。"""
```

```
    x1, x2 = x[..., 0::2], x[..., 1::2]
    T = x.shape[-2]
    c, s = cos[:T], sin[:T]
    out = torch.stack([x1 * c - x2 * s, x1 * s + x2 * c], dim=-1)
    return out.flatten(-2)
```

SwiGLU

FFN 改为门控形式：一路过 SiLU 激活，另一路做线性变换，两者逐元素相乘：

$$\mathrm{FFN}(x) = (\mathrm{SiLU}(xW_1) \odot xW_3)W_2, \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

多了一个矩阵 W_3，为了让参数量与基线持平，隐藏维取 $\frac{2}{3} \times 256$ 向下取整到 8 的倍数，即 168。

```
class FFN(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        self.kind = c.ffn
        if c.ffn == "swiglu":
            h = int(2 * c.DFF / 3) // 8 * 8                 # 三个矩阵，隐藏维取 2/3 使参数量持平
            self.w1, self.w3, self.w2 = nn.Linear(c.D, h), nn.Linear(c.D, h), nn.Linear(h, c.D)
        else:
            self.w1, self.w2 = nn.Linear(c.D, c.DFF), nn.Linear(c.DFF, c.D)

    def forward(self, x):
        if self.kind == "swiglu":
            return self.w2(F.silu(self.w1(x)) * self.w3(x))   # 门控：silu(xW1) ⊙ (xW3)
        act = F.relu if self.kind == "relu" else F.gelu
        return self.w2(act(self.w1(x)))
```

GQA（分组查询注意力）

保留 4 个 Q 头，但只用 2 组（或 1 组）K/V，每组 K/V 被 2 个（或 4 个）Q 头共享。K/V 投影变窄，参数量和 KV cache 都随之减少：

```
class Attn(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        self.c, self.dh = c, c.D // c.NH
        self.q = nn.Linear(c.D, c.NH * self.dh)
        self.kv = nn.Linear(c.D, 2 * c.NKV * self.dh)   # GQA：K/V 投影更窄
        self.proj = nn.Linear(c.D, c.D)
        if c.pos == "rope":
            cos, sin = rope_cache(c.T, self.dh)
            self.register_buffer("cos", cos, persistent=False)
            self.register_buffer("sin", sin, persistent=False)

    def forward(self, x):
        B, T, D = x.shape
        c, dh = self.c, self.dh
        q = self.q(x).view(B, T, c.NH, dh).transpose(1, 2)
        k, v = self.kv(x).split(c.NKV * dh, dim=2)
        k = k.view(B, T, c.NKV, dh).transpose(1, 2)
        v = v.view(B, T, c.NKV, dh).transpose(1, 2)
        if c.pos == "rope":
            q, k = apply_rope(q, self.cos, self.sin), apply_rope(k, self.cos, self.sin)
        if c.NKV < c.NH:                                    # 每组 NH/NKV 个 Q 头共享一个 K/V 头
            k = k.repeat_interleave(c.NH // c.NKV, dim=1)
            v = v.repeat_interleave(c.NH // c.NKV, dim=1)
        o = F.scaled_dot_product_attention(q, k, v, is_causal=True,
                                           dropout_p=c.dropout if self.training else 0.0)
        return self.proj(o.transpose(1, 2).contiguous().view(B, T, D))
```

结果

配置	参数量	验证 loss	相对基线	每 token 的 KV cache
基线（LN + 学习式位置 + ReLU + MHA）	638,080	5.019	—	100%
GELU 替换 ReLU	638,080	5.014	−0.005	100%
RMSNorm	637,504	4.974	−0.045	100%
RoPE（去掉 E_{pos}）	633,984	**4.903**	**−0.116**	100%
SwiGLU	636,352	4.931	−0.088	100%
GQA，2 组 K/V	621,440	4.948	−0.071	50%
MQA，1 组 K/V	613,120	5.116	+0.097	25%
四项全换 + dropout 0.1 + 余弦退火	615,040	4.941	−0.078	50%

几点观察：

1. **RoPE 收益最大**，而且参数更少（省掉了 64×64 的位置嵌入表）。诗句的规律主要是相对位置（“五言的第 5 个字后面常是逗号”、“对仗的上下句相隔 5 或 7 个字”），RoPE 直接提供了这种归纳偏置。
2. **SwiGLU 和 RMSNorm 都有稳定的小幅收益**，都超过了种子噪声（0.012）的 3 倍。
3. **GQA 用更少的参数、一半的 KV cache 拿到了更好的结果**；但压到 MQA（1 组）就开始掉点。2 组 K/V 在这里是一个很好的折中，这也是 GQA 在生产模型中流行的原因：推理时 KV cache 是显存大头（第 8.2 节）。
4. **“全都加上”不等于“效果最好”**。最后一行比单独用 RoPE 还差，因为里面混进了 dropout 0.1，而 7.3 节已经说明 dropout 在欠拟合区是有害的。组合改动之前，先逐项做消融。

> **关于这些数字的适用范围**
>
> 这是 64 万参数、600 万 token 的小规模实验，每个配置只跑了一个种子（基线跑了三个）。它能说明各部件的作用方向，但具体幅度在大模型上会不同。例如 GELU 与 ReLU 的差别在这里看不出来，在更大的模型上通常更明显。

7.5 模型变大时

把 d 从 64 改为 32 或 128（层数不变，$d_{ff}=4d$），各训练 6,000 步：

配置	参数量	学习率	训练 loss	验证 loss
$d=32$	269,888	3×10^{-3}	5.03	5.142
$d=64$	638,080	3×10^{-3}	4.62	4.851
$d=64$	638,080	1×10^{-3}	4.71	4.897
$d=128$	1,669,376	3×10^{-3}	4.65	4.911
$d=128$	1,669,376	1×10^{-3}	4.22	**4.640**

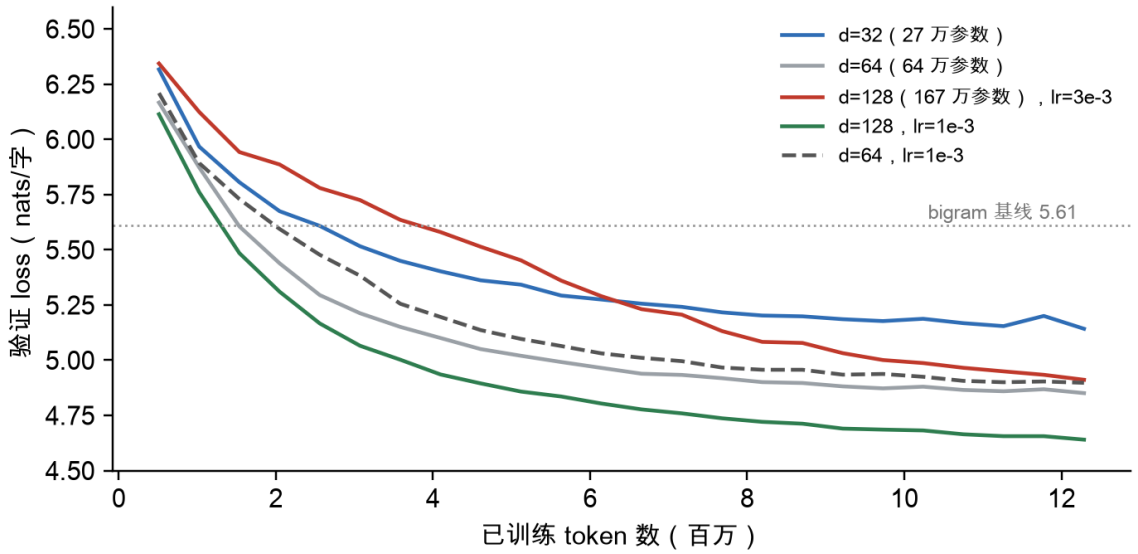


图 7-2 不同规模、不同学习率下验证 loss 随训练 token 数的变化

同样用 3×10^{-3}，$d=128$ 反而不如 $d=64$；把学习率降到 1×10^{-3} 后，它变成了最好的那个。而 $d=64$ 降低学习率反而变差了。也就是说，最优学习率随模型变大而变小：$d=64$ 的最优值在 3×10^{-3} 附近，$d=128$ 的在 1×10^{-3} 附近。**更大的模型通常需要更小的学习率**，这是大模型训练里最基本的经验之一。实际的大模型训练会随规模调整学习率，或者用 μP 一类的参数化方法让最优学习率在不同规模间可迁移。

另外注意 $d=128$、低学习率那一行：训练 loss 4.22 与验证 loss 4.64 之间已经出现了 0.42 的差距，模型开始记住训练集了。到了这个阶段，7.3 节里“有害”的 dropout 才可能变得有用。

7.6 计算量与显存的估算

计算量。训练时每个 token 的计算量约为 $6N$ 次浮点运算，N 是参与矩阵乘的参数量：前向每个参数一次乘加（2 次运算），反向要算对输入和对权重两份梯度（4 次运算）。注意力的 $QK^\top$ 和 PV 另算，每层每 token 约 $3\times4Td$ 次。

以本章基线为例：

部分	参数量（参与矩阵乘）	每 token 训练 FLOPs
4 个 Block 的线性层	$4\times(4d^2+2d\cdot d_{ff})=196{,}608$	1.18 M
输出投影（$V\times d$）	$6{,}780\times64=433{,}920$	2.60 M
注意力的 $QK^\top$、PV	—	0.20 M
合计		约 4.0 M

每步 2,048 个 token，约 8.2 GFLOP；3,000 步约 24 TFLOP，实测用时约 60 秒，即约 0.4 TFLOP/s。一个反直觉的细节：**这个模型约 65% 的计算花在输出投影上**。词表 6,780 而 d 只有 64，最后那个 $T\times d$ 乘 $d\times V$ 的矩阵乘比所有 Block 加起来还贵。大词表、小模型时这很常见，也是大模型的词表大小要和模型宽度一起权衡的原因之一。

显存。用 Adam 做 fp32 训练时，每个参数要存参数本身、梯度、m、v 四份，共 16 字节。本章基线只要 10 MB；GPT-2 small（1.24 亿参数）要约 2 GB，还没算激活值。激活值随 batch、序列长度、层数线性增长，在大模型训练里往往比参数和优化器状态更大，这就是 4.5 节提到的 activation checkpointing 存在的原因。

用同一个公式估算 GPT-2 small 的训练量：nanoGPT 仓库复现 GPT-2 small 的设置大约训练 3,000 亿个 token，$6 \times 1.24 \times 10^8 \times 3 \times 10^{11} \approx 2.2 \times 10^{20}$ FLOP，是本章基线的约 900 万倍。

7.7 小结

- 训练技巧要对症：学习率调度在训练后期才显效；梯度裁剪治的是梯度尖峰，不是学习率过大；dropout 只在训练与验证有差距时才有用。
- 四个现代组件各有稳定收益，RoPE 最大（−0.12），GQA 在省一半 KV cache 的同时还略有提升。组合之前先逐项消融。
- 模型变大时要同步调小学习率，否则更大的模型反而更差。
- 估算训练成本用 $6N \times$token 数；估算显存，Adam fp32 每个参数 16 字节，外加激活值。

第 8 章 推理与性能：Go 的主场

8.1 一句话说清

训练的主流语言是 Python，推理和基础设施这一侧，Go、Rust、C++ 的需求真实存在。本章用 Go 做四件事：给生成加上 KV cache 并测出加速比；用 goroutine 手写一个单机数据并行；加载 PyTorch 训练出的权重做推理；把模型包成一个 HTTP 服务。

8.2 KV cache

原理

第 4.9 节的 `Generate` 每生成一个字，就把最近的整个窗口重新前向一遍。但因果注意力有一个性质：位置 j 的 key 和 value 只依赖 $x_0, \dots, x_j$，生成后面的字时它们不会变。所以每一步只需要为**新来的那一个 token** 计算 q、k、v，把 k、v 追加进缓存，再让 q 和缓存里的全部 key 做注意力。其余部分（LN、FFN、输出投影）本来就是逐位置独立的，也只算新 token 这一行。

```
// KVCache 每层缓存已处理位置的 K、V（每层一个 T×D 矩阵，前 N 行有效）。
type KVCache struct {
    K, V []*Mat
    N    int
}
```

```
// Step 只为"新来的一个 token"做前向：算出它的 q、k、v，把 k、v 追加进缓存，
// q 与缓存中全部历史 k 做注意力。返回该位置的 logits（长度 V）。
// 新 token 只能看到它之前的位置，所以这里天然满足因果掩码。
func (m *Model) Step(id int, kv *KVCache) []float64 {
    c := m.Cfg
    D, dh, t := c.D, c.Dh(), kv.N
    if t >= c.T {
        panic("KV cache 已满：总长度不能超过 block_size")
    }
    x := NewMat(1, D)
    for j := 0; j < D; j++ {
        x.D[j] = m.TokE.D[id*D+j] + m.PosE.D[t*D+j]
    }
    scale := 1.0 / math.Sqrt(float64(dh))
    for l, ly := range m.Layers {
        xln1, _, _ := LayerNorm(x, ly.LN1g, ly.LN1b)
        q := MM(xln1, ly.Wq, false, false)
        AddBias(q, ly.Bq)
        k := MM(xln1, ly.Wk, false, false)
        AddBias(k, ly.Bk)
        v := MM(xln1, ly.Wv, false, false)
        AddBias(v, ly.Bv)
        copy(kv.K[l].D[t*D:(t+1)*D], k.D) // 追加到缓存
        copy(kv.V[l].D[t*D:(t+1)*D], v.D)

        cat := NewMat(1, D)
        p := make([]float64, t+1)
        for h := 0; h < c.NHead; h++ {
            o := h * dh
            mx := math.Inf(-1)
            for j := 0; j <= t; j++ { // 与历史每个位置的 k 做点积
                var s float64
                for e := 0; e < dh; e++ {
                    s += q.D[o+e] * kv.K[l].D[j*D+o+e]
                }
```

```
                p[j] = s * scale
                mx = math.Max(mx, p[j])
            }
            var sum float64
            for j := 0; j <= t; j++ {
                p[j] = math.Exp(p[j] - mx)
                sum += p[j]
            }
            for j := 0; j <= t; j++ { // 输出 = Σ_j p_j · v_j
                w := p[j] / sum
                for e := 0; e < dh; e++ {
                    cat.D[o+e] += w * kv.V[l].D[j*D+o+e]
                }
            }
        }
        attn := MM(cat, ly.Wo, false, false)
        AddBias(attn, ly.Bo)
        AddInplace(x, attn)
        xln2, _, _ := LayerNorm(x, ly.LN2g, ly.LN2b)
        hid := MM(xln2, ly.W1, false, false)
        AddBias(hid, ly.B1)
        f := MM(ReLU(hid), ly.W2, false, false)
        AddBias(f, ly.B2)
        AddInplace(x, f)
    }
    kv.N++
    xf, _, _ := LayerNorm(x, m.LNfg, m.LNfb)
    return MM(xf, m.TokE, false, true).D
}
```

新 token 只能看到它之前的位置（包括自己），所以这里不需要显式的掩码，因果性是天然满足的。

正确性

KV cache 是纯粹的计算优化，输出必须与不带 cache 的版本完全一致。`TestKVCacheMatchesFull` 逐个位置比较 `Step` 返回的 logits 与完整前向最后一行的 logits，最大差为 0，一个比特都不差：两种算法对每个数做的是完全相同的浮点运算序列。

```
func TestKVCacheMatchesFull(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    kv := m.NewKVCache()
    seq := ids[:cfg.T]
    worst := 0.0
    for t0 := 0; t0 < cfg.T; t0++ {
        step := m.Step(seq[t0], kv)
        full := m.Forward(seq[:t0+1]).Logits
        row := full.D[t0*cfg.V : (t0+1)*cfg.V]
        for j := range row {
            worst = math.Max(worst, math.Abs(row[j]-step[j]))
        }
    }
    if worst > 1e-12 {
        t.Fatalf("KV cache 与完整前向不一致：%.2e", worst)
    }
    t.Logf("KV cache 逐位置 logits 最大差 %.2e", worst)
}
```

测速

本书的模型 $T=16$，窗口太短，看不出 cache 的价值。`kvbench` 换了一个更像样的配置：$d=64$、4 层、4 头、$T=512$，权重随机（测速与权重无关），共 251,328 个参数。用贪心解码（温度 0）分别生成不同长度的文本：

生成 token 数	无 cache（ms）	KV cache（ms）	加速比	输出一致
32	50.9	3.0	17 倍	是
64	196.8	6.2	32 倍	是
128	845.8	14.0	61 倍	是
256	4,084.7	35.0	117 倍	是
508	21,710.9	95.7	227 倍	是

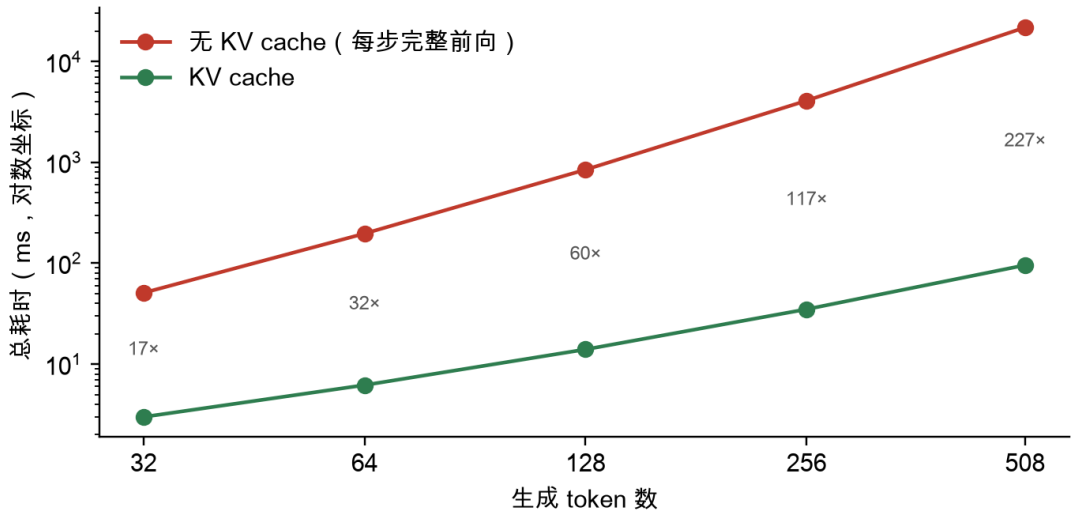


图 8-1 KV cache 的加速效果（双对数坐标）

复杂度分析能解释这张表。设已生成 t 个 token：

- **无 cache**：第 t 步要对 t 个位置做完整前向，矩阵乘部分的代价约为 t，注意力部分约为 t^2。生成 n 个 token 的总代价约为 n^2 到 n^3 之间。实测长度翻倍，耗时变为 3.9 到 5.3 倍，倍数随长度增加而上升，因为 t^2 那一项的占比越来越大。
- **有 cache**：第 t 步的矩阵乘只算一行，代价是常数；注意力要和 t 个 key 做点积，代价约为 t。总代价约为 n 到 n^2 之间。实测长度翻倍，耗时变为 2.1 到 2.7 倍。

显存代价

cache 用内存换计算。每个 token 要在每一层存一个 k 和一个 v，每个 token 的 cache 字节数为：

$$M_{token} = 2 \times L \times n_{kv} \times d_h \times b$$

其中 n_{kv} 是 K/V 头数，b 是每个数的字节数。

本节配置（float64）：$2 \times 4 \times 64 \times 8 = 4$ KB/token，512 个 token 共 2 MB。换成一个 32 层、$d=4096$、fp16 的模型，每个 token 要 $2 \times 32 \times 4096 \times 2 = 512$ KB；上下文 4,096 个 token 时，一条序列就要 2 GB。这就是生产推理里 KV cache 往往比模型权重还占显存的原因，也是 GQA 存在的理由：K/V 头数从 32 降到 8，cache 就缩小为四分之一（第 7.4 节有 GQA 的效果实验）。

> **本书 KV cache 的一个限制**
>
> `GenerateKV` 要求提示词加生成的总长度不超过 T。原因是本书使用学习式的绝对位置嵌入：窗口一旦滑动，所有 token 的位置编号都会变，缓存里的 k、v 就全部失效，只能重算。生产系统一般让上下文窗口足够长，并配合分页管理缓存（PagedAttention）、超长时截断或摘要等策略。

8.3 单机数据并行

实现

一个 batch 里的 B 个窗口互相独立，可以分给多个 goroutine 同时做前向和反向。每个 worker 有一套私有的梯度缓冲区，算完后把所有 worker 的梯度加起来，再统一除以 B。代码在 `LossAndGrads` 的 `else` 分支（第 4.8 节）。

这就是数据并行的完整结构，与多 GPU 训练一一对应：

本书	多 GPU 数据并行
一个 goroutine	一张 GPU
每个 worker 一套私有梯度	每张卡各自算出本地梯度
最后把各 worker 的梯度求和	AllReduce（规约求和后分发给每张卡）
模型参数只读、所有 worker 共享	每张卡持有一份完整的参数副本

`TestParallelMatchesSerial` 检查并行与串行的 loss 和梯度之差小于 10^{-12}。不是恰好为 0，因为浮点加法不满足结合律，求和顺序变了，末位就可能不同。

加速比

`go run . speed` 的结果（每种配置 100 步取平均）：

B	worker 数	每步耗时（ms）	加速比
4	1	2.46	1.00
4	2	1.59	1.55
4	4	1.16	2.13
32	1	19.29	1.00
32	2	11.52	1.67
32	4	7.46	2.59
32	8	4.89	3.94

加速比明显低于 worker 数，原因有三：

1. **串行部分**。梯度归约和 Adam 更新都在主 goroutine 里串行执行，每步要处理 26,912 个参数乘以 worker 数。按 Amdahl 定律，这部分决定了加速比的上限。
2. **每步分配内存**。每个 worker 每步新分配一套梯度缓冲区，带来分配和 GC 的开销。改成预分配、每步清零，能再快一些。

3. **任务太小**。单个窗口的前向加反向只要约 0.6 ms，goroutine 调度和同步的开销占比不低。$B=32$ 时每个 worker 分到更多窗口，加速比也就更高。

同样的三个问题在多 GPU 训练里都有对应：AllReduce 的通信时间、显存分配、单卡计算量太小导致通信占比过高。生产系统用“计算与通信重叠”、梯度分桶、增大每卡 batch 等手段应对。

8.4 PyTorch 训练，Go 推理

训练用 Python 生态，推理部署用 Go，是一种常见的分工。关键是约定一个权重格式。本书直接复用第 6 章对齐时用的 JSON：参数按名字对应，矩阵一律按 Go 的布局（$y=xW$，W 的形状为 输入×输出）行优先展平。

PyTorch 侧训练 3000 步后导出（`py/export.py`），转换逻辑与 `align.py` 相同，只是方向相反：

```
def to_go_layout(model, c):
    """把 PyTorch 模型的参数（或梯度）按 Go 的命名与形状导出，便于逐个比较。"""
    def get(t, grad):
        return (t.grad if grad else t.data).detach()

    def export(grad):
        out = {"tok": get(model.tok.weight, grad), "pos": get(model.pos.weight, grad)}
        D = c.D
        for l, b in enumerate(model.blocks):
            p = f"l{l}."
            W, bias = get(b.qkv.weight, grad), get(b.qkv.bias, grad)  # (3D, D), (3D,)
            out[p + "ln1.g"], out[p + "ln1.b"] = get(b.ln1.weight, grad), get(b.ln1.bias, grad)
            for i, n in enumerate("qkv"):
                out[p + "w" + n] = W[i * D:(i + 1) * D].T      # nn.Linear 存的是 Wᵀ
                out[p + "b" + n] = bias[i * D:(i + 1) * D]
            out[p + "wo"], out[p + "bo"] = get(b.proj.weight, grad).T, get(b.proj.bias, grad)
            out[p + "ln2.g"], out[p + "ln2.b"] = get(b.ln2.weight, grad), get(b.ln2.bias, grad)
            out[p + "w1"], out[p + "b1"] = get(b.ffn[0].weight, grad).T, get(b.ffn[0].bias, grad)
            out[p + "w2"], out[p + "b2"] = get(b.ffn[2].weight, grad).T, get(b.ffn[2].bias, grad)
        out["lnf.g"], out["lnf.b"] = get(model.lnf.weight, grad), get(model.lnf.bias, grad)
        return out
    return export
```

Go 侧按名字逐个拷贝，并校验形状：

```
// LoadWeights 按配置建一个空模型，再按名字逐个拷贝参数，并校验形状。
func LoadWeights(path string) *Model {
    b, err := os.ReadFile(path)
    if err != nil {
        panic(err)
    }
    var w WeightsFile
    if err := json.Unmarshal(b, &w); err != nil {
        panic(err)
    }
    m := NewModel(w.Config, rand.New(rand.NewSource(0)))
    byName := map[string]int{}
    for i, n := range w.Names {
        byName[n] = i
    }
    for i, p := range m.Params {
        j, ok := byName[m.Names[i]]
        if !ok {
            panic("权重文件缺少参数 " + m.Names[i])
        }
        if w.Shapes[j] != [2]int{p.R, p.C} {
            panic(fmt.Sprintf("%s 形状不匹配: 文件 %v, 模型 %d×%d", m.Names[i], w.Shapes[j], p.R, p.C))
        }
        copy(p.D, w.Params[j])
    }
```

```
    return m
}
```

实测：

```
$ python export.py
PyTorch 侧：全部窗口平均 loss 0.164963
已导出 ../out/py_weights.json

$ go run . gen -load ../out/py_weights.json -prompt 月落 -n 40
已加载 ../out/py_weights.json：参数量 26912，全部窗口平均 loss 0.164963
"月落乌啼霜满天，江枫渔火对愁眠。姑苏城外寒山寺，夜半钟声到客船。\n渭城朝雨浥轻尘，客"

$ go run . gen -prompt 两岸 -n 40 -temp 0
"两岸青山相对出，孤帆一片日边来。\n两个黄鹂鸣翠柳，一行白鹭上青天。窗含西岭千秋雪，门"
```

两边在全部 612 个窗口上的平均 loss 小数点后 6 位完全一致：PyTorch 用 float32 计算，导出后 Go 用 float64 计算，两者之差小于打印精度 5×10^{-7}。这同时验证了导出格式和 Go 的前向实现。

8.5 一个最小的 HTTP 推理服务

```
// cmdServe 一个最小的 HTTP 推理服务：GET /generate?prompt=月落&n=30&temp=0.8
func cmdServe(args []string) {
    fs := flag.NewFlagSet("serve", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径（只用来重建词表）")
    load := fs.String("load", "../out/py_weights.json", "权重文件")
    addr := fs.String("addr", "127.0.0.1:8080", "监听地址")
    fs.Parse(args)

    _, tk, _ := loadCorpus(*corpus)
    m := LoadWeights(*load) // 模型只读，可被多个请求并发使用
    http.HandleFunc("/generate", func(w http.ResponseWriter, r *http.Request) {
        q := r.URL.Query()
        n, _ := strconv.Atoi(q.Get("n"))
        if n <= 0 || n > 200 {
            n = 30
        }
        temp, err := strconv.ParseFloat(q.Get("temp"), 64)
        if err != nil {
            temp = 0.8
        }
        prompt := q.Get("prompt")
        for _, ch := range prompt { // 词表外的字直接拒绝，而不是 panic
            if _, ok := tk.stoi[ch]; !ok {
                http.Error(w, "字符不在词表中: "+string(ch), http.StatusBadRequest)
                return
            }
        }
        if prompt == "" {
            prompt = "月"
        }
        t0 := time.Now()
        rng := rand.New(rand.NewSource(time.Now().UnixNano())) // 每个请求独立的随机源
        out := tk.Decode(m.Generate(tk.Encode(prompt), n, temp, 10, rng))
        w.Header().Set("Content-Type", "application/json; charset=utf-8")
        json.NewEncoder(w).Encode(map[string]any{"text": out, "ms": time.Since(t0).Milliseconds()})
    })
    fmt.Println("listening on", *addr)
    if err := http.ListenAndServe(*addr, nil); err != nil {
        panic(err)
    }
}
```

```
$ go run . serve -addr 127.0.0.1:8080 &
$ curl -G http://127.0.0.1:8080/generate --data-urlencode prompt=故人 -d n=24 -d temp=0.5
{"ms":4,"text":"故人。\n黄河远影碧空尽，唯见长江天际流。\n天门中断楚"}
```

```
$ curl -G http://127.0.0.1:8080/generate --data-urlencode prompt=电脑
字符不在词表中: 电
```

几个工程细节：

- **模型只读，天然并发安全**。`Forward` 不修改任何参数，多个请求可以同时调用，不需要加锁。
- **每个请求一个独立的随机源**。`math/rand` 的 `*rand.Rand` 不是并发安全的，不能在请求之间共享。
- **输入校验**。词表外的字返回 400，而不是让 `Encode` panic 拖垮整个进程；生成长度设上限（200）。

离生产还差什么：请求级 KV cache 与批处理（把多个请求的同一步拼成一个矩阵一起算，即连续批处理）、流式返回（SSE）、超时与取消（`context.Context`）、限流、指标与日志、模型热更新。这些正是后台工程师熟悉的领域，只是负载从“查数据库”变成了“做矩阵乘”。

8.6 小结

- KV cache 利用因果注意力“历史 k、v 不变”的性质，把每步计算从整个窗口降到一个 token。生成 508 个 token 时加速 227 倍，输出与无 cache 逐比特一致。代价是每个 token 每层要存一份 k、v，GQA 用减少 K/V 头数来降低这部分显存。
- goroutine 按窗口并行就是单机版的数据并行，梯度求和对应 AllReduce。加速比受串行归约、内存分配和单任务粒度限制。
- 约定好权重格式，就能用 PyTorch 训练、用 Go 推理，两边的 loss 在小数点后 6 位一致。

第 9 章 从玩具到生产

9.1 一句话说清

本书的 GPT 与生产大模型共享同一个计算骨架：嵌入、注意力、归一化、残差、FFN、交叉熵、Adam。两者的差距，一部分在部件的演进（第 7 章），更大的部分在规模带来的工程问题。本章把这些问题列成一张地图，并说明后台工程师的哪些经验可以直接迁移。

9.2 规模对照

维度	本书主线	本书第 7 章	nanoGPT 复现 GPT-2 small	生产前沿模型
参数量	26,912	64 万到 167 万	1.24 亿	数百亿到万亿级（部分为 MoE，每个 token 只激活一部分）
训练数据	628 字符	123 万字	OpenWebText，约 90 亿 token	数万亿 token
上下文长度	16	64	1,024	数万到百万
词表	289（字符级）	6,780（字符级）	50,257（BPE）	约 10 万到 25 万（BPE）
每步 token 数	64	2,048	约 50 万	数百万到数千万
硬件与耗时	1 个 CPU 核，约 1 秒	1 块笔记本 GPU，1 到 2 分钟	8 张 A100，约 4 天	万卡级集群，数周到数月

从本书主线到 GPT-2 small，参数量涨了约 3.7 个数量级；到千亿参数，约 6.6 个数量级。

9.3 七道工程鸿沟

① 数据管道。 本书：`open().read()` 读 628 个字。生产：TB 级语料的抓取、清洗、去重（MinHash/LSH）、质量过滤、领域配比、文档拼接（packing）、跨节点分片，以及断点可恢复的数据加载。本质上是一个超大规模的 ETL 与流式分发系统。

② 数值精度。 本书：Go 用 float64，PyTorch 用 float32。生产：bf16 混合精度是标配，fp8 正在普及；主权重保留 fp32 副本，fp16 训练还要做 loss scaling 防止梯度下溢。本书 Go 版刻意用 float64，换来梯度检验和跨实现对齐的精确性（第 6.2 节：float64 下误差 10^{-15}，float32 下 10^{-7}）；生产里为了吞吐主动接受精度损失。

③ 算子性能。 本书：三重循环矩阵乘。生产：FlashAttention 通过分块计算，避免把 $T \times T$ 的注意力矩阵写回显存，显存占用从 $O(T^2)$ 降到 $O(T)$；算子融合减少显存往返；通信与计算重叠。第 5 章那一行 `F.scaled_dot_product_attention` 背后就是这套东西。

④ 分布式并行。 本书：goroutine 数据并行（第 8.3 节）。生产：数据并行与 ZeRO（把优化器状态、梯度、参数切分到各卡）、张量并行（把一个矩阵乘切到多卡）、流水线并行（把层切到多卡）、序列并行（切超长上下文）、专家并行（MoE）。万卡训练有一半的工作量在容错：机器一定会坏，检查点必须能断点

续训。

⑤ **训练治理。** warmup 与余弦退火、梯度裁剪、权重衰减（AdamW）、loss spike 的自动检测与回滚、验证集监控、实验管理。第 6 章的排错清单和第 7 章的消融方法，在生产里会自动化成监控与告警。

⑥ **推理服务。** 本书：KV cache、单请求 HTTP 服务（第 8 章）。生产：连续批处理、PagedAttention（按页管理 KV cache，减少碎片）、投机解码、量化（INT8、INT4）、预填充与解码分离部署（PD 分离）。推理是一个带状态、高并发、对延迟敏感的在线服务，要在延迟、吞吐、显存三者之间权衡。

⑦ **对齐与应用。** 预训练只是起点：指令微调（SFT）、基于偏好的对齐（RLHF、DPO）、工具调用、检索增强（RAG）。数学上仍是同一个模型的继续训练，工程上主要是数据和评估的问题。

9.4 后台工程师的优势在哪里

1. **分布式系统的直觉可以直接迁移。** 数据并行类似无状态服务的水平扩容；ZeRO 类似分片存储加去冗余；AllReduce 是一种集合通信，相当于“各节点的数据先分片规约、再广播回所有节点”，和 MapReduce 的 shuffle 加 reduce 是同一类模式；检查点与断点续训就是容灾设计。
2. **性能工程的方法论是通用的。** profiling、找瓶颈、判断是计算、通信还是 IO 受限、再对症下手。这套流程在 CUDA kernel 调优和 Go 服务调优里是一样的。第 8.3 节分析 goroutine 并行加速比的方法，换到多卡训练上同样适用。
3. **Go 在推理侧有真实的用武之地。** 推理网关、调度与路由、请求批处理、KV cache 的管理与复用、向量检索与 RAG 中间件，大量推理侧基础设施由 Go、Rust、C++ 编写。
4. **你已经手写过反向传播。** loss 异常、梯度爆炸、收敛诡异时，你能下钻到数学层去查：先看初始 loss 是否等于 $\ln V$，再做梯度检验，再对齐另一份实现，而不是只会调学习率。

9.5 推荐的后续路线

1. **复现 nanoGPT 原版。** 先在莎士比亚字符级数据上跑通，再尝试 GPT-2 small 的配置，体会数据管道、混合精度和多卡训练。
2. **把本书的 Go 版扩展成 GPT-2 推理引擎。** 加 GELU、BPE 分词器、读取 GPT-2 的公开权重，用第 6 章的方法与 PyTorch 逐层对齐。这是一个很好的练手项目。
3. **读 llama.cpp 与 ggml 的源码。** 看 C 语言如何组织与本书 Go 版同构的算子，以及量化、内存映射、KV cache 管理的工业化做法。
4. **跑通 vLLM，读 PagedAttention 的实现。** 把第 8 章的 KV cache 对上生产级的显存管理。
5. **读几篇大模型训练的故障复盘。** 各大机构公开的训练日志和事故报告，是第 6 章排错清单的生产级版本。

9.6 全书小结

- 我们用两种技术栈实现了同一个 26,912 参数的 GPT：Go 手写一切，每个梯度都能单独检验；PyTorch 用框架，每个梯度自动生成。
- 两套实现在同一份权重下逐元素对齐，float64 误差为 10^{-15} 量级。这是“两边都写对了”的真正证据。
- 三个可复现的坑说明：初始 loss 偏离 $\ln V$ 要立刻查初始化；loss 下降证明不了反向正确，要靠梯度检验；梯度检验证明不了前向正确，要靠因果性测试。
- 换到 123 万字的语料后，训练技巧要对症使用；RoPE、SwiGLU、RMSNorm、GQA 各有收益；模型变大要同步调小学习率。
- KV cache、数据并行、PyTorch 训练加 Go 推理、HTTP 服务，是从玩具走向工程的第一步。

亲手写下最后一个梯度、亲眼看到模型写出第一句诗之后，前面几册的公式就不再只是公式了。

附录 A 完整源码

本附录收录全部源码，与随书仓库一一对应。Go 部分按里程碑顺序排列；Python 部分依次为 PyTorch 版、跨实现对齐、权重导出、对照实验、第 7 章实验与语料预处理。画图脚本 `figures.py` 和测速脚本 `speed.py` 未收录。

A.1 Go：矩阵与工具（mat.go）

```
package main

import "fmt"

// Mat 行优先稠密矩阵：第 i 行第 j 列存放在 D[i*C+j]。
// 全书约定：矩阵的一行 = 一个 token 位置的向量。
type Mat struct {
    R, C int
    D    []float64
}

func NewMat(r, c int) *Mat { return &Mat{r, c, make([]float64, r*c)} }

func (m *Mat) At(i, j int) float64 { return m.D[i*m.C+j] }

func (m *Mat) Clone() *Mat {
    o := NewMat(m.R, m.C)
    copy(o.D, m.D)
    return o
}

func (m *Mat) Zero() {
    for i := range m.D {
        m.D[i] = 0
    }
}

// MM 通用矩阵乘：out = op(a)·op(b)，ta/tb 为 true 表示先转置。
// 反向传播里绝大多数矩阵乘只有两种形态：
//
//    dW = Xᵀ·dY  →  MM(x, dy, true, false)
//    dX = dY·Wᵀ  →  MM(dy, w, false, true)
func MM(a, b *Mat, ta, tb bool) *Mat {
    ar, ac := a.R, a.C
    if ta {
        ar, ac = a.C, a.R
    }
    br, bc := b.R, b.C
    if tb {
        br, bc = b.C, b.R
    }
    if ac != br {
        panic(fmt.Sprintf("MM 形状不匹配: (%d×%d)·(%d×%d)", ar, ac, br, bc))
    }
    out := NewMat(ar, bc)
    for i := 0; i < ar; i++ {
        orow := out.D[i*bc : (i+1)*bc]
        for k := 0; k < ac; k++ {
            var av float64
            if ta {
                av = a.D[k*a.C+i]
            } else {
                av = a.D[i*a.C+k]
            }
            if tb { // b 转置：按列读 b，步长为 b.C
```

```
                for j := 0; j < bc; j++ {
                    orow[j] += av * b.D[j*b.C+k]
                }
            } else { // 不转置：b 的第 k 行是连续内存，内层循环对缓存友好
                brow := b.D[k*b.C : (k+1)*b.C]
                for j := 0; j < bc; j++ {
                    orow[j] += av * brow[j]
                }
            }
        }
    }
    return out
}

// AddInplace a += b，用于梯度累加与残差相加。
func AddInplace(a, b *Mat) {
    if a.R != b.R || a.C != b.C {
        panic(fmt.Sprintf("AddInplace 形状不匹配: %d×%d += %d×%d", a.R, a.C, b.R, b.C))
    }
    for i := range a.D {
        a.D[i] += b.D[i]
    }
}

// AddBias 每一行加上同一个偏置行向量 b（1×C），即广播加法。
func AddBias(x, b *Mat) {
    for i := 0; i < x.R; i++ {
        for j := 0; j < x.C; j++ {
            x.D[i*x.C+j] += b.D[j]
        }
    }
}

// ColSum 按列求和得到 1×C，是偏置梯度的计算方式：∂L/∂b = colsum(dY)。
func ColSum(x *Mat) *Mat {
    out := NewMat(1, x.C)
    for i := 0; i < x.R; i++ {
        for j := 0; j < x.C; j++ {
            out.D[j] += x.D[i*x.C+j]
        }
    }
    return out
}

// Scale m *= s
func Scale(m *Mat, s float64) {
    for i := range m.D {
        m.D[i] *= s
    }
}

// SliceCols 取第 [c0, c1) 列，拷贝成新矩阵。多头注意力"切头"就是它。
func SliceCols(m *Mat, c0, c1 int) *Mat {
    out := NewMat(m.R, c1-c0)
    for i := 0; i < m.R; i++ {
        copy(out.D[i*out.C:(i+1)*out.C], m.D[i*m.C+c0:i*m.C+c1])
    }
    return out
}

// SetCols 把 src 写入 dst 从第 c0 列开始的位置。多头"拼回"就是它。
func SetCols(dst, src *Mat, c0 int) {
    for i := 0; i < src.R; i++ {
        copy(dst.D[i*dst.C+c0:i*dst.C+c0+src.C], src.D[i*src.C:(i+1)*src.C])
    }
}
```

A.2 Go：算子（ops.go）

```
package main

import "math"

const lnEps = 1e-5

// LayerNorm 按行归一化：x^ = (x−μ)·r，r = 1/√(σ²+ε)，y = γ⊙x^ + β。
// 返回 y 以及反向需要的缓存 x^ 和每行的 r。
func LayerNorm(x, gamma, beta *Mat) (y, xhat *Mat, rstd []float64) {
    n := x.C
    y, xhat = NewMat(x.R, n), NewMat(x.R, n)
    rstd = make([]float64, x.R)
    for i := 0; i < x.R; i++ {
        row := x.D[i*n : (i+1)*n]
        var mu, va float64
        for _, v := range row {
            mu += v
        }
        mu /= float64(n)
        for _, v := range row {
            d := v - mu
            va += d * d
        }
        va /= float64(n) // 有偏方差（除以 n），与 PyTorch 一致
        r := 1.0 / math.Sqrt(va+lnEps)
        rstd[i] = r
        for j, v := range row {
            h := (v - mu) * r
            xhat.D[i*n+j] = h
            y.D[i*n+j] = gamma.D[j]*h + beta.D[j]
        }
    }
    return
}

// LNBackward 令 g = γ⊙dy（逐元素），则
//
//    dx_j = r · [ g_j − mean(g) − x^_j · mean(g⊙x^) ]
//
// dγ、dβ 是累加的：同一批里所有行共享一组 γ、β。
func LNBackward(dy, xhat, gamma *Mat, rstd []float64, dgamma, dbeta *Mat) *Mat {
    n := dy.C
    dx := NewMat(dy.R, n)
    g := make([]float64, n)
    for i := 0; i < dy.R; i++ {
        var sumG, sumGX float64
        for j := 0; j < n; j++ {
            d, h := dy.D[i*n+j], xhat.D[i*n+j]
            g[j] = d * gamma.D[j]
            sumG += g[j]
            sumGX += g[j] * h
            dgamma.D[j] += d * h // 累加
            dbeta.D[j] += d      // 累加
        }
        meanG, meanGX := sumG/float64(n), sumGX/float64(n)
        for j := 0; j < n; j++ {
            dx.D[i*n+j] = rstd[i] * (g[j] - meanG - xhat.D[i*n+j]*meanGX)
        }
    }
    return dx
}

// Softmax 按行计算；causal 为 true 时第 i 行只看第 0..i 列（j>i 的概率直接置 0，
// 等价于把这些分数设成 −∞）。减去行最大值保证 exp 不溢出。
func Softmax(s *Mat, causal bool) *Mat {
    p := NewMat(s.R, s.C)
```

```
    for i := 0; i < s.R; i++ {
        last := s.C
        if causal {
            last = i + 1
        }
        row := s.D[i*s.C : i*s.C+last]
        mx := row[0]
        for _, v := range row {
            if v > mx {
                mx = v
            }
        }
        var sum float64
        for j, v := range row {
            e := math.Exp(v - mx)
            p.D[i*s.C+j] = e
            sum += e
        }
        for j := 0; j < last; j++ {
            p.D[i*s.C+j] /= sum
        }
    }
    return p
}

// SoftmaxBackward 按行：ds_j = p_j · (dp_j − Σ_k dp_k p_k)。
// 被掩码的位置 p=0，梯度自然为 0，不需要特殊处理。
func SoftmaxBackward(dp, p *Mat) *Mat {
    ds := NewMat(dp.R, dp.C)
    for i := 0; i < dp.R; i++ {
        var dot float64
        for j := 0; j < dp.C; j++ {
            dot += dp.D[i*dp.C+j] * p.D[i*p.C+j]
        }
        for j := 0; j < dp.C; j++ {
            ds.D[i*dp.C+j] = p.D[i*p.C+j] * (dp.D[i*dp.C+j] - dot)
        }
    }
    return ds
}

func ReLU(h *Mat) *Mat {
    u := NewMat(h.R, h.C)
    for i, v := range h.D {
        if v > 0 {
            u.D[i] = v
        }
    }
    return u
}

// ReLUBackward 正区间梯度照传，其余置零。h 是 ReLU 的输入。
func ReLUBackward(du, h *Mat) *Mat {
    dh := NewMat(du.R, du.C)
    for i, v := range h.D {
        if v > 0 {
            dh.D[i] = du.D[i]
        }
    }
    return dh
}

// CrossEntropy 逐位置交叉熵并对 T 个位置取平均。
// 返回 loss 和 ∂L/∂logits = (softmax(logits) − onehot(y)) / T。
func CrossEntropy(logits *Mat, targets []int) (float64, *Mat) {
    T, V := logits.R, logits.C
    dlogits := NewMat(T, V)
    var loss float64
    for t := 0; t < T; t++ {
```

```
        row := logits.D[t*V : (t+1)*V]
        mx := row[0]
        for _, v := range row {
            if v > mx {
                mx = v
            }
        }
        var sum float64
        for _, v := range row {
            sum += math.Exp(v - mx)
        }
        // -log softmax_y = -logit_y + log Σ exp(logit) = -logit_y + mx + log Σ exp(logit-mx)
        loss += -row[targets[t]] + mx + math.Log(sum)
        for j, v := range row {
            p := math.Exp(v-mx) / sum
            if j == targets[t] {
                p -= 1
            }
            dlogits.D[t*V+j] = p / float64(T)
        }
    }
    return loss / float64(T), dlogits
}
```

A.3 Go：模型定义（model.go）

```
package main

import (
    "fmt"
    "math/rand"
)

// Config 模型超参数。默认值见 DefaultConfig。
type Config struct {
    V      int     // 词表大小（由语料决定）
    D      int     // d_model：嵌入/隐藏维度
    NHead  int     // 注意力头数
    NLayer int     // Transformer Block 层数
    DFF    int     // FFN 隐藏维
    T      int     // block_size：上下文窗口长度
    Std    float64 // 权重初始化标准差
}

func DefaultConfig(V int) Config {
    return Config{V: V, D: 32, NHead: 2, NLayer: 2, DFF: 64, T: 16, Std: 0.02}
}

func (c Config) Dh() int { return c.D / c.NHead }

// Bug 用于第 6 章的"故意制造 bug"实验，正常训练时为 BugNone。
type Bug int

const (
    BugNone           Bug = iota
    BugResidualDouble     // 残差 2 处把直连梯度多加一份：dx2 = 2·dx + dxFfn
    BugNoMask             // 前向不加因果掩码：模型可以"偷看"未来
)

type Layer struct {
    LN1g, LN1b     *Mat // 1×D
    Wq, Wk, Wv, Wo *Mat // D×D
    Bq, Bk, Bv, Bo *Mat // 1×D
    LN2g, LN2b     *Mat // 1×D
    W1, B1, W2, B2 *Mat // D×DFF, 1×DFF, DFF×D, 1×D
}
```

```
// Model 没有 autograd，我们自己维护"参数 ↔ 梯度"的一一对应（参数注册表）。
type Model struct {
    Cfg        Config
    Bug        Bug
    TokE, PosE *Mat // V×D, T×D；TokE 同时充当输出投影（weight tying）
    Layers     []*Layer
    LNfg, LNfb *Mat
    Params     []*Mat
    Names      []string
    Grads      []*Mat
    idx        map[*Mat]int
}

func (m *Model) reg(name string, p *Mat) *Mat {
    m.idx[p] = len(m.Params)
    m.Params = append(m.Params, p)
    m.Names = append(m.Names, name)
    m.Grads = append(m.Grads, NewMat(p.R, p.C))
    return p
}

// NewGrads 分配一套与 Params 同形状的全零梯度（并行训练时每个 worker 一套）。
func (m *Model) NewGrads() []*Mat {
    g := make([]*Mat, len(m.Params))
    for i, p := range m.Params {
        g[i] = NewMat(p.R, p.C)
    }
    return g
}

func (m *Model) ZeroGrads() {
    for _, g := range m.Grads {
        g.Zero()
    }
}

func (m *Model) NumParams() int {
    n := 0
    for _, p := range m.Params {
        n += len(p.D)
    }
    return n
}

func randn(r, c int, std float64, rng *rand.Rand) *Mat {
    m := NewMat(r, c)
    for i := range m.D {
        m.D[i] = rng.NormFloat64() * std
    }
    return m
}

func ones(r, c int) *Mat {
    m := NewMat(r, c)
    for i := range m.D {
        m.D[i] = 1
    }
    return m
}

// NewModel 初始化：权重 ~ N(0, Std²)，偏置为 0，LN 的 γ=1、β=0。
func NewModel(cfg Config, rng *rand.Rand) *Model {
    if cfg.D%cfg.NHead != 0 {
        panic("D 必须能被 NHead 整除")
    }
    m := &Model{Cfg: cfg, idx: map[*Mat]int{}}
    D, s := cfg.D, cfg.Std
    m.TokE = m.reg("tok", randn(cfg.V, D, s, rng))
    m.PosE = m.reg("pos", randn(cfg.T, D, s, rng))
```

```
    for l := 0; l < cfg.NLayer; l++ {
        p := fmt.Sprintf("l%d.", l)
        ly := &Layer{}
        ly.LN1g, ly.LN1b = m.reg(p+"ln1.g", ones(1, D)), m.reg(p+"ln1.b", NewMat(1, D))
        ly.Wq, ly.Bq = m.reg(p+"wq", randn(D, D, s, rng)), m.reg(p+"bq", NewMat(1, D))
        ly.Wk, ly.Bk = m.reg(p+"wk", randn(D, D, s, rng)), m.reg(p+"bk", NewMat(1, D))
        ly.Wv, ly.Bv = m.reg(p+"wv", randn(D, D, s, rng)), m.reg(p+"bv", NewMat(1, D))
        ly.Wo, ly.Bo = m.reg(p+"wo", randn(D, D, s, rng)), m.reg(p+"bo", NewMat(1, D))
        ly.LN2g, ly.LN2b = m.reg(p+"ln2.g", ones(1, D)), m.reg(p+"ln2.b", NewMat(1, D))
        ly.W1, ly.B1 = m.reg(p+"w1", randn(D, cfg.DFF, s, rng)), m.reg(p+"b1", NewMat(1, cfg.DFF))
        ly.W2, ly.B2 = m.reg(p+"w2", randn(cfg.DFF, D, s, rng)), m.reg(p+"b2", NewMat(1, D))
        m.Layers = append(m.Layers, ly)
    }
    m.LNfg, m.LNfb = m.reg("lnf.g", ones(1, D)), m.reg("lnf.b", NewMat(1, D))
    return m
}
```

A.4 Go：前向（forward.go）

```
package main

import "math"

// HeadCache 单个注意力头反向需要的中间结果。
type HeadCache struct{ Q, K, V, P *Mat } // Q/K/V: T×dh，P: T×T（softmax 后）

// LayerCache 一个 Block 反向需要的全部中间结果。
type LayerCache struct {
    Xhat1 *Mat      // LN1 内部标准化值 x^
    Rstd1 []float64 // LN1 每行的 r
    Xln1  *Mat      // LN1 输出（QKV 的共同输入）
    Heads []HeadCache
    Cat   *Mat // 多头拼接结果 T×D
    Xhat2 *Mat
    Rstd2 []float64
    Xln2  *Mat // LN2 输出（FFN 输入）
    H, U  *Mat // FFN 隐藏层 ReLU 之前 / 之后
}

type FwdCache struct {
    Ids    []int
    Layers []LayerCache
    Xhatf  *Mat
    Rstdf  []float64
    Xf     *Mat // 最终 LN 输出
    Logits *Mat // T×V
}

// Forward 单窗口前向，返回带全部缓存的 FwdCache。ids 长度 ≤ Cfg.T。
// 只读模型参数，可以被多个 goroutine 并发调用。
func (m *Model) Forward(ids []int) *FwdCache {
    c := m.Cfg
    T, D, dh := len(ids), c.D, c.Dh()
    fc := &FwdCache{Ids: ids}

    // ① 嵌入：X⁰[t] = TokE[ids[t]] + PosE[t]
    x := NewMat(T, D)
    for t, id := range ids {
        for j := 0; j < D; j++ {
            x.D[t*D+j] = m.TokE.D[id*D+j] + m.PosE.D[t*D+j]
        }
    }

    scale := 1.0 / math.Sqrt(float64(dh))
    for _, ly := range m.Layers {
        var lc LayerCache
        // ② 注意力子层（Pre-LN）
```

```
        xln1, xhat1, rstd1 := LayerNorm(x, ly.LN1g, ly.LN1b)
        lc.Xhat1, lc.Rstd1, lc.Xln1 = xhat1, rstd1, xln1
        q := MM(xln1, ly.Wq, false, false)
        AddBias(q, ly.Bq)
        k := MM(xln1, ly.Wk, false, false)
        AddBias(k, ly.Bk)
        v := MM(xln1, ly.Wv, false, false)
        AddBias(v, ly.Bv)

        cat := NewMat(T, D)
        for h := 0; h < c.NHead; h++ {
            qh := SliceCols(q, h*dh, (h+1)*dh)
            kh := SliceCols(k, h*dh, (h+1)*dh)
            vh := SliceCols(v, h*dh, (h+1)*dh)
            s := MM(qh, kh, false, true) // S = Q·Kᵀ / √dh   (T×T)
            Scale(s, scale)
            p := Softmax(s, m.Bug != BugNoMask)         // 因果掩码 + softmax
            SetCols(cat, MM(p, vh, false, false), h*dh) // C = P·V
            lc.Heads = append(lc.Heads, HeadCache{qh, kh, vh, p})
        }
        lc.Cat = cat
        attn := MM(cat, ly.Wo, false, false)
        AddBias(attn, ly.Bo)
        x2 := x.Clone()
        AddInplace(x2, attn) // 残差 1：X' = X + A

        // ③ FFN 子层（Pre-LN）
        xln2, xhat2, rstd2 := LayerNorm(x2, ly.LN2g, ly.LN2b)
        lc.Xhat2, lc.Rstd2, lc.Xln2 = xhat2, rstd2, xln2
        hid := MM(xln2, ly.W1, false, false)
        AddBias(hid, ly.B1)
        u := ReLU(hid)
        lc.H, lc.U = hid, u
        f := MM(u, ly.W2, false, false)
        AddBias(f, ly.B2)
        x3 := x2.Clone()
        AddInplace(x3, f) // 残差 2：X = X' + F

        fc.Layers = append(fc.Layers, lc)
        x = x3
    }

    // ④ 最终 LN + 共享嵌入的输出投影：logits = Xf · TokEᵀ
    xf, xhatf, rstdf := LayerNorm(x, m.LNfg, m.LNfb)
    fc.Xhatf, fc.Rstdf, fc.Xf = xhatf, rstdf, xf
    fc.Logits = MM(xf, m.TokE, false, true) // (T×D)·(D×V) = T×V
    return fc
}
```

A.5 Go：反向（backward.go）

```
package main

import "math"

// Backward 单窗口反向：把 ∂L/∂θ 累加进 grads（与 m.Params 一一对应）。
// 对照第 3.5 节的反向链逐行翻译。grads 传 m.Grads 即为默认行为。
func (m *Model) Backward(fc *FwdCache, dlogits *Mat, grads []*Mat) {
    c := m.Cfg
    T, D, dh := len(fc.Ids), c.D, c.Dh()
    g := func(p *Mat) *Mat { return grads[m.idx[p]] }

    // logits = Xf·TokEᵀ：TokE 的第一份梯度（weight tying）
    AddInplace(g(m.TokE), MM(dlogits, fc.Xf, true, false)) // (V×T)·(T×D)
    dxf := MM(dlogits, m.TokE, false, false)               // (T×V)·(V×D)
    dx := LNBackward(dxf, fc.Xhatf, m.LNfg, fc.Rstdf, g(m.LNfg), g(m.LNfb))
```

```
    scale := 1.0 / math.Sqrt(float64(dh))
    for li := c.NLayer - 1; li >= 0; li-- { // 逆序过层
        ly, lc := m.Layers[li], fc.Layers[li]

        // ---- FFN 分支: F = ReLU(Xln2·W1 + b1)·W2 + b2 ----
        AddInplace(g(ly.B2), ColSum(dx))
        AddInplace(g(ly.W2), MM(lc.U, dx, true, false))
        du := MM(dx, ly.W2, false, true)
        dH := ReLUBackward(du, lc.H)
        AddInplace(g(ly.B1), ColSum(dH))
        AddInplace(g(ly.W1), MM(lc.Xln2, dH, true, false))
        dxln2 := MM(dH, ly.W1, false, true)
        dxFfn := LNBackward(dxln2, lc.Xhat2, ly.LN2g, lc.Rstd2, g(ly.LN2g), g(ly.LN2b))

        // ★ 残差 2: X = X' + F  ⇒  ∂L/∂X' = dx（直连） + dxFfn（分支），各加一次
        dx2 := dx.Clone()
        if m.Bug == BugResidualDouble {
            AddInplace(dx2, dx) // 故意制造的 bug: 直连梯度多加了一份
        }
        AddInplace(dx2, dxFfn)

        // ---- 注意力分支: A = Cat·Wo + bo ----
        AddInplace(g(ly.Bo), ColSum(dx2))
        AddInplace(g(ly.Wo), MM(lc.Cat, dx2, true, false))
        dcat := MM(dx2, ly.Wo, false, true)

        dq, dk, dv := NewMat(T, D), NewMat(T, D), NewMat(T, D)
        for h := 0; h < c.NHead; h++ {
            hc := lc.Heads[h]
            dC := SliceCols(dcat, h*dh, (h+1)*dh)
            dP := MM(dC, hc.V, false, true)               // dP = dC·Vᵀ
            SetCols(dv, MM(hc.P, dC, true, false), h*dh)  // dV = Pᵀ·dC
            dS := SoftmaxBackward(dP, hc.P)               // 对缩放后分数的梯度
            Scale(dS, scale)                              // 乘 1/√dh: 变为对 Q·Kᵀ 的梯度（只乘这一次）
            SetCols(dq, MM(dS, hc.K, false, false), h*dh) // dQ = dS·K
            SetCols(dk, MM(dS, hc.Q, true, false), h*dh)  // dK = dSᵀ·Q
        }

        // QKV 三个投影: 结构完全相同
        AddInplace(g(ly.Bq), ColSum(dq))
        AddInplace(g(ly.Bk), ColSum(dk))
        AddInplace(g(ly.Bv), ColSum(dv))
        AddInplace(g(ly.Wq), MM(lc.Xln1, dq, true, false))
        AddInplace(g(ly.Wk), MM(lc.Xln1, dk, true, false))
        AddInplace(g(ly.Wv), MM(lc.Xln1, dv, true, false))
        // ★ Xln1 同时喂给 Q、K、V，梯度是三条链之和
        dxln1 := MM(dq, ly.Wq, false, true)
        AddInplace(dxln1, MM(dk, ly.Wk, false, true))
        AddInplace(dxln1, MM(dv, ly.Wv, false, true))
        dxAttn := LNBackward(dxln1, lc.Xhat1, ly.LN1g, lc.Rstd1, g(ly.LN1g), g(ly.LN1b))

        // ★ 残差 1: X' = X + A  ⇒  ∂L/∂X = dx2 + dxAttn
        AddInplace(dx2, dxAttn)
        dx = dx2
    }

    // ★ 嵌入的反向是 scatter-add: 哪一行被查过，梯度就加回哪一行
    gT, gP := g(m.TokE), g(m.PosE) // TokE 的第二份梯度
    for t, id := range fc.Ids {
        for j := 0; j < D; j++ {
            gT.D[id*D+j] += dx.D[t*D+j]
            gP.D[t*D+j] += dx.D[t*D+j]
        }
    }
}
```

A.6 Go：分词、优化器与训练步（train.go）

```
package main

import (
    "math"
    "math/rand"
    "sort"
    "strings"
    "sync"
)

// ---------------- 分词器：字符级 ----------------

// Tokenizer 字符级分词器。词表按 Unicode 码点排序后编号，
// 与 Python 版 sorted(set(text)) 完全一致，两边的 id 可以直接互换。
type Tokenizer struct {
    stoi map[rune]int
    itos []rune
}

func NewTokenizer(text string) *Tokenizer {
    seen := map[rune]bool{}
    for _, r := range text { // range string 按 rune 遍历，不会把汉字拆成字节
        seen[r] = true
    }
    tk := &Tokenizer{stoi: map[rune]int{}}
    for r := range seen {
        tk.itos = append(tk.itos, r)
    }
    sort.Slice(tk.itos, func(i, j int) bool { return tk.itos[i] < tk.itos[j] })
    for i, r := range tk.itos {
        tk.stoi[r] = i
    }
    return tk
}

func (tk *Tokenizer) Size() int { return len(tk.itos) }

func (tk *Tokenizer) Encode(s string) []int {
    ids := make([]int, 0, len(s))
    for _, r := range s {
        id, ok := tk.stoi[r]
        if !ok {
            panic("字符不在词表中: " + string(r))
        }
        ids = append(ids, id)
    }
    return ids
}

func (tk *Tokenizer) Decode(ids []int) string {
    var b strings.Builder
    for _, id := range ids {
        b.WriteRune(tk.itos[id])
    }
    return b.String()
}

// GetBatch 随机取 B 个长度为 T 的窗口；y 是 x 整体右移一位。
// 合法起点 i ∈ [0, N-T-1]，共 N-T 个，所以是 Intn(N-T)。
func GetBatch(ids []int, B, T int, rng *rand.Rand) (x, y [][]int) {
    x, y = make([][]int, B), make([][]int, B)
    for b := 0; b < B; b++ {
        i := rng.Intn(len(ids) - T)
        x[b] = ids[i : i+T]
        y[b] = ids[i+1 : i+T+1]
    }
```

```
    return
}

// ---------------- 优化器 ----------------

type Adam struct {
    LR, B1, B2, Eps float64
    BiasCorrection  bool // 关掉它用于第 3.7 节的对照实验
    t               int
    m, v            []*Mat
}

func NewAdam(model *Model, lr float64) *Adam {
    return &Adam{LR: lr, B1: 0.9, B2: 0.999, Eps: 1e-8, BiasCorrection: true,
        m: model.NewGrads(), v: model.NewGrads()}
}

func (a *Adam) Step(params, grads []*Mat) {
    a.t++
    bc1, bc2 := 1.0, 1.0
    if a.BiasCorrection {
        bc1 = 1 - math.Pow(a.B1, float64(a.t))
        bc2 = 1 - math.Pow(a.B2, float64(a.t))
    }
    for i, p := range params {
        g, m, v := grads[i].D, a.m[i].D, a.v[i].D
        for j := range p.D {
            m[j] = a.B1*m[j] + (1-a.B1)*g[j]
            v[j] = a.B2*v[j] + (1-a.B2)*g[j]*g[j]
            mh, vh := m[j]/bc1, v[j]/bc2
            p.D[j] -= a.LR * mh / (math.Sqrt(vh) + a.Eps)
        }
    }
}

// SGDStep 朴素梯度下降，用于和 Adam 对比。
func SGDStep(params, grads []*Mat, lr float64) {
    for i, p := range params {
        for j := range p.D {
            p.D[j] -= lr * grads[i].D[j]
        }
    }
}

// ---------------- 一个训练步 ----------------

// LossAndGrads 对 B 个窗口做前向+反向，梯度写入 m.Grads 并按 B 取平均，返回平均 loss。
// workers>1 时按窗口并行：每个 worker 一套私有梯度，最后归约（单机版"数据并行 + AllReduce"）。
func (m *Model) LossAndGrads(xs, ys [][]int, workers int) float64 {
    B := len(xs)
    m.ZeroGrads()
    var loss float64
    if workers <= 1 {
        for b := 0; b < B; b++ {
            fc := m.Forward(xs[b])
            l, dl := CrossEntropy(fc.Logits, ys[b])
            loss += l
            m.Backward(fc, dl, m.Grads)
        }
    } else {
        local := make([][]*Mat, workers)
        losses := make([]float64, workers)
        var wg sync.WaitGroup
        for w := 0; w < workers; w++ {
            wg.Add(1)
            go func(w int) {
                defer wg.Done()
                local[w] = m.NewGrads()
                for b := w; b < B; b += workers {
```

```
                    fc := m.Forward(xs[b])
                    l, dl := CrossEntropy(fc.Logits, ys[b])
                    losses[w] += l
                    m.Backward(fc, dl, local[w])
                }
            }(w)
        }
        wg.Wait()
        for w := 0; w < workers; w++ { // 归约：所有 worker 的梯度求和
            loss += losses[w]
            for i := range m.Grads {
                AddInplace(m.Grads[i], local[w][i])
            }
        }
    }
    for _, g := range m.Grads {
        Scale(g, 1/float64(B))
    }
    return loss / float64(B)
}

// Loss 只做前向，返回 B 个窗口的平均 loss（梯度检验用）。
func (m *Model) Loss(xs, ys [][]int) float64 {
    var loss float64
    for b := range xs {
        l, _ := CrossEntropy(m.Forward(xs[b]).Logits, ys[b])
        loss += l
    }
    return loss / float64(len(xs))
}
```

A.7 Go：采样、生成与 KV cache（generate.go）

```
package main

import (
    "math"
    "math/rand"
    "sort"
)

// Sample 从一行 logits 中采样：temp=0 取 argmax；topK>0 时只在概率最高的 K 个里采样。
func Sample(logits []float64, temp float64, topK int, rng *rand.Rand) int {
    V := len(logits)
    if temp == 0 {
        best := 0
        for i, v := range logits {
            if v > logits[best] {
                best = i
            }
        }
        return best
    }
    idx := make([]int, V)
    for i := range idx {
        idx[i] = i
    }
    if topK > 0 && topK < V {
        sort.Slice(idx, func(a, b int) bool { return logits[idx[a]] > logits[idx[b]] })
        idx = idx[:topK]
    }
    mx := math.Inf(-1)
    for _, i := range idx {
        mx = math.Max(mx, logits[i]/temp)
    }
    probs := make([]float64, len(idx))
    var sum float64
```

```
    for k, i := range idx {
        probs[k] = math.Exp(logits[i]/temp - mx)
        sum += probs[k]
    }
    r := rng.Float64() * sum // 轮盘赌
    for k, p := range probs {
        r -= p
        if r <= 0 {
            return idx[k]
        }
    }
    return idx[len(idx)-1]
}

// Generate 自回归生成（无 KV cache）：每生成一个字都把最近 T 个字完整前向一遍。
func (m *Model) Generate(prompt []int, n int, temp float64, topK int, rng *rand.Rand) []int {
    ids := append([]int{}, prompt...)
    T, V := m.Cfg.T, m.Cfg.V
    for i := 0; i < n; i++ {
        ctx := ids
        if len(ctx) > T {
            ctx = ctx[len(ctx)-T:] // 只看最近 T 个
        }
        lg := m.Forward(ctx).Logits
        last := lg.D[(lg.R-1)*V : lg.R*V] // 只用最后一个位置的 logits
        ids = append(ids, Sample(last, temp, topK, rng))
    }
    return ids
}

// ---------------- KV cache ----------------

// KVCache 每层缓存已处理位置的 K、V（每层一个 T×D 矩阵，前 N 行有效）。
type KVCache struct {
    K, V []*Mat
    N    int
}

func (m *Model) NewKVCache() *KVCache {
    kv := &KVCache{}
    for range m.Layers {
        kv.K = append(kv.K, NewMat(m.Cfg.T, m.Cfg.D))
        kv.V = append(kv.V, NewMat(m.Cfg.T, m.Cfg.D))
    }
    return kv
}

// Step 只为"新来的一个 token"做前向：算出它的 q、k、v，把 k、v 追加进缓存，
// q 与缓存中全部历史 k 做注意力。返回该位置的 logits（长度 V）。
// 新 token 只能看到它之前的位置，所以这里天然满足因果掩码。
func (m *Model) Step(id int, kv *KVCache) []float64 {
    c := m.Cfg
    D, dh, t := c.D, c.Dh(), kv.N
    if t >= c.T {
        panic("KV cache 已满：总长度不能超过 block_size")
    }
    x := NewMat(1, D)
    for j := 0; j < D; j++ {
        x.D[j] = m.TokE.D[id*D+j] + m.PosE.D[t*D+j]
    }
    scale := 1.0 / math.Sqrt(float64(dh))
    for l, ly := range m.Layers {
        xln1, _, _ := LayerNorm(x, ly.LN1g, ly.LN1b)
        q := MM(xln1, ly.Wq, false, false)
        AddBias(q, ly.Bq)
        k := MM(xln1, ly.Wk, false, false)
        AddBias(k, ly.Bk)
        v := MM(xln1, ly.Wv, false, false)
        AddBias(v, ly.Bv)
```

```
        copy(kv.K[l].D[t*D:(t+1)*D], k.D) // 追加到缓存
        copy(kv.V[l].D[t*D:(t+1)*D], v.D)

        cat := NewMat(1, D)
        p := make([]float64, t+1)
        for h := 0; h < c.NHead; h++ {
            o := h * dh
            mx := math.Inf(-1)
            for j := 0; j <= t; j++ { // 与历史每个位置的 k 做点积
                var s float64
                for e := 0; e < dh; e++ {
                    s += q.D[o+e] * kv.K[l].D[j*D+o+e]
                }
                p[j] = s * scale
                mx = math.Max(mx, p[j])
            }
            var sum float64
            for j := 0; j <= t; j++ {
                p[j] = math.Exp(p[j] - mx)
                sum += p[j]
            }
            for j := 0; j <= t; j++ { // 输出 = Σ_j p_j · v_j
                w := p[j] / sum
                for e := 0; e < dh; e++ {
                    cat.D[o+e] += w * kv.V[l].D[j*D+o+e]
                }
            }
        }
        attn := MM(cat, ly.Wo, false, false)
        AddBias(attn, ly.Bo)
        AddInplace(x, attn)
        xln2, _, _ := LayerNorm(x, ly.LN2g, ly.LN2b)
        hid := MM(xln2, ly.W1, false, false)
        AddBias(hid, ly.B1)
        f := MM(ReLU(hid), ly.W2, false, false)
        AddBias(f, ly.B2)
        AddInplace(x, f)
    }
    kv.N++
    xf, _, _ := LayerNorm(x, m.LNfg, m.LNfb)
    return MM(xf, m.TokE, false, true).D
}

// GenerateKV 带 KV cache 的生成：先逐个 token 预填充（prefill）提示词，再逐字解码。
// 为了与 Generate 逐 token 等价，要求 len(prompt)+n ≤ T。
func (m *Model) GenerateKV(prompt []int, n int, temp float64, topK int, rng *rand.Rand) []int {
    if len(prompt)+n > m.Cfg.T {
        panic("GenerateKV 要求 len(prompt)+n ≤ block_size")
    }
    kv := m.NewKVCache()
    ids := append([]int{}, prompt...)
    var last []float64
    for _, id := range prompt {
        last = m.Step(id, kv)
    }
    for i := 0; i < n; i++ {
        nxt := Sample(last, temp, topK, rng)
        ids = append(ids, nxt)
        if i < n-1 {
            last = m.Step(nxt, kv)
        }
    }
    return ids
}
```

A.8 Go：梯度检验（gradcheck.go）与 bug 梯度分析（bugcos.go）

```
package main

import (
    "math"
    "math/rand"
)

// GradCheckResult 一个参数矩阵的梯度检验结果。
type GradCheckResult struct {
    Name   string
    N      int     // 抽查的元素个数
    MaxRel float64 // 最大相对误差 |a-n| / (|a|+|n|)
    MaxAbs float64 // 抽查元素中解析梯度的最大绝对值（判断梯度量级用）
    WAna   float64 // 相对误差最大的那个元素：解析梯度
    WNum   float64 // 相对误差最大的那个元素：数值梯度
}

// GradCheck 数值梯度检验：对每个参数矩阵随机抽 per 个元素，用中心差分
//
//    ∂L/∂θ ≈ [L(θ+ε) - L(θ-ε)] / 2ε,  ε = 1e-6
//
// 与 Backward 给出的解析梯度比较。全程 float64。
// ε 不能太大：初始化时残差流很小（std≈0.03），LayerNorm 把它放大约 35 倍，
// 函数曲率很大，ε=1e-5 时截断误差就可能到 1e-2；ε 也不能太小，否则舍入误差占主导。
func GradCheck(m *Model, xs, ys [][]int, per int, rng *rand.Rand) []GradCheckResult {
    const eps = 1e-6
    m.LossAndGrads(xs, ys, 1)
    var res []GradCheckResult
    for i, p := range m.Params {
        r := GradCheckResult{Name: m.Names[i]}
        n := per
        if n > len(p.D) {
            n = len(p.D)
        }
        for s := 0; s < n; s++ {
            j := rng.Intn(len(p.D))
            if n == len(p.D) {
                j = s
            }
            orig := p.D[j]
            p.D[j] = orig + eps
            lp := m.Loss(xs, ys)
            p.D[j] = orig - eps
            lm := m.Loss(xs, ys)
            p.D[j] = orig
            num := (lp - lm) / (2 * eps)
            ana := m.Grads[i].D[j]
            // 分母设下限 1e-4：梯度本身很小时（如 1e-7），差分的舍入误差（约 1e-10）
            // 会让"相对误差"虚高，此时改为按绝对误差/1e-4 衡量。
            den := math.Max(math.Abs(ana)+math.Abs(num), 1e-4)
            rel := math.Abs(ana-num) / den
            if rel > r.MaxRel {
                r.MaxRel, r.WAna, r.WNum = rel, ana, num
            }
            r.MaxAbs = math.Max(r.MaxAbs, math.Abs(ana))
            r.N++
        }
        res = append(res, r)
    }
    return res
}
```

```
package main

import (
    "flag"
```

```
    "fmt"
    "math"
    "math/rand"
)

// cmdBugCos 对同一个 batch 分别用正确的反向和"残差多加一份"的反向求梯度，
// 逐个参数矩阵比较两者的方向（余弦相似度）和大小（范数之比）。
// 用来解释第 6.5 节的现象：bug 版的梯度明显错了，loss 却照样下降。
func cmdBugCos(args []string) {
    fs := flag.NewFlagSet("bugcos", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径")
    fs.Parse(args)
    _, tk, ids := loadCorpus(*corpus)
    m := NewModel(DefaultConfig(tk.Size()), rand.New(rand.NewSource(1)))
    xs, ys := GetBatch(ids, 4, m.Cfg.T, rand.New(rand.NewSource(7)))

    m.Bug = BugNone
    m.LossAndGrads(xs, ys, 1)
    good := snapshot(m.Grads)
    m.Bug = BugResidualDouble
    m.LossAndGrads(xs, ys, 1)
    bad := snapshot(m.Grads)

    fmt.Printf("%-8s %10s %10s %14s\n", "参数", "余弦相似度", "范数之比", "Adam 首步同号比例")
    for i, name := range m.Names {
        var dot, ng, nb float64
        same := 0
        for j := range good[i] {
            dot += good[i][j] * bad[i][j]
            ng += good[i][j] * good[i][j]
            nb += bad[i][j] * bad[i][j]
            if (good[i][j] > 0) == (bad[i][j] > 0) {
                same++
            }
        }
        if ng == 0 || nb < 1e-30 {
            fmt.Printf("%-8s %10s %10s %14s\n", name, "-", "-", "-")
            continue
        }
        fmt.Printf("%-8s %10.4f %10.3f %13.1f%%\n", name, dot/math.Sqrt(ng*nb), math.Sqrt(nb/ng),
            100*float64(same)/float64(len(good[i])))
    }
}
```

A.9 Go：权重读写与推理服务（weights.go）

```
package main

import (
    "encoding/json"
    "flag"
    "fmt"
    "math/rand"
    "net/http"
    "os"
    "strconv"
    "time"
)

// WeightsFile 权重文件格式：Go 与 PyTorch 共用（PyTorch 侧见 py/export.py）。
// 参数按名字对应，矩阵一律按 Go 的布局（y = x·W，W 形状为 输入×输出）行优先展平。
type WeightsFile struct {
    Config Config      `json:"config"`
    Names  []string    `json:"names"`
    Shapes [][2]int    `json:"shapes"`
    Params [][]float64 `json:"params"`
}
```

```
func SaveWeights(m *Model, path string) {
    w := WeightsFile{Config: m.Cfg, Names: m.Names, Params: snapshot(m.Params)}
    for _, p := range m.Params {
        w.Shapes = append(w.Shapes, [2]int{p.R, p.C})
    }
    f, err := os.Create(path)
    if err != nil {
        panic(err)
    }
    defer f.Close()
    json.NewEncoder(f).Encode(w)
}

// LoadWeights 按配置建一个空模型，再按名字逐个拷贝参数，并校验形状。
func LoadWeights(path string) *Model {
    b, err := os.ReadFile(path)
    if err != nil {
        panic(err)
    }
    var w WeightsFile
    if err := json.Unmarshal(b, &w); err != nil {
        panic(err)
    }
    m := NewModel(w.Config, rand.New(rand.NewSource(0)))
    byName := map[string]int{}
    for i, n := range w.Names {
        byName[n] = i
    }
    for i, p := range m.Params {
        j, ok := byName[m.Names[i]]
        if !ok {
            panic("权重文件缺少参数 " + m.Names[i])
        }
        if w.Shapes[j] != [2]int{p.R, p.C} {
            panic(fmt.Sprintf("%s 形状不匹配: 文件 %v, 模型 %d×%d", m.Names[i], w.Shapes[j], p.R, p.C))
        }
        copy(p.D, w.Params[j])
    }
    return m
}

// cmdGen 加载权重（可以来自 PyTorch 训练）并生成文本。
func cmdGen(args []string) {
    fs := flag.NewFlagSet("gen", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径（只用来重建词表）")
    load := fs.String("load", "../out/py_weights.json", "权重文件")
    prompt := fs.String("prompt", "月落", "提示词")
    n := fs.Int("n", 30, "生成字数")
    temp := fs.Float64("temp", 0.8, "采样温度，0 表示贪心")
    topK := fs.Int("topk", 10, "top-k")
    seed := fs.Int64("seed", 1, "采样种子")
    fs.Parse(args)

    _, tk, ids := loadCorpus(*corpus)
    m := LoadWeights(*load)
    var all float64
    nwin := 0
    for i := 0; i+m.Cfg.T < len(ids); i++ {
        l, _ := CrossEntropy(m.Forward(ids[i:i+m.Cfg.T]).Logits, ids[i+1:i+m.Cfg.T+1])
        all += l
        nwin++
    }
    fmt.Printf("已加载 %s：参数量 %d，全部窗口平均 loss %.6f\n", *load, m.NumParams(), all/float64(nwin))
    out := m.Generate(tk.Encode(*prompt), *n, *temp, *topK, rand.New(rand.NewSource(*seed)))
    fmt.Printf("%q\n", tk.Decode(out))
}

// cmdServe 一个最小的 HTTP 推理服务：GET /generate?prompt=月落&n=30&temp=0.8
func cmdServe(args []string) {
    fs := flag.NewFlagSet("serve", flag.ExitOnError)
```

```
    corpus := fs.String("corpus", "../corpus.txt", "语料路径（只用来重建词表）")
    load := fs.String("load", "../out/py_weights.json", "权重文件")
    addr := fs.String("addr", "127.0.0.1:8080", "监听地址")
    fs.Parse(args)

    _, tk, _ := loadCorpus(*corpus)
    m := LoadWeights(*load) // 模型只读，可被多个请求并发使用
    http.HandleFunc("/generate", func(w http.ResponseWriter, r *http.Request) {
        q := r.URL.Query()
        n, _ := strconv.Atoi(q.Get("n"))
        if n <= 0 || n > 200 {
            n = 30
        }
        temp, err := strconv.ParseFloat(q.Get("temp"), 64)
        if err != nil {
            temp = 0.8
        }
        prompt := q.Get("prompt")
        for _, ch := range prompt { // 词表外的字直接拒绝，而不是 panic
            if _, ok := tk.stoi[ch]; !ok {
                http.Error(w, "字符不在词表中: "+string(ch), http.StatusBadRequest)
                return
            }
        }
        if prompt == "" {
            prompt = "月"
        }
        t0 := time.Now()
        rng := rand.New(rand.NewSource(time.Now().UnixNano())) // 每个请求独立的随机源
        out := tk.Decode(m.Generate(tk.Encode(prompt), n, temp, 10, rng))
        w.Header().Set("Content-Type", "application/json; charset=utf-8")
        json.NewEncoder(w).Encode(map[string]any{"text": out, "ms": time.Since(t0).Milliseconds()})
    })
    fmt.Println("listening on", *addr)
    if err := http.ListenAndServe(*addr, nil); err != nil {
        panic(err)
    }
}
```

A.10 Go：命令行入口（main.go）

```
package main

import (
    "encoding/json"
    "flag"
    "fmt"
    "math"
    "math/rand"
    "os"
    "time"
)

func loadCorpus(path string) (string, *Tokenizer, []int) {
    b, err := os.ReadFile(path)
    if err != nil {
        panic(err)
    }
    text := string(b)
    tk := NewTokenizer(text)
    return text, tk, tk.Encode(text)
}

func parseBug(s string) Bug {
    switch s {
    case "none":
        return BugNone
    case "residual":
        return BugResidualDouble
    case "nomask":
        return BugNoMask
    }
```

```
    panic("未知 bug 类型: " + s)
}

func main() {
    if len(os.Args) < 2 {
        fmt.Println("用法: nanogpt <train|gradcheck|align|kvbench|speed|walk|gen|serve> [flags]")
        os.Exit(2)
    }
    cmd, args := os.Args[1], os.Args[2:]
    switch cmd {
    case "train":
        cmdTrain(args)
    case "gradcheck":
        cmdGradCheck(args)
    case "align":
        cmdAlign(args)
    case "kvbench":
        cmdKVBench(args)
    case "speed":
        cmdSpeed(args)
    case "walk":
        cmdWalk(args)
    case "gen":
        cmdGen(args)
    case "bugcos":
        cmdBugCos(args)
    case "serve":
        cmdServe(args)
    default:
        fmt.Println("未知命令:", cmd)
        os.Exit(2)
    }
}

// ---------------- train ----------------

func cmdTrain(args []string) {
    fs := flag.NewFlagSet("train", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径")
    steps := fs.Int("steps", 600, "训练步数")
    lr := fs.Float64("lr", 3e-3, "学习率")
    B := fs.Int("B", 4, "每步窗口数")
    seed := fs.Int64("seed", 1, "随机种子")
    logPath := fs.String("log", "", "逐步 loss 写入的 CSV 路径")
    bug := fs.String("bug", "none", "none|residual|nomask")
    noBC := fs.Bool("nobc", false, "关闭 Adam 偏差修正")
    opt := fs.String("opt", "adam", "adam|sgd")
    workers := fs.Int("workers", 1, "并行 worker 数")
    every := fs.Int("every", 50, "每隔多少步打印一次")
    T := fs.Int("T", 16, "上下文窗口 block_size")
    D := fs.Int("d", 32, "d_model")
    L := fs.Int("layers", 2, "层数")
    H := fs.Int("heads", 2, "头数")
    F := fs.Int("dff", 0, "FFN 隐藏维（0 表示 2·d）")
    temp := fs.Float64("temp", 0.8, "采样温度")
    topK := fs.Int("topk", 10, "top-k（0 表示不截断）")
    save := fs.String("save", "", "训练结束后把权重保存为 JSON")
    fs.Parse(args)

    text, tk, ids := loadCorpus(*corpus)
    cfg := Config{V: tk.Size(), D: *D, NHead: *H, NLayer: *L, DFF: *F, T: *T, Std: 0.02}
    if cfg.DFF == 0 {
        cfg.DFF = 2 * cfg.D
    }
    m := NewModel(cfg, rand.New(rand.NewSource(*seed)))
    m.Bug = parseBug(*bug)
    dataRng := rand.New(rand.NewSource(*seed + 1000))
    fmt.Printf("语料 %d 字符, 词表 V=%d\n", len([]rune(text)), tk.Size())
    fmt.Printf("参数量: %d (d=%d, %d层, %d头, T=%d)\n", m.NumParams(), cfg.D, cfg.NLayer, cfg.NHead, cfg.T)
    fmt.Printf("理论初始 loss ln(V) = %.4f\n", math.Log(float64(tk.Size())))

    adam := NewAdam(m, *lr)
    adam.BiasCorrection = !*noBC
    var logf *os.File
    if *logPath != "" {
        logf, _ = os.Create(*logPath)
        defer logf.Close()
        fmt.Fprintln(logf, "step,loss")
    }
```

```
    start := time.Now()
    for step := 1; step <= *steps; step++ {
        xs, ys := GetBatch(ids, *B, cfg.T, dataRng)
        loss := m.LossAndGrads(xs, ys, *workers)
        if *opt == "sgd" {
            SGDStep(m.Params, m.Grads, *lr)
        } else {
            adam.Step(m.Params, m.Grads)
        }
        if logf != nil {
            fmt.Fprintf(logf, "%d,%.6f\n", step, loss)
        }
        if step == 1 || step%*every == 0 {
            fmt.Printf("step %4d  loss %.4f\n", step, loss)
        }
    }
    fmt.Printf("训练耗时: %.2fs\n", time.Since(start).Seconds())
    // 在全语料上评估一次平均 loss（所有窗口），比单步 loss 稳定得多
    var all float64
    nwin := 0
    for i := 0; i+cfg.T < len(ids); i++ {
        l, _ := CrossEntropy(m.Forward(ids[i:i+cfg.T]).Logits, ids[i+1:i+cfg.T+1])
        all += l
        nwin++
    }
    fmt.Printf("全部 %d 个窗口的平均 loss: %.4f\n", nwin, all/float64(nwin))

    fmt.Printf("--- 生成样例（temp=%.1f, top-k=%d）---\n", *temp, *topK)
    sampleRng := rand.New(rand.NewSource(*seed + 2000))
    var prompts []string
    for _, p := range []string{"月落", "春风", "两岸"} {
        ok := true
        for _, ch := range p {
            _, in := tk.stoi[ch]
            ok = ok && in
        }
        if ok { // 换了语料时提示词可能含词表外的字，跳过
            prompts = append(prompts, p)
        }
    }
    if len(prompts) == 0 {
        prompts = []string{string([]rune(text)[:2])}
    }
    for _, p := range prompts {
        out := m.Generate(tk.Encode(p), 30, *temp, *topK, sampleRng)
        fmt.Printf("%q\n", tk.Decode(out))
    }
    if *save != "" {
        SaveWeights(m, *save)
    }
}

// ---------------- gradcheck ----------------

func cmdGradCheck(args []string) {
    fs := flag.NewFlagSet("gradcheck", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径")
    bug := fs.String("bug", "none", "none|residual|nomask")
    per := fs.Int("per", 20, "每个参数矩阵抽查的元素数")
    trained := fs.Int("trained", 0, "先训练多少步再检验（让 γ 偏离 1、梯度更有代表性）")
    fs.Parse(args)

    _, tk, ids := loadCorpus(*corpus)
    cfg := DefaultConfig(tk.Size())
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    rng := rand.New(rand.NewSource(7))
    if *trained > 0 {
        adam := NewAdam(m, 3e-3)
        for s := 0; s < *trained; s++ {
            xs, ys := GetBatch(ids, 4, cfg.T, rng)
            m.LossAndGrads(xs, ys, 1)
            adam.Step(m.Params, m.Grads)
        }
    }
    m.Bug = parseBug(*bug)
    xs, ys := GetBatch(ids, 4, cfg.T, rng)
    res := GradCheck(m, xs, ys, *per, rng)
    worst := 0.0
    fmt.Printf("%-8s %4s %10s %10s   %s\n", "参数", "抽查", "最大相对误差", "|梯度|最大值", "最差元素: 解析 vs 数值")
```

```
    for _, r := range res {
        fmt.Printf("%-8s %4d %10.2e %10.2e   %+.6e vs %+.6e\n", r.Name, r.N, r.MaxRel, r.MaxAbs, r.WAna, r.WNum)
        worst = math.Max(worst, r.MaxRel)
    }
    fmt.Printf("全模型最大相对误差: %.2e\n", worst)
}

// ---------------- align：导出给 PyTorch 逐元素比对 ----------------

type AlignDump struct {
    Config Config      `json:"config"`
    Names  []string    `json:"names"`
    Shapes [][2]int    `json:"shapes"`
    Params [][]float64 `json:"params"`
    X      [][]int     `json:"x"`
    Y      [][]int     `json:"y"`
    Logits [][]float64 `json:"logits"` // 每个窗口一份 T×V，按行展平
    Loss   float64     `json:"loss"`
    Grads  [][]float64 `json:"grads"`
    After  [][]float64 `json:"after"` // 一步 Adam 之后的参数
}

func snapshot(ms []*Mat) [][]float64 {
    out := make([][]float64, len(ms))
    for i, p := range ms {
        out[i] = append([]float64{}, p.D...)
    }
    return out
}

func cmdAlign(args []string) {
    fs := flag.NewFlagSet("align", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径")
    out := fs.String("out", "../out/align.json", "输出路径")
    fs.Parse(args)

    _, tk, ids := loadCorpus(*corpus)
    cfg := DefaultConfig(tk.Size())
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    rng := rand.New(rand.NewSource(3))
    // 先训练 50 步，让 γ、β、偏置都不再是初始值，比对更有说服力
    adam := NewAdam(m, 3e-3)
    for s := 0; s < 50; s++ {
        xs, ys := GetBatch(ids, 4, cfg.T, rng)
        m.LossAndGrads(xs, ys, 1)
        adam.Step(m.Params, m.Grads)
    }
    d := AlignDump{Config: cfg, Names: m.Names}
    for _, p := range m.Params {
        d.Shapes = append(d.Shapes, [2]int{p.R, p.C})
    }
    d.Params = snapshot(m.Params)
    d.X, d.Y = GetBatch(ids, 4, cfg.T, rng)
    for _, x := range d.X {
        d.Logits = append(d.Logits, m.Forward(x).Logits.D)
    }
    d.Loss = m.LossAndGrads(d.X, d.Y, 1)
    d.Grads = snapshot(m.Grads)
    fresh := NewAdam(m, 3e-3) // 用一个全新的 Adam（t=1）走一步，PyTorch 侧同样新建优化器
    fresh.Step(m.Params, m.Grads)
    d.After = snapshot(m.Params)
    f, _ := os.Create(*out)
    defer f.Close()
    json.NewEncoder(f).Encode(d)
    fmt.Printf("已导出 %s：%d 个参数矩阵，loss=%.10f\n", *out, len(d.Names), d.Loss)
}

// ---------------- kvbench ----------------

func cmdKVBench(args []string) {
    fs := flag.NewFlagSet("kvbench", flag.ExitOnError)
    fs.Parse(args)
    // 用一个更"像样"的配置测速：T=512 才能体现 KV cache 的价值（权重随机即可）
    cfg := Config{V: 289, D: 64, NHead: 4, NLayer: 4, DFF: 256, T: 512, Std: 0.02}
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    fmt.Printf("配置: d=%d, %d层, %d头, T=%d, 参数量 %d\n", cfg.D, cfg.NLayer, cfg.NHead, cfg.T, m.NumParams())
    prompt := []int{1, 2, 3, 4}
    fmt.Printf("%8s %12s %12s %8s %s\n", "生成长度", "无cache(ms)", "KV cache(ms)", "加速比", "输出一致")
    for _, n := range []int{32, 64, 128, 256, 508} {
```

```
        t0 := time.Now()
        a := m.Generate(prompt, n, 0, 0, nil)
        t1 := time.Since(t0)
        t0 = time.Now()
        b := m.GenerateKV(prompt, n, 0, 0, nil)
        t2 := time.Since(t0)
        same := len(a) == len(b)
        for i := range a {
            same = same && a[i] == b[i]
        }
        fmt.Printf("%8d %12.1f %12.1f %7.1fx %v\n", n, ms(t1), ms(t2), t1.Seconds()/t2.Seconds(), same)
    }
}

func ms(d time.Duration) float64 { return float64(d.Microseconds()) / 1000 }

// ---------------- speed：并行训练加速比 ----------------

func cmdSpeed(args []string) {
    fs := flag.NewFlagSet("speed", flag.ExitOnError)
    corpus := fs.String("corpus", "../corpus.txt", "语料路径")
    fs.Parse(args)
    _, tk, ids := loadCorpus(*corpus)
    cfg := DefaultConfig(tk.Size())
    fmt.Printf("%4s %8s %14s %8s\n", "B", "workers", "每步耗时(ms)", "加速比")
    for _, B := range []int{4, 32} {
        var base float64
        for _, w := range []int{1, 2, 4, 8} {
            if w > B {
                continue
            }
            m := NewModel(cfg, rand.New(rand.NewSource(1)))
            rng := rand.New(rand.NewSource(2))
            adam := NewAdam(m, 3e-3)
            const n = 100
            t0 := time.Now()
            for s := 0; s < n; s++ {
                xs, ys := GetBatch(ids, B, cfg.T, rng)
                m.LossAndGrads(xs, ys, w)
                adam.Step(m.Params, m.Grads)
            }
            per := ms(time.Since(t0)) / n
            if w == 1 {
                base = per
            }
            fmt.Printf("%4d %8d %14.3f %7.2fx\n", B, w, per, base/per)
        }
    }
}

// ---------------- walk：微型数值走读 ----------------

func pm(name string, m *Mat) {
    fmt.Printf("%s (%d×%d)\n", name, m.R, m.C)
    for i := 0; i < m.R; i++ {
        for j := 0; j < m.C; j++ {
            fmt.Printf("%9.4f", m.D[i*m.C+j])
        }
        fmt.Println()
    }
}

func cmdWalk(args []string) {
    // d=4, T=3, V=5, 单头单层：所有中间量都能打印出来手算核对
    cfg := Config{V: 5, D: 4, NHead: 1, NLayer: 1, DFF: 8, T: 3, Std: 0.5}
    m := NewModel(cfg, rand.New(rand.NewSource(42)))
    x, y := []int{0, 3, 1}, []int{3, 1, 4}
    fmt.Println("参数量:", m.NumParams(), " 输入 ids:", x, " 目标:", y)
    fc := m.Forward(x)
    ly, lc := m.Layers[0], fc.Layers[0]
    pm("TokE", m.TokE)
    pm("Xln1 = LN1(X0)", lc.Xln1)
    pm("Q", lc.Heads[0].Q)
    pm("K", lc.Heads[0].K)
    pm("P = softmax(QKᵀ/√d + mask)", lc.Heads[0].P)
    pm("logits", fc.Logits)
    loss, dl := CrossEntropy(fc.Logits, y)
    fmt.Printf("loss = %.6f   (ln V = %.6f)\n", loss, math.Log(5))
    pm("dlogits = (softmax − onehot)/T", dl)
```

```
    m.ZeroGrads()
    m.Backward(fc, dl, m.Grads)
    pm("∂L/∂Wq", m.Grads[m.idx[ly.Wq]])
    pm("∂L/∂TokE", m.Grads[m.idx[m.TokE]])
    res := GradCheck(m, [][]int{x}, [][]int{y}, 1000, rand.New(rand.NewSource(1)))
    worst := 0.0
    for _, r := range res {
        worst = math.Max(worst, r.MaxRel)
    }
    fmt.Printf("全部 %d 个参数逐一做梯度检验，最大相对误差 %.2e\n", m.NumParams(), worst)
}
```

A.11 Go：测试（mat_test.go、ops_test.go、model_test.go）

```
package main

import (
    "math"
    "math/rand"
    "testing"
)

// ---------------- M1：矩阵工具 ----------------

func matOf(r, c int, v ...float64) *Mat { return &Mat{r, c, v} }

func assertClose(t *testing.T, name string, got, want []float64, tol float64) {
    t.Helper()
    if len(got) != len(want) {
        t.Fatalf("%s: 长度 %d != %d", name, len(got), len(want))
    }
    for i := range got {
        if math.Abs(got[i]-want[i]) > tol {
            t.Fatalf("%s[%d] = %v, 期望 %v", name, i, got[i], want[i])
        }
    }
}

func TestMM(t *testing.T) {
    a := matOf(2, 3, 1, 2, 3, 4, 5, 6) // [[1 2 3] [4 5 6]]
    b := matOf(3, 2, 7, 8, 9, 10, 11, 12)
    // 手算：[[1·7+2·9+3·11, 1·8+2·10+3·12], [4·7+5·9+6·11, 4·8+5·10+6·12]]
    assertClose(t, "a·b", MM(a, b, false, false).D, []float64{58, 64, 139, 154}, 0)
    // 四种转置组合：用"先显式转置再乘"作为参照
    at, bt := transpose(a), transpose(b)
    assertClose(t, "aᵀᵀ·b", MM(at, b, true, false).D, []float64{58, 64, 139, 154}, 0)
    assertClose(t, "a·bᵀᵀ", MM(a, bt, false, true).D, []float64{58, 64, 139, 154}, 0)
    assertClose(t, "aᵀᵀ·bᵀᵀ", MM(at, bt, true, true).D, []float64{58, 64, 139, 154}, 0)
}

func transpose(m *Mat) *Mat {
    o := NewMat(m.C, m.R)
    for i := 0; i < m.R; i++ {
        for j := 0; j < m.C; j++ {
            o.D[j*m.R+i] = m.D[i*m.C+j]
        }
    }
    return o
}

func TestBiasColSumSlice(t *testing.T) {
    x := matOf(2, 3, 1, 2, 3, 4, 5, 6)
    AddBias(x, matOf(1, 3, 10, 20, 30))
    assertClose(t, "AddBias", x.D, []float64{11, 22, 33, 14, 25, 36}, 0)
    assertClose(t, "ColSum", ColSum(x).D, []float64{25, 47, 69}, 0)
    s := SliceCols(x, 1, 3)
    assertClose(t, "SliceCols", s.D, []float64{22, 33, 25, 36}, 0)
    y := NewMat(2, 3)
```

```
    SetCols(y, s, 1)
    assertClose(t, "SetCols", y.D, []float64{0, 22, 33, 0, 25, 36}, 0)
}

// ---------------- M2：分词与取 batch ----------------

func TestTokenizerAndBatch(t *testing.T) {
    text := "床前明月光，疑是地上霜。\n"
    tk := NewTokenizer(text)
    // 10 个不重复汉字 + "，" "。" "\n" 三个符号 = 13
    if tk.Size() != 13 {
        t.Fatalf("词表大小 %d，期望 13", tk.Size())
    }
    ids := tk.Encode(text)
    if tk.Decode(ids) != text {
        t.Fatal("encode→decode 往返不一致")
    }
    rng := rand.New(rand.NewSource(1))
    x, y := GetBatch(ids, 8, 4, rng)
    for b := range x {
        for i := 0; i < 3; i++ {
            if x[b][i+1] != y[b][i] {
                t.Fatal("y 应当是 x 右移一位")
            }
        }
    }
    // 合法起点有 N-T 个，最后一个起点 N-T-1 必须能被采到
    seenLast := false
    for k := 0; k < 2000; k++ {
        x, _ := GetBatch(ids, 1, 4, rng)
        if &x[0][0] == &ids[len(ids)-4-1] {
            seenLast = true
        }
    }
    if !seenLast {
        t.Fatal("最后一个合法窗口从未被采到")
    }
}
```

```
package main

import (
    "math"
    "math/rand"
    "testing"
)

// ---------------- M3：单算子梯度检验 ----------------
//
// 套路：构造标量 L = Σ out ⊙ R（R 为随机矩阵），则 ∂L/∂out = R。
// 用解析反向得到 ∂L/∂input，再与中心差分逐元素比较。

func randMat(r, c int, rng *rand.Rand) *Mat { return randn(r, c, 1, rng) }

func dot(a, b *Mat) float64 {
    var s float64
    for i := range a.D {
        s += a.D[i] * b.D[i]
    }
    return s
}

// checkGrad 对 x 的每个元素做中心差分，与 analytic 比较。
func checkGrad(t *testing.T, name string, x, analytic *Mat, loss func() float64) {
    t.Helper()
    const eps = 1e-6
    worst := 0.0
    for j := range x.D {
        orig := x.D[j]
        x.D[j] = orig + eps
        lp := loss()
        x.D[j] = orig - eps
```

```
        lm := loss()
        x.D[j] = orig
        num := (lp - lm) / (2 * eps)
        den := math.Abs(num) + math.Abs(analytic.D[j])
        if den > 1e-9 {
            worst = math.Max(worst, math.Abs(num-analytic.D[j])/den)
        }
    }
    if worst > 1e-6 {
        t.Fatalf("%s 梯度检验失败：最大相对误差 %.2e", name, worst)
    }
    t.Logf("%-14s 最大相对误差 %.2e", name, worst)
}

func TestLinearGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(1))
    x, w, b, R := randMat(5, 4, rng), randMat(4, 3, rng), randMat(1, 3, rng), randMat(5, 3, rng)
    f := func() float64 { y := MM(x, w, false, false); AddBias(y, b); return dot(y, R) }
    checkGrad(t, "Linear dX", x, MM(R, w, false, true), f) // dX = dY·Wᵀ
    checkGrad(t, "Linear dW", w, MM(x, R, true, false), f) // dW = Xᵀ·dY
    checkGrad(t, "Linear db", b, ColSum(R), f)             // db = colsum(dY)
}

func TestReLUGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(2))
    h, R := randMat(4, 6, rng), randMat(4, 6, rng)
    checkGrad(t, "ReLU", h, ReLUBackward(R, h), func() float64 { return dot(ReLU(h), R) })
}

func TestLayerNormGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(3))
    x, R := randMat(4, 8, rng), randMat(4, 8, rng)
    gamma, beta := randMat(1, 8, rng), randMat(1, 8, rng) // γ 刻意取非均匀值
    f := func() float64 { y, _, _ := LayerNorm(x, gamma, beta); return dot(y, R) }
    _, xhat, rstd := LayerNorm(x, gamma, beta)
    dg, db := NewMat(1, 8), NewMat(1, 8)
    dx := LNBackward(R, xhat, gamma, rstd, dg, db)
    checkGrad(t, "LN dx", x, dx, f)
    checkGrad(t, "LN dγ", gamma, dg, f)
    checkGrad(t, "LN dβ", beta, db, f)
}

// 第 1 版书稿里的 LN 反向公式：γ_j 提到括号外，括号内求和却用 dy 而非 γ⊙dy。
// 只有 γ 各分量相等时它才碰巧正确。这个测试证明它在 γ 非均匀时是错的。
func lnBackwardWrong(dy, xhat, gamma *Mat, rstd []float64) *Mat {
    n := dy.C
    dx := NewMat(dy.R, n)
    for i := 0; i < dy.R; i++ {
        var s1, s2 float64
        for k := 0; k < n; k++ {
            s1 += dy.D[i*n+k]
            s2 += dy.D[i*n+k] * xhat.D[i*n+k]
        }
        for j := 0; j < n; j++ {
            dx.D[i*n+j] = gamma.D[j] * rstd[i] * (dy.D[i*n+j] - s1/float64(n) - xhat.D[i*n+j]*s2/float64(n))
        }
    }
    return dx
}

func TestLayerNormWrongFormula(t *testing.T) {
    rng := rand.New(rand.NewSource(3))
    x, R := randMat(4, 8, rng), randMat(4, 8, rng)
    gamma := randMat(1, 8, rng)
    _, xhat, rstd := LayerNorm(x, gamma, NewMat(1, 8))
    right := LNBackward(R, xhat, gamma, rstd, NewMat(1, 8), NewMat(1, 8))
    wrong := lnBackwardWrong(R, xhat, gamma, rstd)
    var maxDiff float64
    for i := range right.D {
        maxDiff = math.Max(maxDiff, math.Abs(right.D[i]-wrong.D[i]))
    }
    if maxDiff < 1e-3 {
        t.Fatal("错误公式在 γ 非均匀时应当与正确结果明显不同")
    }
    t.Logf("γ 非均匀时，错误公式与正确梯度的最大差 %.3f", maxDiff)
}
```

```
func TestSoftmaxGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(4))
    s, R := randMat(5, 5, rng), randMat(5, 5, rng)
    f := func() float64 { return dot(Softmax(s, true), R) }
    checkGrad(t, "Softmax(因果)", s, SoftmaxBackward(R, Softmax(s, true)), f)
}

func TestCrossEntropyGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(5))
    logits, y := randMat(4, 7, rng), []int{1, 6, 0, 3}
    _, dl := CrossEntropy(logits, y)
    checkGrad(t, "CrossEntropy", logits, dl, func() float64 { l, _ := CrossEntropy(logits, y); return l })
}

// 单头注意力：C = softmax(mask(Q·Kᵀ/√dh))·V，验证缩放系数只乘一次。
func TestAttentionGrad(t *testing.T) {
    rng := rand.New(rand.NewSource(6))
    T, dh := 5, 4
    q, k, v, R := randMat(T, dh, rng), randMat(T, dh, rng), randMat(T, dh, rng), randMat(T, dh, rng)
    scale := 1 / math.Sqrt(float64(dh))
    fwd := func() (*Mat, *Mat) {
        s := MM(q, k, false, true)
        Scale(s, scale)
        p := Softmax(s, true)
        return MM(p, v, false, false), p
    }
    f := func() float64 { c, _ := fwd(); return dot(c, R) }
    _, p := fwd()
    dP := MM(R, v, false, true)
    dV := MM(p, R, true, false)
    dS := SoftmaxBackward(dP, p)
    Scale(dS, scale)
    checkGrad(t, "Attn dQ", q, MM(dS, k, false, false), f)
    checkGrad(t, "Attn dK", k, MM(dS, q, true, false), f)
    checkGrad(t, "Attn dV", v, dV, f)
}
```

```
package main

import (
    "math"
    "math/rand"
    "testing"
)

func testSetup(t *testing.T) (*Tokenizer, []int, Config) {
    t.Helper()
    _, tk, ids := loadCorpus("../corpus.txt")
    return tk, ids, DefaultConfig(tk.Size())
}

// ---------------- M5：完整模型 ----------------

func TestParamCount(t *testing.T) {
    tk, _, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    V, D, F := cfg.V, cfg.D, cfg.DFF
    perLayer := 3*(D*D+D) + (D*D + D) + (D*F + F) + (F*D + D) + 2*2*D
    want := V*D + cfg.T*D + cfg.NLayer*perLayer + 2*D
    if m.NumParams() != want {
        t.Fatalf("参数量 %d，手算 %d", m.NumParams(), want)
    }
    t.Logf("V=%d，参数量 %d（每层 %d）", tk.Size(), want, perLayer)
}

func TestInitLoss(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    rng := rand.New(rand.NewSource(2))
    xs, ys := GetBatch(ids, 16, cfg.T, rng)
    loss := m.Loss(xs, ys)
```

```
    if math.Abs(loss-math.Log(float64(cfg.V))) > 0.05 {
        t.Fatalf("初始 loss %.4f 偏离 ln V = %.4f", loss, math.Log(float64(cfg.V)))
    }
    t.Logf("初始 loss %.4f，ln V = %.4f", loss, math.Log(float64(cfg.V)))
}

func TestGradCheckFull(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    rng := rand.New(rand.NewSource(3))
    xs, ys := GetBatch(ids, 2, cfg.T, rng)
    for _, r := range GradCheck(m, xs, ys, 10, rng) {
        if r.MaxRel > 1e-4 {
            t.Fatalf("%s 梯度检验失败：%.2e", r.Name, r.MaxRel)
        }
    }
}

func TestGradCheckCatchesResidualBug(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    m.Bug = BugResidualDouble
    rng := rand.New(rand.NewSource(3))
    xs, ys := GetBatch(ids, 2, cfg.T, rng)
    worst := 0.0
    for _, r := range GradCheck(m, xs, ys, 10, rng) {
        worst = math.Max(worst, r.MaxRel)
    }
    if worst < 1e-2 {
        t.Fatalf("梯度检验没有抓到残差 bug（最大误差 %.2e）", worst)
    }
}

// 因果性测试：改动位置 t+1 之后的输入，位置 0..t 的 logits 必须完全不变。
// 梯度检验抓不到"掩码写错"（前向和反向是自洽的），只有这个测试能抓到。
func TestCausality(t *testing.T) {
    _, ids, cfg := testSetup(t)
    for _, bug := range []Bug{BugNone, BugNoMask} {
        m := NewModel(cfg, rand.New(rand.NewSource(1)))
        m.Bug = bug
        a := append([]int{}, ids[:cfg.T]...)
        b := append([]int{}, a...)
        b[10] = (b[10] + 1) % cfg.V // 只改第 10 个位置
        la, lb := m.Forward(a).Logits, m.Forward(b).Logits
        leak := 0.0
        for i := 0; i < 10*cfg.V; i++ { // 位置 0..9
            leak = math.Max(leak, math.Abs(la.D[i]-lb.D[i]))
        }
        if bug == BugNone && leak != 0 {
            t.Fatalf("正常模型泄露了未来信息：%.2e", leak)
        }
        if bug == BugNoMask && leak == 0 {
            t.Fatal("去掉掩码后应当能检测到泄露")
        }
        t.Logf("bug=%d 位置 0..9 的 logits 最大变化 %.2e", bug, leak)
    }
}

// ---------------- M6：单 batch 过拟合 ----------------

func TestOverfitOneBatch(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    xs, ys := GetBatch(ids, 4, cfg.T, rand.New(rand.NewSource(4)))
    adam := NewAdam(m, 3e-3)
    var loss float64
    for s := 1; s <= 200; s++ {
        loss = m.LossAndGrads(xs, ys, 1)
        adam.Step(m.Params, m.Grads)
```

```
    }
    if loss > 0.05 {
        t.Fatalf("固定一个 batch 训练 200 步后 loss=%.4f，应当 < 0.05", loss)
    }
    t.Logf("单 batch 200 步后 loss = %.4f", loss)
}

// ---------------- 并行与 KV cache ----------------

func TestParallelMatchesSerial(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    xs, ys := GetBatch(ids, 8, cfg.T, rand.New(rand.NewSource(5)))
    l1 := m.LossAndGrads(xs, ys, 1)
    serial := snapshot(m.Grads)
    l4 := m.LossAndGrads(xs, ys, 4)
    if math.Abs(l1-l4) > 1e-12 {
        t.Fatalf("并行 loss 不一致 %v vs %v", l1, l4)
    }
    for i := range serial {
        for j := range serial[i] {
            if math.Abs(serial[i][j]-m.Grads[i].D[j]) > 1e-12 {
                t.Fatalf("%s 并行梯度不一致", m.Names[i])
            }
        }
    }
}

func TestKVCacheMatchesFull(t *testing.T) {
    _, ids, cfg := testSetup(t)
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    kv := m.NewKVCache()
    seq := ids[:cfg.T]
    worst := 0.0
    for t0 := 0; t0 < cfg.T; t0++ {
        step := m.Step(seq[t0], kv)
        full := m.Forward(seq[:t0+1]).Logits
        row := full.D[t0*cfg.V : (t0+1)*cfg.V]
        for j := range row {
            worst = math.Max(worst, math.Abs(row[j]-step[j]))
        }
    }
    if worst > 1e-12 {
        t.Fatalf("KV cache 与完整前向不一致：%.2e", worst)
    }
    t.Logf("KV cache 逐位置 logits 最大差 %.2e", worst)
}

func BenchmarkTrainStepB4(b *testing.B) {
    _, tk, ids := loadCorpus("../corpus.txt")
    cfg := DefaultConfig(tk.Size())
    m := NewModel(cfg, rand.New(rand.NewSource(1)))
    rng := rand.New(rand.NewSource(2))
    for i := 0; i < b.N; i++ {
        xs, ys := GetBatch(ids, 4, cfg.T, rng)
        m.LossAndGrads(xs, ys, 1)
    }
}
```

A.12 Python：PyTorch 版（nanogpt.py）

```
"""nanoGPT · PyTorch 版：与 Go 版同一模型、同一语料、同一训练配置。

运行：python nanogpt.py [--seed 1] [--steps 600] [--init nanogpt|default] [--log out.csv]
"""
import argparse
import json
import math
import time
from dataclasses import dataclass

import torch
import torch.nn as nn
import torch.nn.functional as F


@dataclass
class Config:
    V: int = 289    # 词表大小（由语料决定）
    D: int = 32     # d_model
    NH: int = 2     # 头数
    NL: int = 2     # 层数
    DFF: int = 64   # FFN 隐藏维
    T: int = 16     # block_size


# ---------------- 数据 ----------------

def load_corpus(path):
    text = open(path, encoding="utf-8").read()     # 显式 utf-8，含换行与标点
    chars = sorted(set(text))                       # 按码点排序，与 Go 版词表 id 完全一致
    stoi = {c: i for i, c in enumerate(chars)}
    ids = torch.tensor([stoi[c] for c in text], dtype=torch.long)
    return text, chars, stoi, ids


def get_batch(ids, B, T, gen, device):
    ix = torch.randint(0, len(ids) - T, (B,), generator=gen)  # 合法起点共 N-T 个
    x = torch.stack([ids[i:i + T] for i in ix])
    y = torch.stack([ids[i + 1:i + T + 1] for i in ix])
    return x.to(device), y.to(device)


# ---------------- 模型 ----------------

class Block(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.c = c
        self.ln1 = nn.LayerNorm(c.D)
        self.qkv = nn.Linear(c.D, 3 * c.D)   # Wq/Wk/Wv 合并成一个投影
        self.proj = nn.Linear(c.D, c.D)      # 即 Go 版的 Wo
        self.ln2 = nn.LayerNorm(c.D)
        self.ffn = nn.Sequential(nn.Linear(c.D, c.DFF), nn.ReLU(), nn.Linear(c.DFF, c.D))

    def forward(self, x):
        B, T, D = x.shape
        nh, dh = self.c.NH, D // self.c.NH
        q, k, v = self.qkv(self.ln1(x)).split(D, dim=2)
        q = q.view(B, T, nh, dh).transpose(1, 2)        # (B, nh, T, dh)：切头
        k = k.view(B, T, nh, dh).transpose(1, 2)
        v = v.view(B, T, nh, dh).transpose(1, 2)
        o = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        o = o.transpose(1, 2).contiguous().view(B, T, D)  # 拼回
        x = x + self.proj(o)                              # 残差 1
        x = x + self.ffn(self.ln2(x))                     # 残差 2
        return x
```

```
class GPT(nn.Module):
    def __init__(self, c: Config, init="nanogpt"):
        super().__init__()
        self.c = c
        self.tok = nn.Embedding(c.V, c.D)
        self.pos = nn.Embedding(c.T, c.D)
        self.blocks = nn.ModuleList([Block(c) for _ in range(c.NL)])
        self.lnf = nn.LayerNorm(c.D)
        if init == "nanogpt":
            self.apply(self._init_weights)  # 不调用则使用 PyTorch 默认初始化（见 6.3 节）

    @staticmethod
    def _init_weights(m):
        if isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)
            nn.init.zeros_(m.bias)
        elif isinstance(m, nn.Embedding):
            nn.init.normal_(m.weight, mean=0.0, std=0.02)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        for blk in self.blocks:
            x = blk(x)
        return self.lnf(x) @ self.tok.weight.T            # weight tying


# ---------------- 训练、评估、生成 ----------------

def train(model, ids, steps=600, B=4, lr=3e-3, seed=1, device="cpu", log=None, every=50):
    gen = torch.Generator().manual_seed(seed + 1000)
    opt = torch.optim.Adam(model.parameters(), lr=lr, betas=(0.9, 0.999), eps=1e-8)
    V, T, losses = model.c.V, model.c.T, []
    for step in range(1, steps + 1):
        x, y = get_batch(ids, B, T, gen, device)
        logits = model(x)
        loss = F.cross_entropy(logits.view(-1, V), y.view(-1))
        opt.zero_grad()
        loss.backward()   # ← Go 版的整个 Backward
        opt.step()        # ← Go 版的 Adam.Step
        losses.append(loss.item())
        if log and (step == 1 or step % every == 0):
            log(f"step {step:4d}  loss {loss.item():.4f}")
    return losses


@torch.no_grad()
def eval_all(model, ids):
    """全部窗口上的平均 loss，与 Go 版的同名统计口径一致。"""
    T = model.c.T
    xs = torch.stack([ids[i:i + T] for i in range(len(ids) - T)])
    ys = torch.stack([ids[i + 1:i + T + 1] for i in range(len(ids) - T)])
    dev = next(model.parameters()).device
    logits = model(xs.to(dev))
    return F.cross_entropy(logits.reshape(-1, model.c.V), ys.to(dev).reshape(-1)).item()


@torch.no_grad()
def generate(model, prompt_ids, n, temp=0.8, top_k=10, gen=None):
    idx = torch.tensor([prompt_ids], dtype=torch.long, device=next(model.parameters()).device)
    for _ in range(n):
        logits = model(idx[:, -model.c.T:])[0, -1] / temp   # 只看最近 T 个、只取最后一个位置
        if top_k:
            kth = torch.topk(logits, top_k).values[-1]
            logits[logits < kth] = -float("inf")
        nxt = torch.multinomial(F.softmax(logits, dim=-1), 1, generator=gen)
        idx = torch.cat([idx, nxt.view(1, 1)], dim=1)
    return idx[0].tolist()
```

```
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--corpus", default="../corpus.txt")
    ap.add_argument("--seed", type=int, default=1)
    ap.add_argument("--steps", type=int, default=600)
    ap.add_argument("--init", choices=["nanogpt", "default"], default="nanogpt")
    ap.add_argument("--log", default="")
    ap.add_argument("--device", default="cuda" if torch.cuda.is_available() else "cpu")
    a = ap.parse_args()

    torch.set_num_threads(1)  # 小模型多线程反而更慢，也便于复现
    torch.manual_seed(a.seed)
    text, chars, stoi, ids = load_corpus(a.corpus)
    c = Config(V=len(chars))
    model = GPT(c, init=a.init).to(a.device)
    print(f"语料 {len(text)} 字符, 词表 V={c.V}")
    print(f"参数量: {sum(p.numel() for p in model.parameters())} "
          f"(d={c.D}, {c.NL}层, {c.NH}头, T={c.T})")
    print(f"理论初始 loss ln(V) = {math.log(c.V):.4f}")
    t0 = time.time()
    losses = train(model, ids, steps=a.steps, seed=a.seed, device=a.device, log=print)
    print(f"训练耗时: {time.time() - t0:.2f}s")
    print(f"全部 {len(ids) - c.T} 个窗口的平均 loss: {eval_all(model, ids):.4f}")
    if a.log:
        with open(a.log, "w") as f:
            f.write("step,loss\n")
            f.writelines(f"{i + 1},{l:.6f}\n" for i, l in enumerate(losses))
    print("--- 生成样例（temp=0.8, top-k=10）---")
    gen = torch.Generator().manual_seed(a.seed + 2000)
    for p in ["月落", "春风", "两岸"]:
        out = generate(model, [stoi[ch] for ch in p], 30, gen=gen)
        print(json.dumps("".join(chars[i] for i in out), ensure_ascii=False))  # 与 Go 的 %q 格式一致


if __name__ == "__main__":
    main()
```

A.13 Python：跨实现对齐（align.py）

```
"""跨实现逐元素对齐：加载 Go 导出的权重与 batch，比对 logits、loss、全部梯度和一步 Adam 后的参数。

运行：cd go && ./nanogpt align && cd ../py && python align.py
"""
import json
import sys

import torch
import torch.nn.functional as F

from nanogpt import GPT, Config


def to_go_layout(model, c):
    """把 PyTorch 模型的参数（或梯度）按 Go 的命名与形状导出，便于逐个比较。"""
    def get(t, grad):
        return (t.grad if grad else t.data).detach()

    def export(grad):
        out = {"tok": get(model.tok.weight, grad), "pos": get(model.pos.weight, grad)}
        D = c.D
        for l, b in enumerate(model.blocks):
            p = f"l{l}."
            W, bias = get(b.qkv.weight, grad), get(b.qkv.bias, grad)  # (3D, D), (3D,)
            out[p + "ln1.g"], out[p + "ln1.b"] = get(b.ln1.weight, grad), get(b.ln1.bias, grad)
            for i, n in enumerate("qkv"):
                out[p + "w" + n] = W[i * D:(i + 1) * D].T      # nn.Linear 存的是 Wᵀ
                out[p + "b" + n] = bias[i * D:(i + 1) * D]
            out[p + "wo"], out[p + "bo"] = get(b.proj.weight, grad).T, get(b.proj.bias, grad)
            out[p + "ln2.g"], out[p + "ln2.b"] = get(b.ln2.weight, grad), get(b.ln2.bias, grad)
            out[p + "w1"], out[p + "b1"] = get(b.ffn[0].weight, grad).T, get(b.ffn[0].bias, grad)
```

```
            out[p + "w2"], out[p + "b2"] = get(b.ffn[2].weight, grad).T, get(b.ffn[2].bias, grad)
        out["lnf.g"], out["lnf.b"] = get(model.lnf.weight, grad), get(model.lnf.bias, grad)
        return out
    return export


@torch.no_grad()
def load_from_go(model, d, c):
    P = {n: torch.tensor(v).view(*s) for n, s, v in zip(d["names"], d["shapes"], d["params"])}
    model.tok.weight.copy_(P["tok"])
    model.pos.weight.copy_(P["pos"])
    for l, b in enumerate(model.blocks):
        p = f"l{l}."
        b.ln1.weight.copy_(P[p + "ln1.g"].view(-1)); b.ln1.bias.copy_(P[p + "ln1.b"].view(-1))
        b.qkv.weight.copy_(torch.cat([P[p + "wq"].T, P[p + "wk"].T, P[p + "wv"].T], dim=0))
        b.qkv.bias.copy_(torch.cat([P[p + "bq"], P[p + "bk"], P[p + "bv"]], dim=1).view(-1))
        b.proj.weight.copy_(P[p + "wo"].T); b.proj.bias.copy_(P[p + "bo"].view(-1))
        b.ln2.weight.copy_(P[p + "ln2.g"].view(-1)); b.ln2.bias.copy_(P[p + "ln2.b"].view(-1))
        b.ffn[0].weight.copy_(P[p + "w1"].T); b.ffn[0].bias.copy_(P[p + "b1"].view(-1))
        b.ffn[2].weight.copy_(P[p + "w2"].T); b.ffn[2].bias.copy_(P[p + "b2"].view(-1))
    model.lnf.weight.copy_(P["lnf.g"].view(-1)); model.lnf.bias.copy_(P["lnf.b"].view(-1))


def run(dtype, d):
    torch.set_default_dtype(dtype)
    cf = d["config"]
    c = Config(V=cf["V"], D=cf["D"], NH=cf["NHead"], NL=cf["NLayer"], DFF=cf["DFF"], T=cf["T"])
    model = GPT(c)
    load_from_go(model, d, c)
    export = to_go_layout(model, c)
    x, y = torch.tensor(d["x"]), torch.tensor(d["y"])
    logits = model(x)
    loss = F.cross_entropy(logits.view(-1, c.V), y.view(-1))
    loss.backward()
    go_logits = torch.tensor(d["logits"]).view(len(d["x"]), c.T, c.V)

    def maxdiff(ours, key):
        return max((ours[n].to(torch.float64).reshape(-1) - torch.tensor(v)).abs().max().item()
                   for n, v in zip(d["names"], d[key]))

    r = {"logits": (logits.detach().double() - go_logits).abs().max().item(),
         "loss": abs(loss.item() - d["loss"]),
         "grads": maxdiff(export(True), "grads")}
    worst = max(zip(d["names"], d["grads"]),
                key=lambda nv: (export(True)[nv[0]].double().reshape(-1) - torch.tensor(nv[1])).abs().max())
    r["worst_grad_param"] = worst[0]
    opt = torch.optim.Adam(model.parameters(), lr=3e-3, betas=(0.9, 0.999), eps=1e-8)
    opt.step()
    r["after_adam"] = maxdiff(export(False), "after")
    after = export(False)
    r["worst_after_param"] = max(zip(d["names"], d["after"]),
                                 key=lambda nv: (after[nv[0]].double().reshape(-1) - ↩
                                     torch.tensor(nv[1])).abs().max())[0]
    return r


if __name__ == "__main__":
    path = sys.argv[1] if len(sys.argv) > 1 else "../out/align.json"
    d = json.load(open(path))
    print(f"Go 侧 loss = {d['loss']:.12f}")
    print(f"{'精度':8s} {'logits':>10s} {'loss':>10s} {'全部梯度':>10s} {'Adam一步后':>10s}  误差最大的梯度 / 参数")
    for name, dt in [("float64", torch.float64), ("float32", torch.float32)]:
        r = run(dt, d)
        print(f"{name:8s} {r['logits']:10.2e} {r['loss']:10.2e} {r['grads']:10.2e} "
              f"{r['after_adam']:10.2e}  {r['worst_grad_param']} / {r['worst_after_param']}")
```

A.14 Python：权重导出（export.py）

```
"""PyTorch 训练 → 导出 Go 可加载的权重：python export.py [--steps 3000]

导出后：cd ../go && ./nanogpt gen -load ../out/py_weights.json -prompt 月落
"""
import argparse
import json

import torch

from align import to_go_layout
from nanogpt import GPT, Config, load_corpus, train, eval_all

ap = argparse.ArgumentParser()
ap.add_argument("--steps", type=int, default=3000)
ap.add_argument("--out", default="../out/py_weights.json")
a = ap.parse_args()

torch.set_num_threads(1)
torch.manual_seed(1)
text, chars, stoi, ids = load_corpus("../corpus.txt")
c = Config(V=len(chars))
model = GPT(c)
train(model, ids, steps=a.steps, seed=1)
print(f"PyTorch 侧：全部窗口平均 loss {eval_all(model, ids):.6f}")

params = to_go_layout(model, c)(False)
names = ["tok", "pos"]
for l in range(c.NL):
    names += [f"l{l}.{n}" for n in ["ln1.g", "ln1.b", "wq", "bq", "wk", "bk", "wv", "bv", "wo", "bo",
                                    "ln2.g", "ln2.b", "w1", "b1", "w2", "b2"]]
names += ["lnf.g", "lnf.b"]
shapes = [list(params[n].shape) if params[n].dim() == 2 else [1, params[n].numel()] for n in names]
cfg = {"V": c.V, "D": c.D, "NHead": c.NH, "NLayer": c.NL, "DFF": c.DFF, "T": c.T, "Std": 0.02}
json.dump({"config": cfg, "names": names, "shapes": shapes,
           "params": [params[n].double().reshape(-1).tolist() for n in names]}, open(a.out, "w"))
print(f"已导出 {a.out}")
```

A.15 Python：第 5、6 章对照实验（experiments.py）

```
"""第 5/6 章的 PyTorch 对照实验。输出写入 ../out/。

python experiments.py
"""
import json
import math
import statistics as st

import torch
import torch.nn as nn
import torch.nn.functional as F

from nanogpt import GPT, Config, load_corpus, train, eval_all

torch.set_num_threads(1)
text, chars, stoi, ids = load_corpus("../corpus.txt")
c = Config(V=len(chars))
res = {}


def run(seed, init="nanogpt", act=None, steps=600):
    torch.manual_seed(seed)
    m = GPT(c, init=init)
    if act is not None:
        for b in m.blocks:
            b.ffn[1] = act
```

```
    losses = train(m, ids, steps=steps, seed=seed)
    return m, losses


# 1) 5 个种子的 PyTorch 基线，与 Go 版 5 个种子对照
base = []
for s in range(1, 6):
    m, losses = run(s)
    with open(f"../out/py_s{s}.csv", "w") as f:
        f.write("step,loss\n")
        f.writelines(f"{i + 1},{l:.6f}\n" for i, l in enumerate(losses))
    base.append({"seed": s, "step1": losses[0], "final": losses[-1], "eval": eval_all(m, ids)})
res["py_seeds"] = base

# 2) 初始化：默认 vs nanogpt 惯例；以及"不做 weight tying"的对照组
@torch.no_grad()
def init_stats(m):
    x = torch.stack([ids[i:i + c.T] for i in range(0, 64)])
    y = torch.stack([ids[i + 1:i + c.T + 1] for i in range(0, 64)])
    logits = m(x)
    return logits.std().item(), F.cross_entropy(logits.view(-1, c.V), y.view(-1)).item()


class GPTUntied(GPT):
    """输出头改为独立的 nn.Linear（默认初始化），用于验证"坑一"的根因在 weight tying。"""
    def __init__(self, c, init):
        super().__init__(c, init)
        self.head = nn.Linear(c.D, c.V, bias=False)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        for blk in self.blocks:
            x = blk(x)
        return self.head(self.lnf(x))


rows = []
for name, ctor in [("默认初始化 + tying", lambda: GPT(c, init="default")),
                   ("std=0.02 + tying", lambda: GPT(c, init="nanogpt")),
                   ("默认初始化 + 独立输出头", lambda: GPTUntied(c, init="default"))]:
    stds, ls = [], []
    for s in range(1, 6):
        torch.manual_seed(s)
        sd, l0 = init_stats(ctor())
        stds.append(sd); ls.append(l0)
    rows.append({"name": name, "logit_std": st.mean(stds), "init_loss": st.mean(ls)})
res["init_table"] = rows
res["theory"] = {"sqrtD": math.sqrt(c.D), "lnV": math.log(c.V),
                 "approx_loss_sigma_sqrt2lnV": math.sqrt(c.D) * math.sqrt(2 * math.log(c.V))}

curves = {}
for name, init in [("default", "default"), ("nanogpt", "nanogpt")]:
    m, losses = run(1, init=init)
    curves[name] = losses
    res[f"init_{name}"] = {"step1": losses[0], "step100": losses[99], "step300": losses[299],
                           "step600": losses[-1], "eval": eval_all(m, ids)}
json.dump(curves, open("../out/py_init_curves.json", "w"))

# 3) ReLU vs GELU：5 个种子取均值
for name, act in [("relu", None), ("gelu", nn.GELU())]:
    ev = [eval_all(run(s, act=act)[0], ids) for s in range(1, 6)]
    res[f"act_{name}"] = {"mean": st.mean(ev), "std": st.stdev(ev), "all": ev}

json.dump(res, open("../out/py_experiments.json", "w"), ensure_ascii=False, indent=1)
print(json.dumps(res, ensure_ascii=False, indent=1))
```

A.16 Python：第 7 章实验（modern.py）

```
"""第 7 章：在更大的语料（《全唐诗》前 2 万首，约 123 万字）上做进阶实验。

- 训练治理：warmup + 余弦退火、梯度裁剪、dropout
- 现代组件：RMSNorm、RoPE、SwiGLU、GQA
- 规模：d_model ∈ {32, 64, 128}

python modern.py [实验名 ...]    结果追加写入 ../out/modern_tang.jsonl
"""
import json
import math
import sys
import time
from dataclasses import dataclass, asdict, replace

import torch
import torch.nn as nn
import torch.nn.functional as F


@dataclass
class MCfg:
    V: int = 0
    D: int = 64
    NH: int = 4
    NKV: int = 4            # K/V 头数；< NH 即 GQA，=1 即 MQA
    NL: int = 4
    DFF: int = 256
    T: int = 64
    norm: str = "ln"        # ln | rms
    pos: str = "learned"    # learned | rope
    ffn: str = "relu"       # relu | gelu | swiglu
    dropout: float = 0.0


class RMSNorm(nn.Module):
    """y = x / sqrt(mean(x²) + ε) · γ：去掉了 LayerNorm 的减均值和 β。"""
    def __init__(self, d, eps=1e-5):
        super().__init__()
        self.eps, self.weight = eps, nn.Parameter(torch.ones(d))

    def forward(self, x):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps) * self.weight


def rope_cache(T, dh, base=10000.0):
    inv = 1.0 / base ** (torch.arange(0, dh, 2).float() / dh)      # (dh/2,)
    ang = torch.outer(torch.arange(T).float(), inv)                 # (T, dh/2)：位置 × 频率
    return ang.cos(), ang.sin()


def apply_rope(x, cos, sin):
    """把每个头的向量两两配对，按位置旋转一个角度：q·k 因此只依赖相对位置。"""
    x1, x2 = x[..., 0::2], x[..., 1::2]
    T = x.shape[-2]
    c, s = cos[:T], sin[:T]
    out = torch.stack([x1 * c - x2 * s, x1 * s + x2 * c], dim=-1)
    return out.flatten(-2)


class Attn(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        self.c, self.dh = c, c.D // c.NH
        self.q = nn.Linear(c.D, c.NH * self.dh)
        self.kv = nn.Linear(c.D, 2 * c.NKV * self.dh)   # GQA：K/V 投影更窄
        self.proj = nn.Linear(c.D, c.D)
        if c.pos == "rope":
            cos, sin = rope_cache(c.T, self.dh)
            self.register_buffer("cos", cos, persistent=False)
            self.register_buffer("sin", sin, persistent=False)

    def forward(self, x):
        B, T, D = x.shape
        c, dh = self.c, self.dh
        q = self.q(x).view(B, T, c.NH, dh).transpose(1, 2)
        k, v = self.kv(x).split(c.NKV * dh, dim=2)
        k = k.view(B, T, c.NKV, dh).transpose(1, 2)
```

```
        v = v.view(B, T, c.NKV, dh).transpose(1, 2)
        if c.pos == "rope":
            q, k = apply_rope(q, self.cos, self.sin), apply_rope(k, self.cos, self.sin)
        if c.NKV < c.NH:                                    # 每组 NH/NKV 个 Q 头共享一个 K/V 头
            k = k.repeat_interleave(c.NH // c.NKV, dim=1)
            v = v.repeat_interleave(c.NH // c.NKV, dim=1)
        o = F.scaled_dot_product_attention(q, k, v, is_causal=True,
                                           dropout_p=c.dropout if self.training else 0.0)
        return self.proj(o.transpose(1, 2).contiguous().view(B, T, D))


class FFN(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        self.kind = c.ffn
        if c.ffn == "swiglu":
            h = int(2 * c.DFF / 3) // 8 * 8                 # 三个矩阵，隐藏维取 2/3 使参数量持平
            self.w1, self.w3, self.w2 = nn.Linear(c.D, h), nn.Linear(c.D, h), nn.Linear(h, c.D)
        else:
            self.w1, self.w2 = nn.Linear(c.D, c.DFF), nn.Linear(c.DFF, c.D)

    def forward(self, x):
        if self.kind == "swiglu":
            return self.w2(F.silu(self.w1(x)) * self.w3(x))   # 门控：silu(xW1) ⊙ (xW3)
        act = F.relu if self.kind == "relu" else F.gelu
        return self.w2(act(self.w1(x)))


class Block(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        Norm = RMSNorm if c.norm == "rms" else nn.LayerNorm
        self.n1, self.attn, self.n2, self.ffn = Norm(c.D), Attn(c), Norm(c.D), FFN(c)
        self.drop = nn.Dropout(c.dropout)

    def forward(self, x):
        x = x + self.drop(self.attn(self.n1(x)))
        return x + self.drop(self.ffn(self.n2(x)))


class ModernGPT(nn.Module):
    def __init__(self, c: MCfg):
        super().__init__()
        self.c = c
        self.tok = nn.Embedding(c.V, c.D)
        self.pos = nn.Embedding(c.T, c.D) if c.pos == "learned" else None
        self.blocks = nn.ModuleList([Block(c) for _ in range(c.NL)])
        self.nf = RMSNorm(c.D) if c.norm == "rms" else nn.LayerNorm(c.D)
        self.drop = nn.Dropout(c.dropout)
        for m in self.modules():
            if isinstance(m, (nn.Linear, nn.Embedding)):
                nn.init.normal_(m.weight, std=0.02)
                if getattr(m, "bias", None) is not None:
                    nn.init.zeros_(m.bias)

    def forward(self, idx):
        x = self.tok(idx)
        if self.pos is not None:
            x = x + self.pos(torch.arange(idx.shape[1], device=idx.device))
        x = self.drop(x)
        for b in self.blocks:
            x = b(x)
        return self.nf(x) @ self.tok.weight.T


# ---------------- 数据与训练 ----------------

train_text = open("../out/tang_train.txt", encoding="utf-8").read()
val_text = open("../out/tang_val.txt", encoding="utf-8").read()
chars = sorted(set(train_text + val_text))
stoi = {ch: i for i, ch in enumerate(chars)}
tr_ids = torch.tensor([stoi[ch] for ch in train_text])
va_ids = torch.tensor([stoi[ch] for ch in val_text])
DEV = "mps" if torch.backends.mps.is_available() else "cpu"


@torch.no_grad()
def evaluate(model, ids, T):
    model.eval()
    starts = range(0, min(len(ids) - T, 400 * T), T)        # 取前 400 个不重叠窗口，约 2.6 万 token
```

```
    x = torch.stack([ids[i:i + T] for i in starts]).to(DEV)
    y = torch.stack([ids[i + 1:i + T + 1] for i in starts]).to(DEV)
    tot = 0.0
    for i in range(0, len(x), 64):
        lg = model(x[i:i + 64])
        tot += F.cross_entropy(lg.reshape(-1, model.c.V), y[i:i + 64].reshape(-1), reduction="sum").item()
    model.train()
    return tot / y.numel()


def lr_at(step, steps, lr, sched, warmup=100):
    if sched == "const":
        return lr
    if step < warmup:                                       # 线性 warmup
        return lr * (step + 1) / warmup
    p = (step - warmup) / max(1, steps - warmup)            # 余弦退火到 lr 的 10%
    return lr * (0.1 + 0.9 * 0.5 * (1 + math.cos(math.pi * p)))


def run(name, c: MCfg, steps=3000, B=32, lr=3e-3, sched="const", clip=0.0, seed=1):
    torch.manual_seed(seed)
    c = replace(c, V=len(chars))
    model = ModernGPT(c).to(DEV)
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    gen = torch.Generator().manual_seed(seed)
    hist, ema, t0 = [], None, time.time()
    for step in range(steps):
        for g in opt.param_groups:
            g["lr"] = lr_at(step, steps, lr, sched)
        ix = torch.randint(0, len(tr_ids) - c.T, (B,), generator=gen)
        x = torch.stack([tr_ids[i:i + c.T] for i in ix]).to(DEV)
        y = torch.stack([tr_ids[i + 1:i + c.T + 1] for i in ix]).to(DEV)
        loss = F.cross_entropy(model(x).view(-1, c.V), y.view(-1))
        opt.zero_grad()
        loss.backward()
        gnorm = torch.nn.utils.clip_grad_norm_(model.parameters(), clip if clip > 0 else 1e9).item()
        opt.step()
        ema = loss.item() if ema is None else 0.98 * ema + 0.02 * loss.item()
        if (step + 1) % 250 == 0:
            hist.append({"step": step + 1, "train": ema, "val": evaluate(model, va_ids, c.T),
                         "gnorm": gnorm})
    r = {"name": name, "cfg": asdict(c), "steps": steps, "lr": lr, "sched": sched, "clip": clip,
         "seed": seed, "params": sum(p.numel() for p in model.parameters()),
         "best_val": min(h["val"] for h in hist), "final_val": hist[-1]["val"],
         "final_train": hist[-1]["train"], "secs": time.time() - t0, "hist": hist}
    with open("../out/modern_tang.jsonl", "a") as f:
        f.write(json.dumps(r, ensure_ascii=False) + "\n")
    print(f"{name:14s} params={r['params']:7d} train={r['final_train']:.3f} "
          f"best_val={r['best_val']:.3f} final_val={r['final_val']:.3f} ({r['secs']:.0f}s)", flush=True)
    return r


base = MCfg()
EXPS = {
    "baseline": lambda s: run("baseline", base, seed=s),
    "cosine": lambda s: run("cosine", base, sched="cosine", seed=s),
    "lr3x": lambda s: run("lr3x", base, lr=9e-3, seed=s),
    "lr3x_clip": lambda s: run("lr3x_clip", base, lr=9e-3, clip=1.0, seed=s),
    "dropout0.1": lambda s: run("dropout0.1", replace(base, dropout=0.1), seed=s),
    "dropout0.2": lambda s: run("dropout0.2", replace(base, dropout=0.2), seed=s),
    "gelu": lambda s: run("gelu", replace(base, ffn="gelu"), seed=s),
    "rmsnorm": lambda s: run("rmsnorm", replace(base, norm="rms"), seed=s),
    "rope": lambda s: run("rope", replace(base, pos="rope"), seed=s),
    "swiglu": lambda s: run("swiglu", replace(base, ffn="swiglu"), seed=s),
    "gqa2": lambda s: run("gqa2", replace(base, NKV=2), seed=s),
    "mqa": lambda s: run("mqa", replace(base, NKV=1), seed=s),
    "modern": lambda s: run("modern", replace(base, norm="rms", pos="rope", ffn="swiglu", NKV=2,
                                              dropout=0.1), sched="cosine", seed=s),
    "d32": lambda s: run("d32", replace(base, D=32, DFF=128, NH=2, NKV=2), seed=s),
    "d128": lambda s: run("d128", replace(base, D=128, DFF=512), seed=s),
    "d128_drop": lambda s: run("d128_drop", replace(base, D=128, DFF=512, dropout=0.2), seed=s),
    "d32_6k": lambda s: run("d32_6k", replace(base, D=32, DFF=128, NH=2, NKV=2), steps=6000, seed=s),
    "d64_6k": lambda s: run("d64_6k", base, steps=6000, seed=s),
    "d128_6k": lambda s: run("d128_6k", replace(base, D=128, DFF=512), steps=6000, seed=s),
    "d128_6k_lr1e-3": lambda s: run("d128_6k_lr1e-3", replace(base, D=128, DFF=512), steps=6000, lr=1e-3, seed=s),
    "d64_6k_lr1e-3": lambda s: run("d64_6k_lr1e-3", base, steps=6000, lr=1e-3, seed=s),
}

if __name__ == "__main__":
    torch.set_num_threads(4)
```

```
    names = sys.argv[1:] or list(EXPS)
    print(f"device={DEV}  词表 {len(chars)}  训练 {len(tr_ids)} 字符  验证 {len(va_ids)} 字符", flush=True)
    for n in names:
        EXPS[n](1)
    if not sys.argv[1:]:                                    # 基线再跑两个种子，估计种子噪声
        EXPS["baseline"](2)
        EXPS["baseline"](3)
```

A.17 Python：第 7 章语料预处理（prep_corpus.py）

```
"""把 chinese-poetry 的《全唐诗》前 2 万首（繁体 JSON）转成简体纯文本，按诗划分训练/验证集。

下载：poet.tang.0.json ~ poet.tang.19000.json 放到 ../out/quantang/
python prep_corpus.py  →  ../out/tang_train.txt, ../out/tang_val.txt
"""
import glob
import json
import random
import re

t2s = {}
for line in open("../out/TSCharacters.txt", encoding="utf-8"):
    if line.strip() and not line.startswith("#"):
        trad, simp = line.rstrip("\n").split("\t")
        t2s[trad] = simp.split(" ")[0]          # 一对多时取第一个候选，诗词语境下足够


def to_simp(s):
    return "".join(t2s.get(ch, ch) for ch in s)


poems = []
for path in sorted(glob.glob("../out/quantang/poet.tang.*.json")):
    for p in json.load(open(path, encoding="utf-8")):
        body = "".join(p.get("paragraphs", []))
        body = re.sub(r"[\s□{}\[\]（）()〖〗\-\d]", "", body)   # 去掉空白、缺字符号和校勘注记
        if 10 <= len(body) <= 400:
            poems.append(to_simp(body))

random.Random(0).shuffle(poems)
n_val = len(poems) // 20
val, train = poems[:n_val], poems[n_val:]
open("../out/tang_train.txt", "w", encoding="utf-8").write("\n".join(train) + "\n")
open("../out/tang_val.txt", "w", encoding="utf-8").write("\n".join(val) + "\n")
tr, va = "\n".join(train), "\n".join(val)
trs = set(tr)
print(f"诗 {len(poems)} 首：训练 {len(train)} 首 {len(tr)} 字符，验证 {len(val)} 首 {len(va)} 字符")
print(f"训练集词表 {len(trs)}，验证集中未登录字符占比 {sum(ch not in trs for ch in va) / len(va):.2%}")
print(train[0][:48])
```

A.18 Makefile

```
PY ?= python3

.PHONY: test train-go train-py gradcheck align bugs walk kv speed export gen experiments modern figures

test:           ## M1-M8 全部单元测试
    cd go && go test -v ./...
train-go:       ## Go 版训练 600 步
    cd go && go run . train -log ../out/go_s1.csv
train-py:       ## PyTorch 版训练 600 步
    cd py && $(PY) nanogpt.py --log ../out/py_s1.csv
gradcheck:      ## 全模型梯度检验：正确实现 vs 残差 bug
    cd go && go run . gradcheck -per 50 && go run . gradcheck -per 50 -bug residual
align:          ## 跨实现逐元素对齐
    cd go && go run . align && cd ../py && $(PY) align.py
bugs:           ## 第 6 章两个故意制造的 bug
    cd go && go run . train -bug residual -log ../out/go_residual.csv && go run . train -bug nomask -log ↩
        ../out/go_nomask.csv
```

```
walk:            ## 微型数值走读
    cd go && go run . walk
kv:              ## KV cache 测速
    cd go && go run . kvbench
speed:           ## 并行训练加速比
    cd go && go run . speed
export:          ## PyTorch 训练并导出 Go 可加载的权重
    cd py && $(PY) export.py
gen:             ## Go 加载 PyTorch 权重并生成
    cd go && go run . gen -load ../out/py_weights.json -prompt 月落
experiments:     ## 第 5、6 章 PyTorch 对照实验
    cd py && $(PY) experiments.py
modern:          ## 第 7 章（需先下载全唐诗 JSON，见 7.2 节）
    cd py && $(PY) prep_corpus.py && $(PY) modern.py
figures:         ## 重新生成全部插图
    cd py && $(PY) figures.py
```

附录 B 语料全文

`corpus.txt` 共 20 行，每行一首诗，按下表顺序排列，文件以换行结尾。共 628 个字符（含 20 个换行符），去重后 289 个字符。

SHA-256：`c755662e7a62cb76e6f5c68ca473c36a1bc4551e1ef8c68f89091599ecbe1d53`

行	诗题	作者	体裁
1	静夜思	李白	五言绝句
2	春晓	孟浩然	五言绝句
3	登鹳雀楼	王之涣	五言绝句
4	相思	王维	五言绝句
5	江雪	柳宗元	五言绝句
6	寻隐者不遇	贾岛	五言绝句
7	鹿柴	王维	五言绝句
8	悯农（其二）	李绅	五言绝句
9	枫桥夜泊	张继	七言绝句
10	送元二使安西	王维	七言绝句
11	凉州词	王之涣	七言绝句
12	望庐山瀑布	李白	七言绝句
13	早发白帝城	李白	七言绝句
14	黄鹤楼送孟浩然之广陵	李白	七言绝句
15	望天门山	李白	七言绝句
16	绝句（两个黄鹂鸣翠柳）	杜甫	七言绝句
17	春夜喜雨	杜甫	五言律诗
18	春望	杜甫	五言律诗
19	出塞	王昌龄	七言绝句
20	芙蓉楼送辛渐	王昌龄	七言绝句

```
床前明月光，疑是地上霜。举头望明月，低头思故乡。
春眠不觉晓，处处闻啼鸟。夜来风雨声，花落知多少。
白日依山尽，黄河入海流。欲穷千里目，更上一层楼。
红豆生南国，春来发几枝。愿君多采撷，此物最相思。
千山鸟飞绝，万径人踪灭。孤舟蓑笠翁，独钓寒江雪。
松下问童子，言师采药去。只在此山中，云深不知处。
空山不见人，但闻人语响。返景入深林，复照青苔上。
锄禾日当午，汗滴禾下土。谁知盘中餐，粒粒皆辛苦。
月落乌啼霜满天，江枫渔火对愁眠。姑苏城外寒山寺，夜半钟声到客船。
渭城朝雨浥轻尘，客舍青青柳色新。劝君更尽一杯酒，西出阳关无故人。
黄河远上白云间，一片孤城万仞山。羌笛何须怨杨柳，春风不度玉门关。
日照香炉生紫烟，遥看瀑布挂前川。飞流直下三千尺，疑是银河落九天。
朝辞白帝彩云间，千里江陵一日还。两岸猿声啼不住，轻舟已过万重山。
故人西辞黄鹤楼，烟花三月下扬州。孤帆远影碧空尽，唯见长江天际流。
天门中断楚江开，碧水东流至此回。两岸青山相对出，孤帆一片日边来。
两个黄鹂鸣翠柳，一行白鹭上青天。窗含西岭千秋雪，门泊东吴万里船。
好雨知时节，当春乃发生。随风潜入夜，润物细无声。野径云俱黑，江船火独明。晓看红湿处，花重锦官城。
国破山河在，城春草木深。感时花溅泪，恨别鸟惊心。烽火连三月，家书抵万金。白头搔更短，浑欲不胜簪。
```

秦时明月汉时关，万里长征人未还。但使龙城飞将在，不教胡马度阴山。
寒雨连江夜入吴，平明送客楚山孤。洛阳亲友如相问，一片冰心在玉壶。

附录 C 速查与排错 FAQ

C.1 反向传播速查

前向	反向（已知 dY）	备注
$Y = XW$	$dX = dY\,W^\top$，$dW = X^\top dY$	形状对上就基本推对了
$Y = X + b$（b 按行广播）	$dX = dY$，$db = \mathrm{colsum}(dY)$	广播的反向是求和
$Y = A + B$	$dA = dY$，$dB = dY$	残差：梯度原样复制到两边
Z 被用了两次	$dZ = dZ^{(1)} + dZ^{(2)}$	共享即累加（tying、QKV 共同输入）
$Y = \mathrm{ReLU}(H)$	$dH = dY \odot \mathbf{1}[H > 0]$	
$p = \mathrm{softmax}(z)$（按行）	$dz = p \odot (dp - \langle dp, p\rangle)$	被掩码的位置 $p = 0$，梯度自然为 0
$L = \mathrm{CE}(\mathrm{softmax}(z), y)$，对 T 平均	$dz = (p - \mathrm{onehot}(y))/T$	每行和为 0
$y = \gamma \odot \hat{x} + \beta$（LayerNorm）	$dx = r[g - \bar{g} - \hat{x}\,\overline{g\hat{x}}]$，$g = \gamma \odot dy$	横线表示行内均值
$S = QK^\top/\sqrt{d_h}$	$dQ = dS'\,K$，$dK = dS'^{\top} Q$，$dS' = dS/\sqrt{d_h}$	缩放只乘一次
$X_t = E[x_t]$（查表）	$dE[x_t] \mathrel{+}= dX_t$	scatter-add

C.2 排错 FAQ

现象	最可能的原因	怎么确认
Go 程序 `index out of range`	切片没有 `make`；窗口起点越界	看 panic 的栈；检查 `GetBatch` 的 `Intn` 参数
`MM 形状不匹配`	转置标记写反	对照 3.3 节，用形状把乘法重新拼一遍
初始 loss 远大于 $\ln V$	初始化尺度过大，尤其是 tying 下的嵌入矩阵	打印初始 logits 的标准差，应约为 $\sqrt{d}\,\sigma$
初始 loss 明显小于 $\ln V$	标签泄露：掩码缺失、x 与 y 没有错位	跑因果性测试；检查 y 是否为 x 右移一位
loss 是 NaN	softmax 或交叉熵没有减最大值；学习率过大	打印第一次出现 NaN 之前的 logits 最大值
loss 卡在 $\ln V$ 附近不动	梯度没有写回参数；优化器符号写反；学习率过小	打印一步更新前后某个参数的差
loss 正常下降，但单 batch 过拟合做不到	学习率量级不对；模型容量不足；反向有错	学习率放大和缩小 10 倍各试一次；再做梯度检验
梯度检验个别参数误差在 10^{-3} 附近	差分步长 ε 不合适；ReLU 折点；梯度本身太小	换 $\varepsilon = 10^{-6}$；分母设下限（3.13 节）
梯度检验某一层往下全部误差为 1	那一层的残差或累加写错	看误差从哪个参数开始爆炸（6.5 节）
loss 异常地低，生成却是乱码	掩码缺失，模型在“偷看”答案	因果性测试（6.6 节）
PyTorch 搬到 GPU 后报设备不一致	`forward` 里新建的张量还在 CPU	新建张量时指定 `device=idx.device`

现象	最可能的原因	怎么确认
两套实现结果对不上	权重布局（`nn.Linear` 存的是 $W^\top$）、词表编号顺序、LN 的 ε	用 6.2 节的方法逐个参数比对
生成的字明显不在训练集里	解码映射错位（两个词表编号顺序不同）	`Decode(Encode(text)) == text`

附录 D 第 1 版勘误对照

页码为第 1 版 PDF 的文件页序。

D.1 公式与概念

#	第 1 版位置	问题	第 2 版处理
1	p11 § 3.4.2	LayerNorm 反向公式把 γ_j 提到括号外，括号内的求和却用 dy_k；γ 非均匀时错误	3.6 节重新推导：$dx = r[g - \bar{g} - \hat{x}\,\overline{g\hat{x}}]$，$g = \gamma \odot dy$；`TestLayerNormWrongFormula` 保留反例
2	p12 § 3.4.5	注意力反向的缩放系数 $1/\sqrt{d_h}$ 乘了两次	3.9 节明确只乘一次；`TestAttentionGrad` 验证
3	p14 § 3.7	“没有偏差修正，初期更新会明显偏小”，方向说反	3.14 节改为偏大，给出 3.16 到 6.53 倍的放大系数和关闭修正的对照实验
4	p23 § 4.8	自检问题 2 的答案（“dx 还要参与 FFN 反传”）不成立	4.7 节改为“防御性写法，必须保证的是各加一次”
5	p33 § 6.4	坑二的数字自相矛盾（偏大 2 倍却报 10^{-2} 的误差；“卡在 0.6~0.8”）	6.5 节用真实可复现的 bug 重写，给出梯度检验报告与余弦相似度分析
6	p28、p33	默认初始化导致 loss 偏高的根因归于“假设有归一化和残差的深网”	6.4 节用方差推导和“独立输出头”对照组，定位到 weight tying
7	p13 § 3.4.6	“真实框架里 embedding 反向用稀疏更新”	3.10 节改为默认稠密，`sparse=True` 可选
8	p10 – 12	符号 $\hat{X}$ 同时表示 LN2 输出和 LN 内部标准化值	0.6 节统一符号表：$\tilde{X}$、$\bar{X}$、$\hat{x}$ 分开

D.2 代码

#	第 1 版位置	问题	第 2 版处理
9	p8 § 2.4	`getBatch` 的命名返回值未 `make`，运行即 panic	2.4 节修复，`TestTokenizerAndBatch` 覆盖
10	p8、p28	窗口起点 `Intn(N-T-1)`，最后一个合法起点取不到；顺序遍历公式缺 batch 下标	改为 `Intn(N-T)`，测试确认最后一个窗口能被采到
11	p31 § 5.9	“加一行 `model.cuda()` 零改动加速”会报设备不一致	5.3、5.7 节：统一 `device`，`torch.arange(T, device=idx.device)`
12	p20 – 23	`Forward`、`Backward` 代码截断，右侧溢出	正文按函数完整给出，附录 A 收录全部源码
13	全书	十余个工具函数、Adam 初始化、`main` 等缺失	附录 A
14	全书	语料未给出，579 字符的口径不明	0.4 节与附录 B：给出全文、统计口径与 SHA-256
15	p19 – 22	文字说“前向必须缓存 $\hat{x}$ 和 r”，代码却丢弃缓存、反向时重算	4.5 节统一为缓存，并说明重算即 activation checkpointing
16	全书	Go 与 Python 词表编号规则不同（首次出现 vs 排序），权重不能互换	两边都按码点排序

D.3 数据与表述

#	第 1 版位置	问题	第 2 版处理
17	p4、p32	以“曲线几乎重合”为最强证据，与 §6.4 自相矛盾，数据也不支持	第 6 章改为跨实现逐元素对齐，曲线降为辅助证据，并给出曲线区分不了 bug 的实测
18	p14 vs p25	§3.8 写 Go 版 5.66→0.39，实测为 5.63→0.20	全书数字统一来自第 2 版实测
19	p28 vs p33	默认初始化 step 1 的 loss 一处 21.2、一处 22.64	统一为实测 21.00（5 个种子平均 21.4）
20	p26/30 vs p25	PyTorch 代码量 120 行与 150 行混用	5.1、5.8 节按实际文件给出
21	p15 vs p30	Go 版耗时“几分钟”与“秒级”矛盾	实测 1.40 秒，注明机器
22	p30	图中写 AdamW，代码是 Adam	统一为 Adam
23	p5	26,400 写成“2.7 万”	第 2 版参数量为 26,912
24	p9、p32	“loss 降到 0.3 以下”“300 步后降到 1 以下”与实测不符	改用全窗口平均 loss 与实测步数
25	p9	《静夜思》“22 字”	实验 2-1 给出：25 个字符（含标点与换行）
26	p5 等	系列各册的交叉引用编号不一致	第 2 版去掉了无法核实的册号引用，改为按主题指向本书章节
27	p5、p35	“与千亿参数的 GPT-4 架构同构”“参数量涨了 5 个数量级”“结构一个组件都没多”	1.3 节改为“与 GPT-2 同构”；9.2 节给出 3.7 与 6.6 个数量级；第 7 章列出部件演进
28	p8	“随机起点采样等价于 shuffle”	2.4 节改为“有放回采样，与 shuffle 近似但不等价”
29	p34 §6.5	“单 batch 过拟合做不到就一定有 bug，与环境、运气无关”	6.7 节改为“先排除学习率量级，再怀疑实现”
30	p36 §7.4	“AllReduce ≈ 一致性协议的性能变体”	9.4 节改为集合通信（分片规约再广播）
31	p5	架构图文字被裁切、溢出	图 1-1 重绘